FROM SCRATCH

BUILD A

Reasoning Model

Sebastian Raschka



MEAP

MANNING

Build a Reasoning Model (From Scratch)

1. welcome
2. 1_Understanding_reasoning_models
3. 2_Generating_text_with_a_pre-trained_LLM
4. 3_Evaluating_reasoning_models
5. Appendix_A._References_and_further_reading
6. Appendix_B._Exercise_solutions
7. Appendix_C._Qwen3_LLM_source_code
8. Appendix_F._Common_Approaches_to_Model_Evaluation

OceanofPDF.com

welcome

Thank you for purchasing the MEAP for *Build A Reasoning Model (From Scratch)*.

If you are like most people these days, LLMs are already part of your everyday toolkit. Maybe you have asked an LLM to proofread an email, debug a tricky piece of code, or explain a concept that sent you down an unexpected rabbit hole. Since 2022, and the launch of ChatGPT, these models have moved from experimental novelties to essential tools in our daily work and learning.

It’s been quite a journey to get here. The earliest GPT models, introduced in 2018, could generate text that was more or less human-like, but they were primarily text-completion models, largely unable to answer even simple queries, and the response quality was nowhere near that of the LLMs we use today.

Then came instruction fine-tuning and alignment with human preferences, which ChatGPT popularized in 2022. The techniques behind ChatGPT transformed LLMs into the everyday problem solvers we use today. Currently, we are in the latest phase: developing reasoning models. Reasoning is the ability for an LLM to tackle more complex problems step-by-step.

Reasoning is one of the most exciting and important recent advances in improving LLMs, but it’s also one of the easiest to misunderstand if you only hear the term reasoning and read about it in theory. That’s why this book takes a hands-on approach. We’ll start with a pre-trained base LLM and then add reasoning capabilities ourselves, step by step in code, so you can see exactly how it works.

This book isn’t a “production deployment” manual, and we won’t use any third-party LLM

libraries. Instead, think of it as a behind-the-scenes tour where you get to develop the machinery yourself.

By the end, you will not only understand what reasoning is and how it works, but you will also have built it from scratch. That's a perspective that will serve you well whether you are using, developing, or planning to deploy LLMs in the future.

Please be sure to post any questions, comments, or suggestions you have about the book in the liveBook discussion forum.

— Sebastian Raschka, PhD

In this book

welcome 1 Understanding reasoning models 2 Generating text with a pre-trained LLM 3 Evaluating reasoning models
Appendix A. References and further reading Appendix B. Exercise solutions Appendix C. Qwen3 LLM source code Appendix F. Common Approaches to Model Evaluation

OceanofPDF.com

1 Understanding reasoning models

This chapter covers

- What "reasoning" means specifically in the context of LLMs
- How reasoning differs from pattern matching
- The conventional pre-training and post-training stages of LLMs
- Key approaches to improving reasoning abilities in LLMs
- Why building reasoning models from scratch can improve our understanding of their strengths, limitations, and practical trade-offs

Welcome to the next stage of large language models (LLMs): *reasoning*. LLMs have transformed how we process and generate text, but their success has been largely driven by statistical pattern recognition. However, new advances in reasoning methodologies now enable LLMs to tackle more complex tasks, such as solving logical puzzles and advanced math problems involving multi-step arithmetic. Importantly, reasoning is not just an academic pursuit, but it is also an essential technique for making "agentic" AI practical. Understanding reasoning methodologies is the central focus of this book.

In *Build a Reasoning Model (From Scratch)*, you will learn the inner workings of LLM reasoning methods through a hands-on, code-first approach. We will start from a pre-trained LLM and extend it step by step with reasoning capabilities. We implement these reasoning components ourselves, from scratch, to see how these methods work in practice.

If you are curious about how LLMs themselves are built and trained, my earlier book *Build a Large Language Model (From Scratch)* published by Manning (http://mng.bz/orYv) provides a detailed coverage of these foundations, but it is not required for following along here. By the end of this *Build a Reasoning Model (From Scratch)* book, you will understand how reasoning models work and be equipped to design, prototype, and evaluate the main methods for improving reasoning in LLMs.

With its focus on practical applications and explanations, this book is written to speak to LLM engineers, machine learning researchers, applied scientists, and software developers alike.

This first chapter introduces the foundational concepts before the following chapters shift toward practical, hands-on coding examples to directly implement reasoning techniques for LLMs.

1.1 Defining reasoning in the context of LLMs

What is LLM-based reasoning? The answer and discussion of this question itself would provide enough content to fill a book. However, this would be a different kind of book than this practical, hands-on coding focused book that implements LLM reasoning methods from scratch rather than arguing about reasoning on a conceptual level. Nonetheless, I think it's important to briefly define what we mean by reasoning in the context of LLMs.

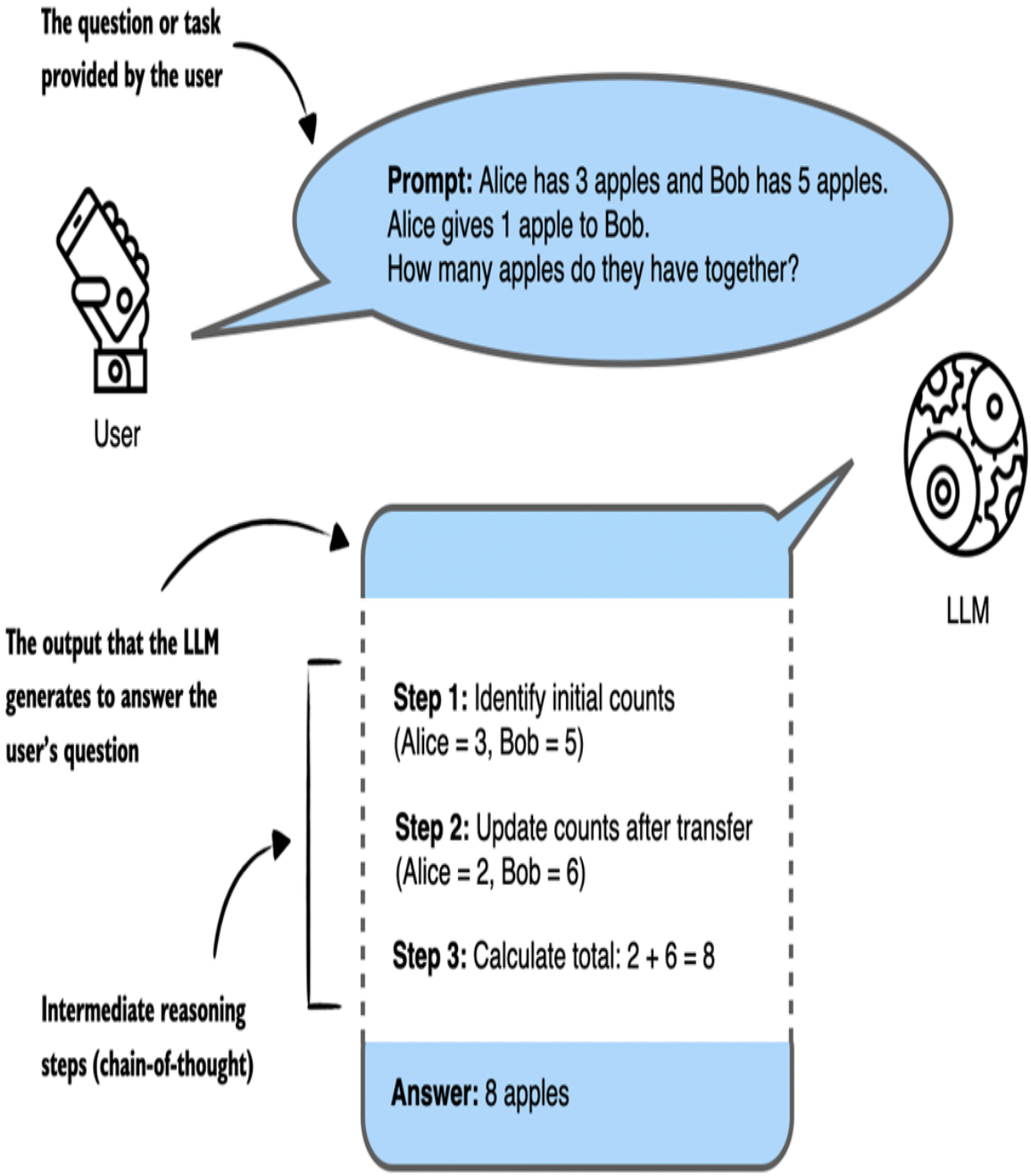
So, before we transition to the coding portions of this book in the upcoming chapters, I want to kick off this book with this section that defines reasoning in the context of LLMs, and briefly explain how it relates to pattern matching and logical reasoning. This will lay the groundwork for further discussions on how LLMs are currently built, how they handle reasoning tasks, and what they are good and not so good at.

This book's definition of reasoning, in the context of LLMs, is as follows:

Reasoning, in the context of LLMs, refers to the model's ability to produce intermediate steps before providing a final answer. This is a process that is often described as chain-of-thought (CoT) reasoning. In CoT reasoning, the LLM explicitly generates a structured sequence of statements or computations that illustrate how it arrives at its conclusion.

Figure 1.1 illustrates a simple example of multi-step (CoT) reasoning in an LLM.

Figure 1.1 A simplified illustration of how an LLM might tackle a multi-step reasoning task. Rather than just recalling a fact, the model needs to combine several intermediate reasoning steps to arrive at the correct conclusion. The intermediate reasoning steps may or may not be shown to the user, depending on the implementation.



LLM-produced intermediate reasoning steps, as shown in figure 1.1, look very much like a person articulating internal thoughts aloud. Yet how closely these methods (and the resulting reasoning processes) mirror human reasoning remains an open research question, one this book does not attempt

to answer. It's not even clear that such a question can be definitively answered.

While figure 1.1 is a typical example of chain-of-thought reasoning, it is important to emphasize that LLM reasoning differs from traditional, deterministic reasoning. For instance, a symbolic logic engine or theorem prover follows strict, rule-based steps that guarantee consistency and correctness. In contrast, an LLM generates reasoning autoregressively, predicting one token at a time based on statistical patterns in its training data. As a result, the LLM's "reasoning steps" are not guaranteed to be logically sound, even if they look convincing.

Instead, this book focuses on explaining and implementing the fundamental techniques that improve LLM-based reasoning and thus make LLMs better at handling complex tasks. My hope is that by gaining hands-on experience with these methods, you will be better prepared to understand and improve those reasoning methods being developed and maybe even explore how they compare to human reasoning.

LLM versus human reasoning

Reasoning processes in LLMs may superficially resemble human thought, particularly in how intermediate steps are articulated. However, it is important to recognize a key difference: humans can engage in deterministic reasoning by deliberately applying rules of logic or by reasoning over an internal model of the world. In contrast, current LLM reasoning is probabilistic, meaning that it generates one token at a time based on statistical patterns in training data, without guarantees of logical consistency.

Humans often reason by consciously manipulating concepts, intuitively understanding abstract relationships, or generalizing from a few examples. LLMs, by contrast, rely on large-scale statistical associations rather than explicit cognitive structures or conscious reflection, as highlighted in a recent study from Apple, The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity (https://machinelearning.apple.com/research/illusion-of-thinking

).

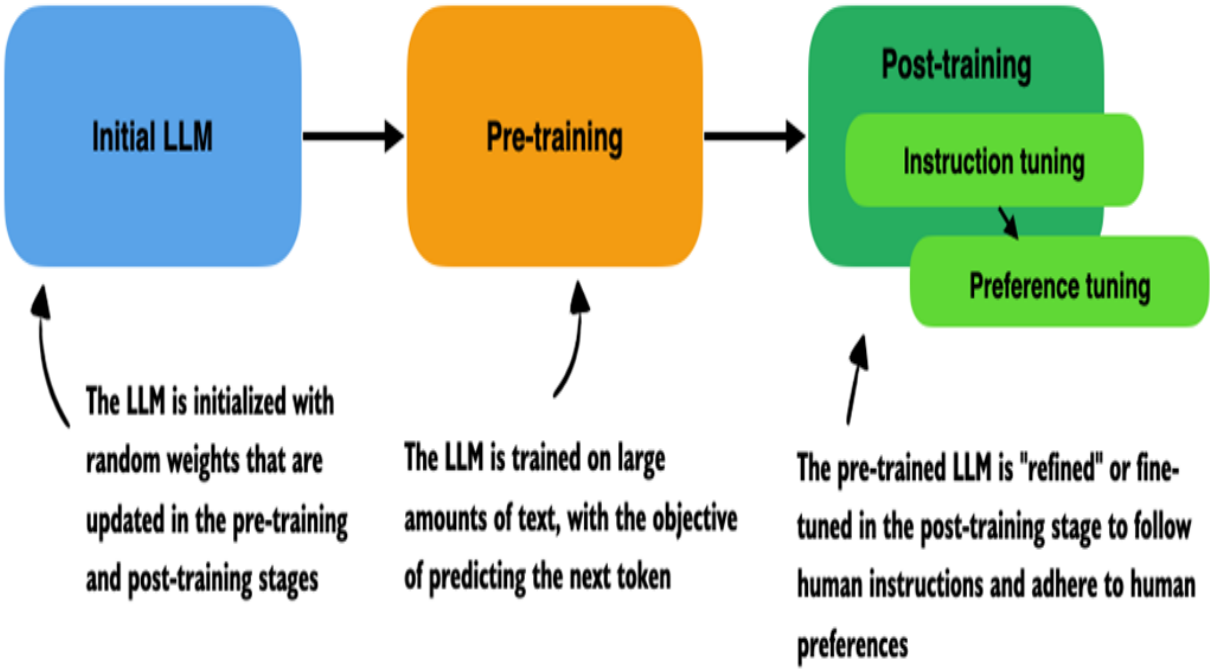
In short, although the outputs of reasoning-enhanced LLMs can appear human-like, the underlying mechanisms differ substantially and remain an active area of exploration.

1.2 Understanding the standard LLM training pipeline

This section briefly summarizes how conventional (non-reasoning) LLMs are typically trained so that we can understand where their limitations lie. This background will also help frame our upcoming discussions on the differences between pattern matching and logical reasoning.

Before applying any specific reasoning methodology, conventional LLM training is usually structured into two stages: *pre-training* and *post-training*, which are illustrated in figure 1.2.

Figure 1.2 Overview of a typical LLM training pipeline. The process begins with an initial model initialized with random weights, followed by pre-training on large-scale text data to learn language patterns by predicting the next token. Post-training then refines the model through instruction fine-tuning and preference fine-tuning, which enables the LLM to follow human instructions better and align with human preferences.In the pre-training stage, LLMs are trained on massive amounts (many terabytes) of unlabeled text, which includes books, websites, research articles, and many other sources. The pre-training objective for the LLM is to learn to predict the next word (or token) in these texts.



In the pre-training stage of a typical LLM training pipeline, as shown in figure 1.2, LLMs are trained on massive amounts (many terabytes) of unlabeled text, which includes books, websites, research articles, and many other sources. The pre-training objective (goal) for the LLM is to learn to predict the next word (i.e., *token*) in these texts.

Words and tokens

A token is a small unit of text that a language model processes. A token can be a full word, part of a word, or even punctuation, depending on how the text is split by a so-called tokenizer.

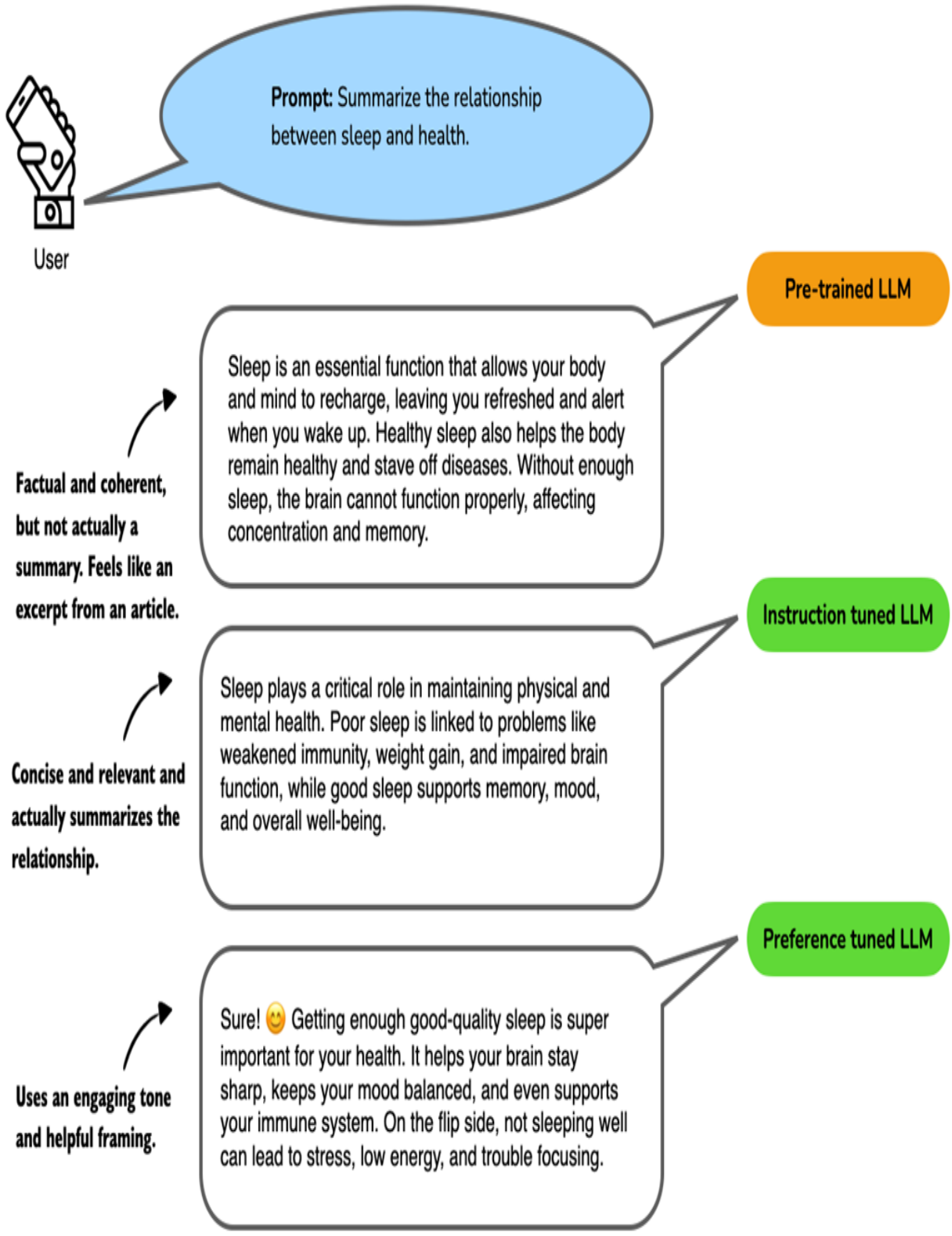
For example, the sentence "An LLM can be useful." might be broken into tokens like "An", " L", "LM", " can", " be", " useful", and "." by a common tokenizer. These tokens are then converted into numerical IDs that the model can ingest.

A tokenizer is a model or tool that is not directly part of the LLM itself but is nonetheless a critical component of the LLM text processing and generation pipeline. We will see how tokenization works in practice in the next chapter.

LLMs become highly capable when pre-trained on massive datasets, which typically involves several terabytes of text (equivalent to around 300 to 400 billion tokens). This training requires thousands of GPUs running for many months and can cost millions of dollars. Here, "capable" means that the LLMs begin to generate text that closely resembles human writing. Also, to some extent, pre-trained LLMs will begin to exhibit so-called emergent properties, which means that they will be able to perform tasks that they were not explicitly trained to do, including translation, code generation, and so on.

However, these pre-trained models merely serve as base models for the post-training stage, which uses two key techniques: *supervised fine-tuning* (often abbreviated as *SFT* in the literature and also known as *instruction tuning*) and *preference tuning* (often implemented via a technique called *Reinforcement Learning with Human Feedback*) to teach LLMs to respond to user queries, which are illustrated in figure 1.3.

Figure 1.3 Example responses from a language model at different training stages. The prompt asks for a summary of the relationship between sleep and health. The pre-trained LLM produces a relevant but unfocused answer without directly following the instructions. The instruction-tuned LLM generates a concise and accurate summary aligned with the prompt. The preference-tuned LLM further improves the response by using a friendly tone and engaging language, which makes the answer more relatable and user-centered.



As shown in figure 1.3, instruction tuning improves an LLM's capabilities of personal assistance-like tasks like question-answering, summarizing and

translating text, and many more. The preferences tuning stage then refines these capabilities. As the term implies, preference tuning helps tailor responses to user preferences. (Some readers may be familiar with terms like *Reinforcement Learning Human Feedback or RLHF*, which are specific techniques to implement preference tuning.)

In short, we can think of pre-training as "raw language prediction" (via next-token prediction) that gives the LLM some basic properties and capabilities to produce coherent texts. The post-training stage then improves the task understanding of LLMs via instruction tuning and refines the LLM to create answers with preferred stylistic choices via preference tuning.

However, it is worth noting that even an instruction-tuned model is not yet a “chatbot.” A chat interface adds another layer that guides the model's responses in an interactive, multi-turn setting. This typically involves a system prompt, conversation history management, and other orchestration (an example of this is implemented in appendix G).

NOTE

These pre-training and post-training stages mentioned above are covered in my book "Build A Large Language Model (From Scratch)" (http://mng.bz/orYv) published by Manning. The book you are reading now does not require detailed knowledge of these stages. Concretely, in the next chapter, we will load a model that has already undergone the expensive pre-training and post-training stages mentioned above, so that we can focus on the methodology that is specific to reasoning models in the subsequent chapters.

1.3 Modeling language through pattern matching

As mentioned in the previous section, during pre-training, LLMs are exposed to vast quantities of text and learn to predict the next token by identifying and reproducing statistical associations in that data. This process enables them to generate fluent and coherent text, but it is fundamentally rooted in surface-level correlations rather than deep understanding.

LLMs respond to prompts by generating continuations that are statistically consistent with the patterns seen during training. In essence, they match patterns between input and output, rather than deducing answers through logical inference.

Consider the following example:

The capital of Germany is…

AI-Prompt End [blank-line]

Response

Berlin.

AI-Response End [blank-line]

An LLM producing the answer "Berlin" is not logically deducing the answer. Instead, it is recalling a strong statistical association learned from training data. This behavior reflects what we refer to as pattern matching, which means that the model completes text based on learned correlations and not by applying structured reasoning steps.

But what about tasks that go beyond pattern recognition, i.e., tasks where a correct answer depends on drawing conclusions from given facts? This brings us to a different kind of capability: logical reasoning.

Logical reasoning involves systematically deriving conclusions from premises using rules or structured inference. Unlike pattern matching, it depends on intermediate reasoning steps and the ability to recognize contradictions or draw implications based on formal relationships.

Consider the following prompt as an example:

- Prompt: "All birds can fly. A penguin is a bird. Can a penguin fly?"

There are two ways to evaluate this.

First, in a closed-world (prompt only) setting, from the two premises in the prompt ("All birds can fly" and "A penguin is a bird"), the deductively valid answer is "Yes, a penguin can fly."

Second, in an open-world (with background knowledge) setting, if we also allow background knowledge not included in the prompt (for example, that penguins cannot fly), this external fact conflicts with the conclusion derived from the premises, as shown in figure 1.4. A reasoning system should detect the inconsistency and either ask for clarification or weaken the first statement (for example, "Most birds can fly, with exceptions such as penguins").

Figure 1.4 Illustration of how contradictory premises lead to a logical inconsistency. From "All birds can fly" and "A penguin is a bird," we infer "Penguin can fly." This conclusion conflicts with the established fact "Penguin cannot fly," which results in a contradiction.

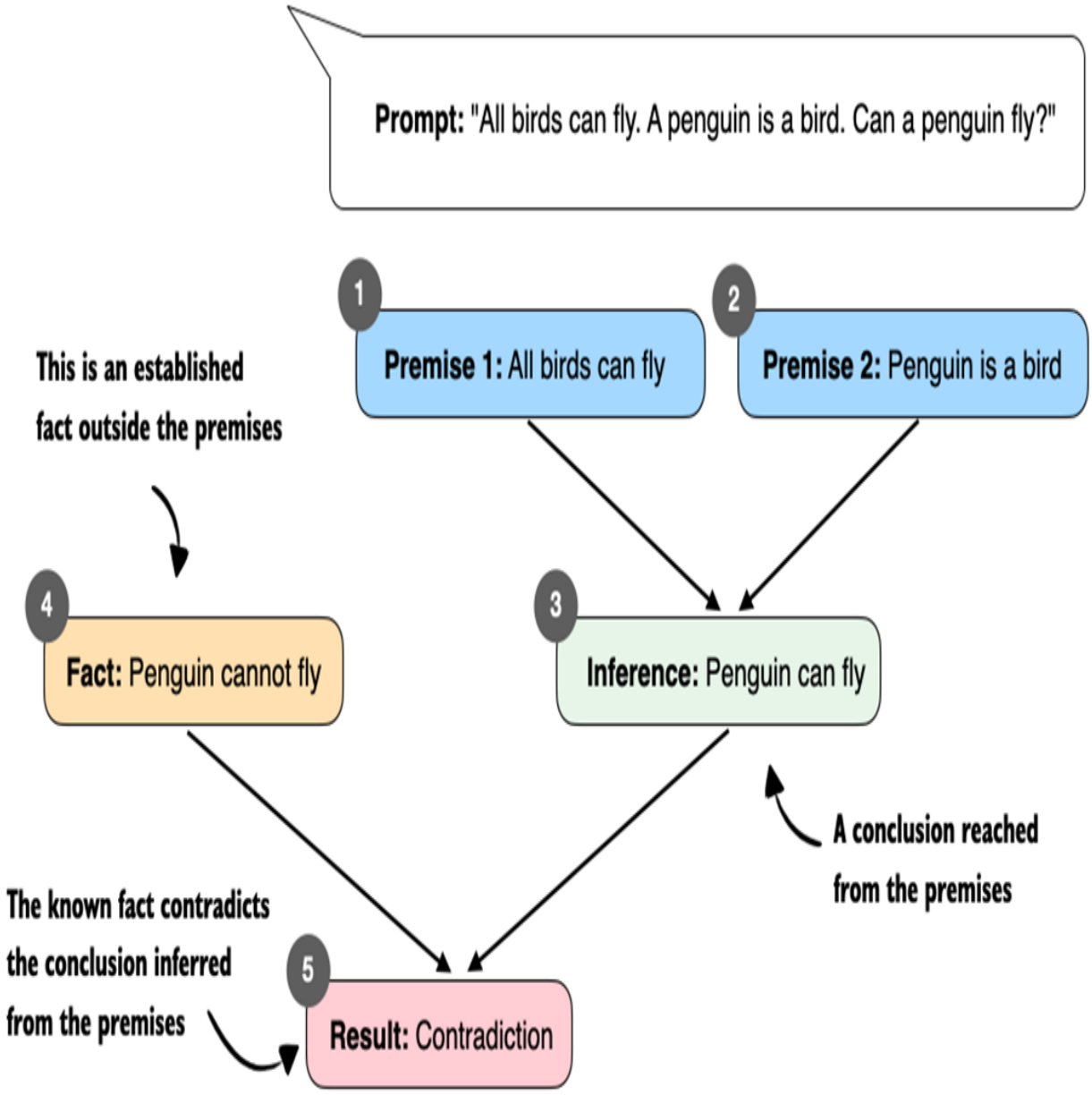


Figure 1.4 shows how a system based on logical reasoning could process the previously introduced "All birds can fly..." prompt.

In contrast, a statistical (pattern-matching) LLM does *not* explicitly track contradictions as shown in figure 1.4 but instead predicts based on learned text distributions. For instance, if information such as "All birds can fly" is reinforced strongly in training data, the model may confidently answer: "Yes, penguins can fly."

In the next section, we will look at a concrete example of how an LLM handles this "All birds can fly..." prompt.

Logical reasoning and rule-based systems

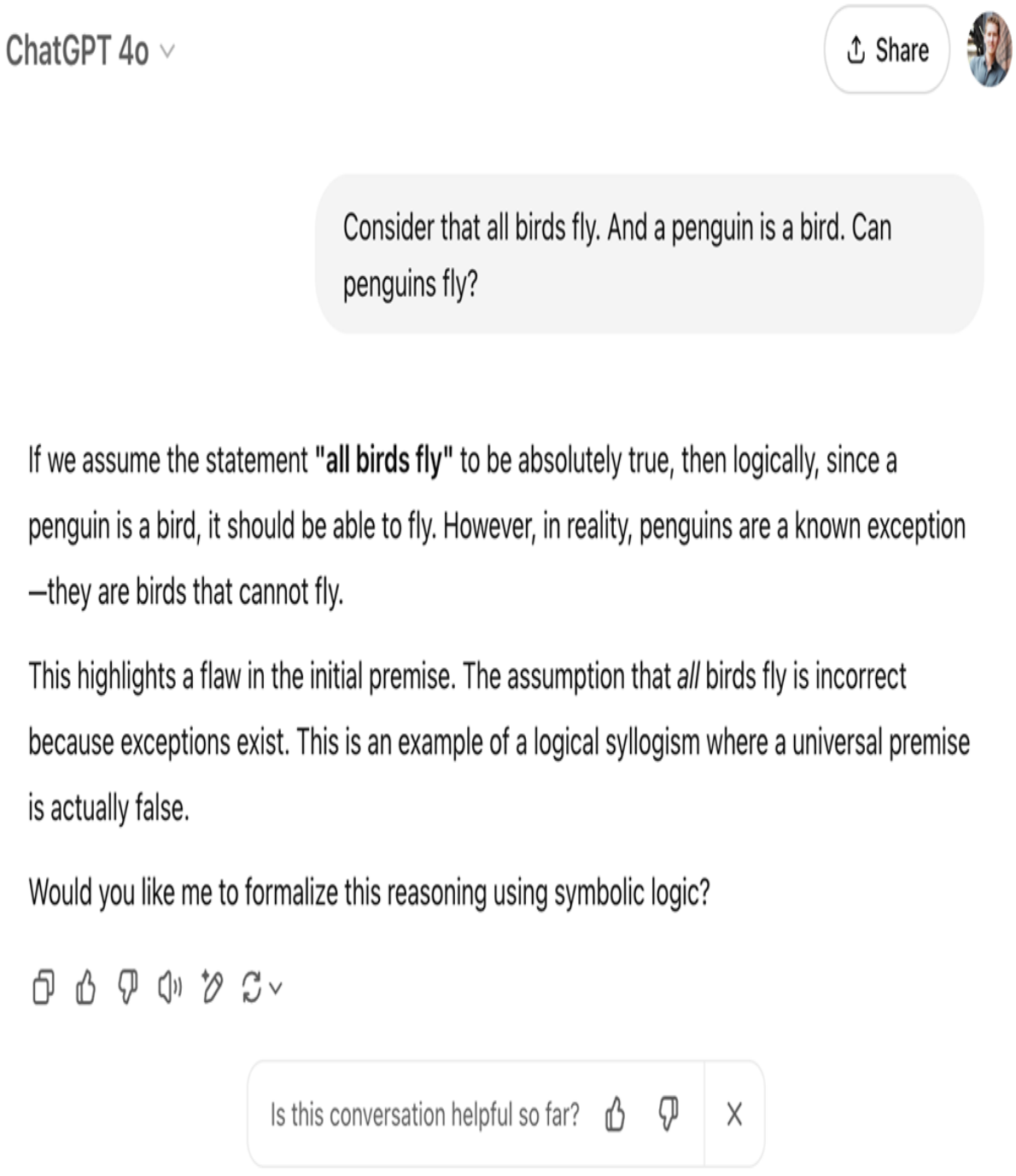
Why are explicit rule-based systems not more popular? Rule-based systems were used widely in the '80s and '90s for medical diagnosis, legal decisions, and engineering. They are still used in critical domains (medicine, law, aerospace), which often require explicit inference and transparent decision processes. However, they are hard to implement as they largely rely on human-crafted heuristics. In contrast, deep neural networks, including LLMs, do not implement hand-written rules; they learn decision patterns from data and can be highly flexible when trained at scale.

1.4 Simulating reasoning without explicit rules

In the previous section, we saw how contradictory premises can lead to logical inconsistencies. A conventional LLM does not explicitly track contradictions but generates responses based on learned text distributions.

Let's see a concrete example, shown in figure 1.5, of how a non-reasoning-enhanced LLM like GPT-4o in OpenAI's ChatGPT responds to the "All birds can fly..." prompt discussed in the previous section.

Figure 1.5 An illustrative example of how a language model (GPT-4o in ChatGPT) appears to "reason" about a contradictory premise.



The example in figure 1.5 shows that GPT-4o appears to answer correctly even though this model is not considered a reasoning model, unlike OpenAI's other offerings like o1, o3, o4-mini, and more recent GPT-5, which have been explicitly developed with reasoning methodology.

So, how did the 4o model generate its answer? Does this mean GPT-4o explicitly reasons logically? No, not necessarily. However, at a minimum 4o is highly effective at simulating logical reasoning in familiar contexts.

GPT-4o does not implement explicit contradiction-checking and instead generates answers based on probability-weighted patterns. This approach works well enough if training data includes many instances correcting the contradiction (e.g., text like "penguins cannot fly") so that the model learns a statistical association between "penguins" and "not flying." As we see in figure 1.5, this allows the model to answer correctly without explicitly implementing rule-based or explicit logical reasoning methodologies.

In other words, the model recognizes the contradiction implicitly because it has frequently encountered this exact reasoning scenario during training. This effectiveness relies heavily on statistical associations built from abundant exposure to reasoning-like patterns in training data.

So, even when a conventional LLM seems to perform logical deduction as shown in figure 1.5, it's not executing explicit, rule-based logic but is instead leveraging patterns from its vast training data.

Nonetheless, GPT-4o's success here is a great illustration of how powerful implicit pattern matching can become when trained at a massive scale. However, these types of pattern-based reasoning models usually struggle in scenarios where:

- The logical scenario is novel (not previously encountered in training data).
- Reasoning complexity is high, involving intricate, multi-step logical relationships.
- Structured reasoning is required, and no direct prior exposure to similar reasoning patterns exists in training data.

Logical reasoning and current reasoning LLM offerings

While GPT-4o is not officially labeled as a reasoning model, OpenAI offers several dedicated reasoning models, including o1, o3, o4-mini, and GPT-5. Moreover, other companies have been developing LLMs with explicit

reasoning capabilities. As of this writing, popular examples include Anthropic's Claude 4, xAI's Grok 4, Google's Gemini 2.5, DeepSeek's R1, Alibaba's Qwen3, and many more. The techniques employed by these models are the focus of this book. As we will see, this is achieved without implementing a rule-based reasoning pipeline (figure 1.4 illustrates the general idea of rule-based reasoning). Instead, the LLM learns or improves its reasoning capabilities as a result of the modified inference and training methodologies.

We might say that LLMs simulate logical reasoning through learned patterns, and we can improve it further with specific reasoning methods that include inference-compute scaling and post-training strategies, but they are not explicitly executing any rule-based logic internally.

Moreover, it's worth mentioning that reasoning in LLMs exists on a spectrum. This means that even before the advent of dedicated reasoning models such as OpenAI's o1 and DeepSeek-R1, LLMs were capable of simulating reasoning behavior. For instance, these models exhibited behaviors aligning with our earlier definition, such as generating intermediate steps to arrive at correct conclusions. What we now explicitly label a "reasoning model" is essentially a more refined version of this capability. And these improved reasoning capabilities are achieved by leveraging specific inference-compute scaling techniques and targeted post-training methods, as explained in the next section, which are designed to improve and reinforce reasoning behavior.

The rest of this book focuses specifically on these advanced methods that improve LLMs to solve complex problems, helping you better understand how to improve the implicit reasoning capabilities in LLMs.

1.5 Improving reasoning with training and inference techniques

Reasoning in the context of LLMs became popular in the public eye with the announcement of OpenAI's o1 in ChatGPT on September 12, 2024, which popularized the concept of reasoning in LLMs. In the announcement article (https://openai.com/index/introducing-openai-o1-preview/), OpenAI

mentioned that "We've developed a new series of AI models designed to spend more time thinking before they respond."

Furthermore, OpenAI wrote: "These enhanced reasoning capabilities may be particularly useful if you're tackling complex problems in science, coding, math, and similar fields."

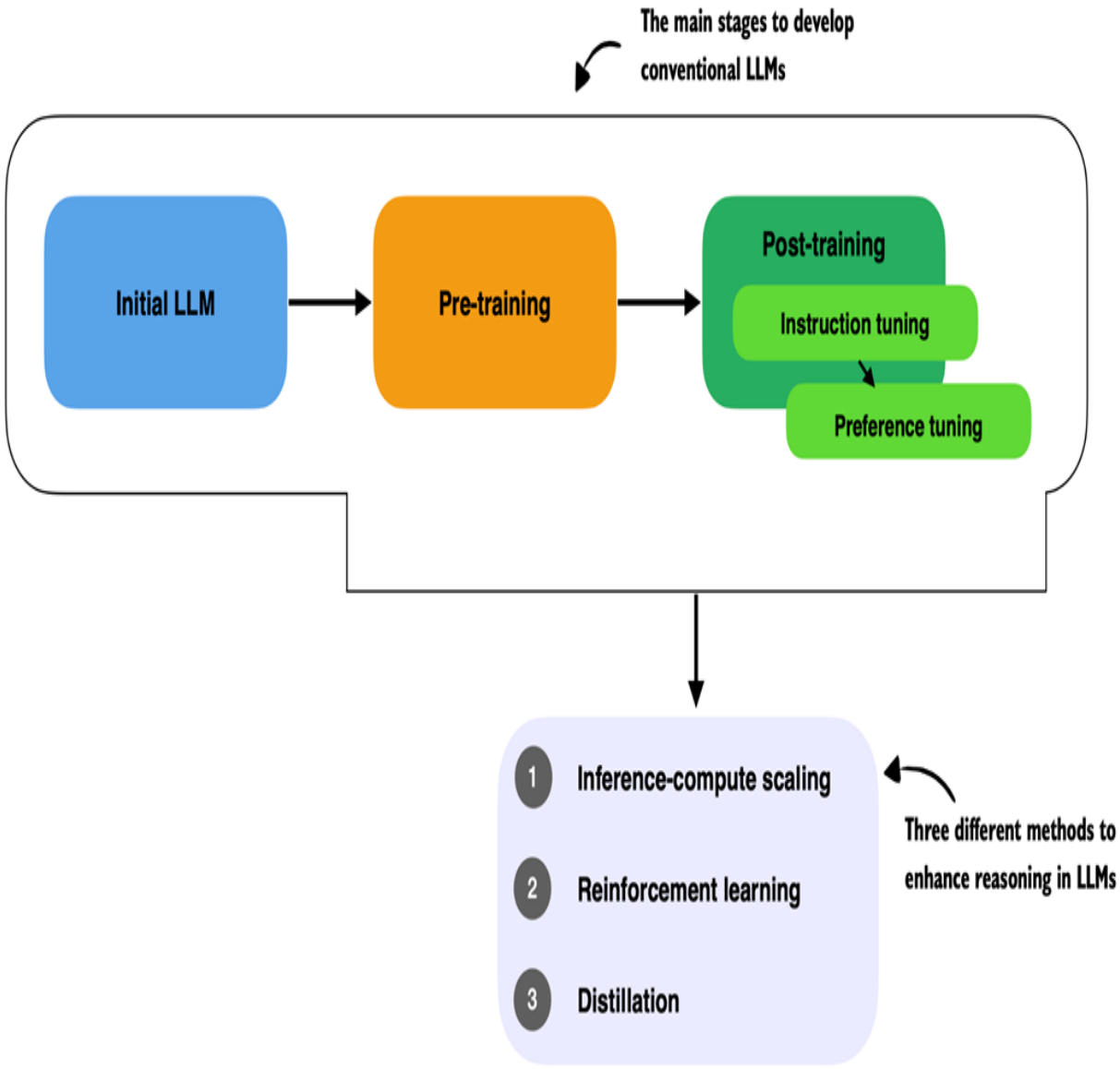
While the training and implementation details of OpenAI's o1 are not publicly available, the common perception is that the o1 model is based on one of the predecessors, like GPT-4, but uses extensive inference-compute scaling (more on that later in this section) to achieve these enhanced reasoning capabilities.

A few months later, in January 2025, DeepSeek released the DeepSeek-R1 model and technical report (https://arxiv.org/abs/2501.12948), which details training methodologies to develop reasoning models, which made big waves as they not only made freely and openly available a model that competes with and exceeds the performance of the proprietary o1 model but also shared a blueprint on how to train such model.

This book aims to explain how these methodologies used to develop reasoning models work by implementing similar methods from scratch.

The different approaches to developing and improving an LLM's reasoning capabilities can be grouped into three broad categories, as illustrated in figure 1.6.

Figure 1.6 Three approaches commonly used to improve reasoning capabilities in LLM). These methods (inference-compute scaling, reinforcement learning, and distillation) are typically applied after the conventional training stages (initial model training, pre-training, and post-training with instruction and preference tuning).



As illustrated in figure 1.6, these methods are applied to LLMs that have undergone the conventional pre-training and post-training phases, including instruction and preference tuning.

1. **Inference-time compute scaling.** *Inference-time compute scaling* (also often called *inference compute scaling*, *test-time scaling*, or other variations) includes methods that improve model reasoning capabilities at inference time (when a user prompts the model) without training or modifying the underlying model weights. The core idea is to trade off increased computational resources for improved performance, which helps make even fixed models more capable through techniques such as

chain-of-thought reasoning, and various sampling procedures. This topic will be the focus of chapter 4.

2. **Reinforcement learning.** *Reinforcement learning* (*RL*) refers to training methods that improve a model's reasoning capabilities by encouraging it to take actions that lead to high reward signals. These rewards can be broad, such as task success or heuristic scores, or they can be narrowly defined and verifiable, such as correct answers in math problems or coding tasks.
   Unlike scaling compute at inference time, which can improve reasoning performance without modifying the model, RL updates the model's weights during training. This enables the model to learn and refine reasoning strategies through trial and error, based on the feedback it receives from the environment. We will explore RL in more detail in chapter 5.

Reinforcement learning for reasoning and preference tuning

In the context of developing reasoning models, it is important to distinguish the RL approach here from reinforcement learning with human feedback (RLHF), which is used during preference tuning when developing a conventional LLM as illustrated previously in figure 1.2.

Both settings use the same underlying process (RL) but they differ primarily in how the reward is obtained and validated (human judgments for RLHF versus automated verifiers or environments for reasoning RL).

RLHF incorporates explicit human evaluations or rankings of model outputs as reward signals, directly guiding the model toward human-preferred behaviors. In contrast, RL in the context of reasoning models typically relies on automated or environment-based reward signals, which can be more objective but potentially less aligned with human preferences. For instance, RL in a reasoning model development pipeline might train a model to excel at mathematical proofs by providing explicit rewards for correctness. In contrast, RLHF would involve human evaluators ranking various responses to encourage outputs that align closely with human standards and subjective preferences.

3. **Supervised fine-tuning and model distillation.** *Distillation* involves transferring complex reasoning patterns learned by powerful, larger models into smaller or more efficient models. Within the context of LLMs, this typically means performing *supervised fine-tuning* (*SFT*) using high-quality labeled instruction datasets generated by a larger, more capable model. This technique is commonly referred to as *knowledge distillation* or simply *distillation* in LLM literature. However, it's important to note that this differs slightly from traditional knowledge distillation in deep learning, where a smaller ("student") model typically learns from both the outputs and the logits produced by a larger ("teacher") model. This topic is discussed further in Chapter 6.

1.6 Why build reasoning models from scratch?

Following the release of DeepSeek-R1 in January 2025, improving the reasoning abilities of LLMs has become one of the hottest topics in AI, and for good reason. Stronger reasoning skills allow LLMs to tackle more complex problems, making them more capable across various tasks users care about.

This shift is also reflected in a February 12, 2025, statement from OpenAI's CEO:

> "We will next ship GPT-4.5, the model we called Orion internally, as our last non-chain-of-thought model. After that, a top goal for us is to unify o-series models and GPT-series models by creating systems that can use all our tools, know when to think for a long time or not, and generally be useful for a very wide range of tasks."

The quote above underlines the major shift from leading LLM providers towards reasoning models, where "chain-of-thought" refers to a prompting technique that guides language models to reason step-by-step to improve their reasoning capabilities, which we will cover in more detail in Chapter 4.

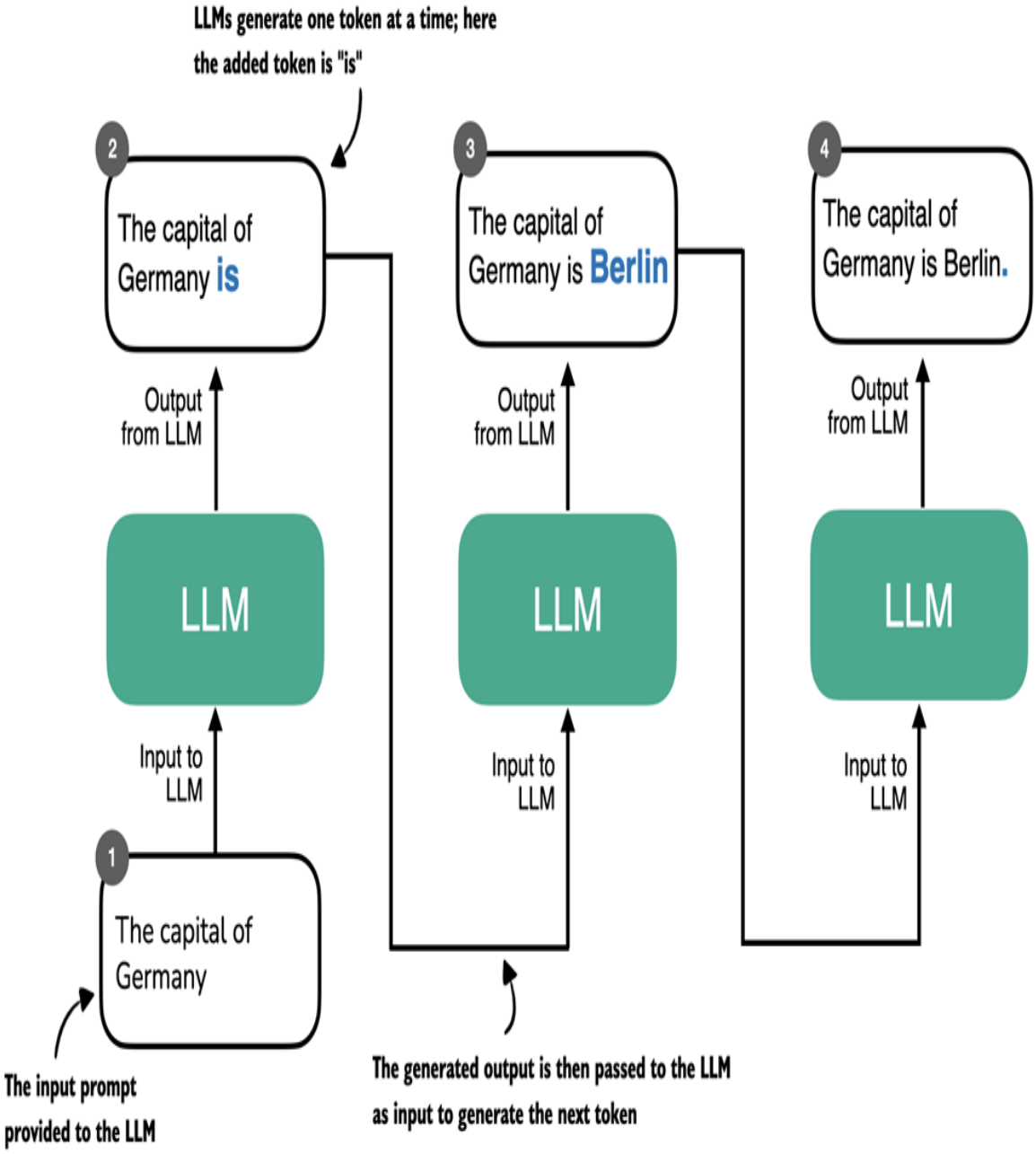
Also noteworthy is the mention of knowing "when to think for a long time or not." This hints at an important design consideration: reasoning is not always necessary or desirable

For instance, reasoning models are designed to be good at complex tasks such as solving puzzles, advanced math problems, and challenging coding tasks. However, they are not necessary for simpler tasks like summarization, translation, or knowledge-based question answering. In fact, using reasoning models for everything can be inefficient and expensive. For instance, reasoning models are typically more expensive to use, more verbose, and sometimes more prone to errors due to "overthinking." Also, here, the simple rule applies: Use the right tool (or type of LLM) for the task.

Reasoning models are often more expensive than non-reasoning models for two reasons.

First, they tend to produce longer outputs because they include intermediate steps that explain how an answer is derived. As figure 1.7 illustrates, LLMs generate text one token at a time and each token requires a full forward pass. If a reasoning model's answer is twice as long, generation involves roughly twice as many forward passes, which increases compute costs.

Figure 1.7 Token-by-token generation in an LLM. At each step, the LLM takes the full sequence generated so far and predicts the next token, which may represent a word, subword, or punctuation mark depending on the tokenizer. The newly generated token is appended to the sequence and used as input for the next step. This iterative decoding process is used in both standard language models and reasoning-focused models.



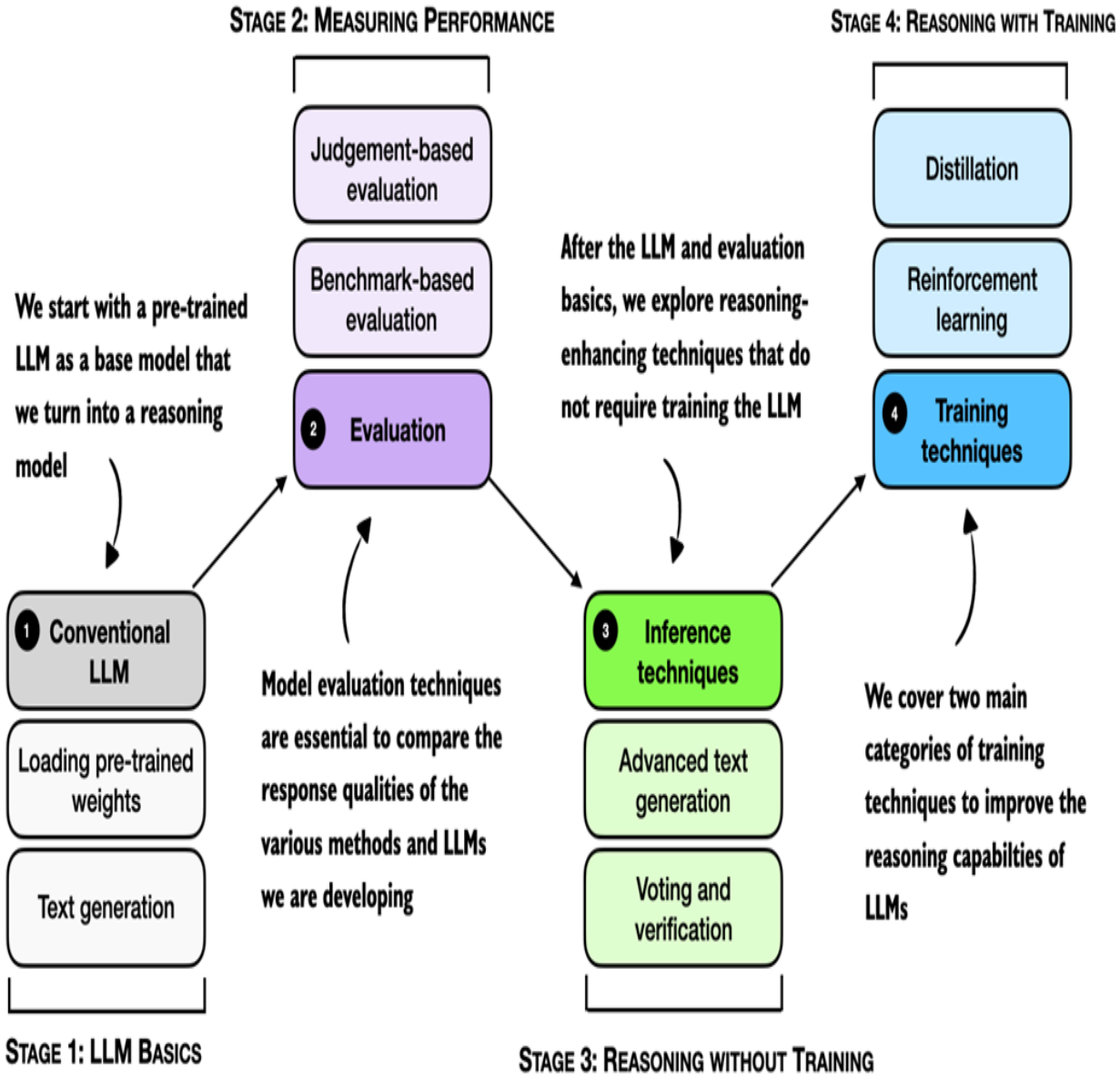
Second, many reasoning workflows involve multiple inference calls for a single task, for example sampling several candidate solutions, calling tools, or running a verifier. These additional calls multiply the total number of tokens processed and further increase cost beyond the single-call behavior shown in figure 1.7.

This directly highlights the importance of implementing LLMs and reasoning methods from scratch. It's one of the best ways to understand how they work. And if we understand how LLMs and these reasoning models work, we can better understand these trade-offs.

1.7 A roadmap to reasoning models from scratch

Now that we have discussed reasoning in LLMs from a bird's-eye view, the subsequent chapters will guide you through the process of coding and applying reasoning methods from scratch. We will tackle this in multiple stages, as outlined in figure 1.8.

Figure 1.8 A mental model of the main reasoning model development stages covered in this book. We start with a conventional LLM as base model (stage 1). In stage 2, we cover evaluation strategies to track the reasoning improvements introduced via the reasoning methods in stages 3 and 4.



As shown in figure 1.8, we cover the reasoning model development in several stages. In stage 1 (next chapter), we load a conventional LLM that has already undergone the basic pre-training and instruction fine-tuning stages. Then, in stage 2, we cover common methods for evaluating LLMs and reasoning capabilities, so that we can measure the improvements we make when we apply reasoning-enhancing methods in stages 3 and 4.

Stage 3 covers inference techniques that can improve the response quality and reasoning behavior of LLMs. Note that these techniques can be applied to improve any LLM, conventional LLMs and LLMs that have been trained

as reasoning models. Stage 4 will introduce training methods to develop reasoning models.

I hope you are as enthusiastic as I am about the journey ahead!

1.8 Summary

- Conventional LLM training occurs in several stages:
  - Pre-training, where the model learns language patterns from vast amounts of text.
  - Instruction fine-tuning, which improves the model's responses to user prompts.
  - Preference tuning, which aligns model outputs with human preferences.
- Reasoning methods are applied on top of a conventional LLM.
- Reasoning in LLMs involves systematically solving multi-step tasks using intermediate steps (chain-of-thought).
- Reasoning in LLMs is different from rule-based reasoning and it also likely works differently than human reasoning; currently, the common consensus is that reasoning in LLMs relies on statistical pattern matching.
- Pattern matching in LLMs relies purely on statistical associations learned from data, which enables fluent text generation but lacks explicit logical inference.
- Improving reasoning in LLMs can be achieved through:
  - Inference-time compute scaling, enhancing reasoning without retraining (e.g., chain-of-thought prompting).
  - Reinforcement learning, training models explicitly with reward signals.
  - Supervised fine-tuning and distillation, using examples from stronger reasoning models.
- Building reasoning models from scratch provides practical insights into LLM capabilities, limitations, and computational trade-offs.

OceanofPDF.com

2 Generating text with a pre-trained LLM

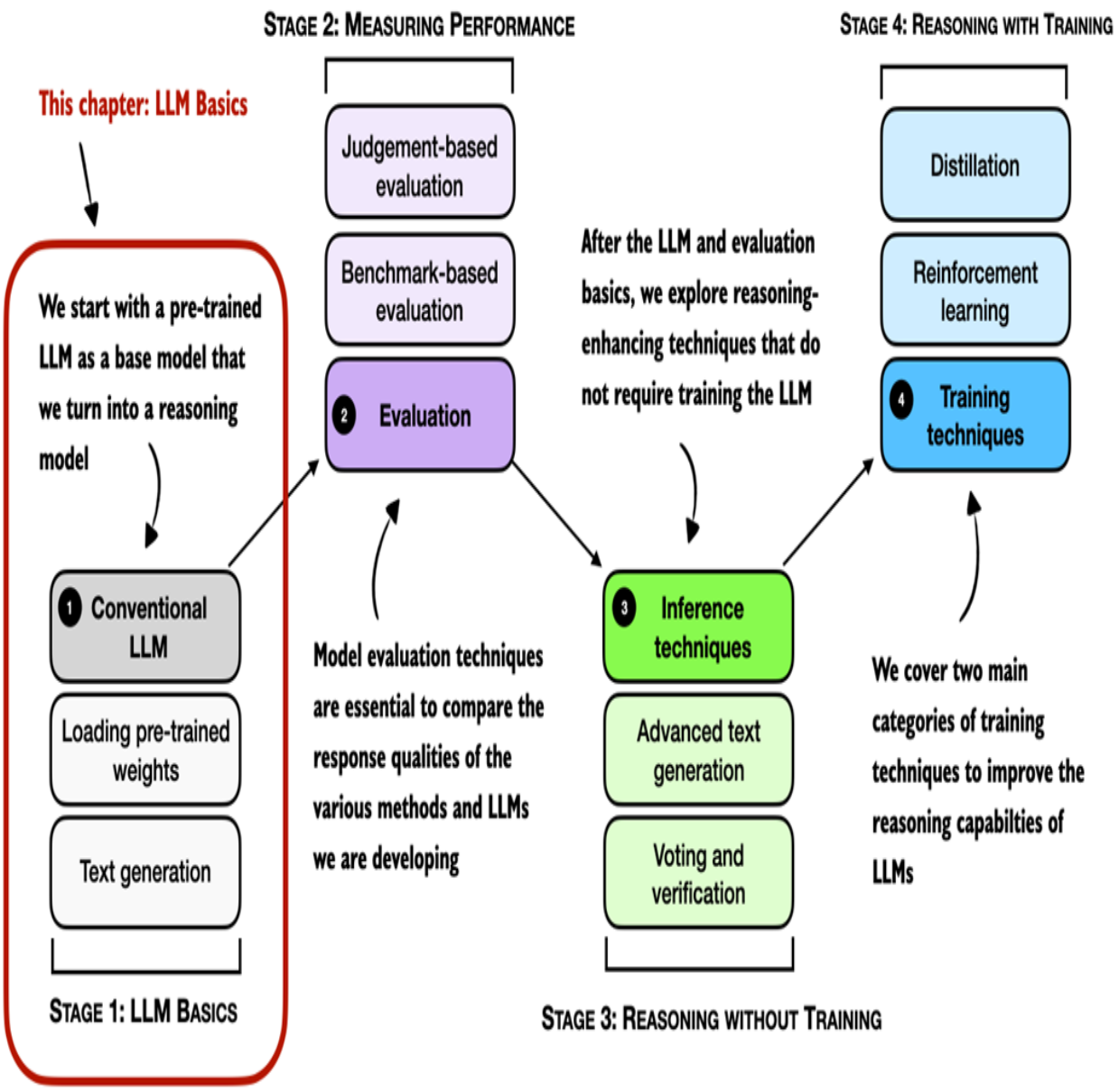
This chapter covers

- Setting up the code environment for working with LLMs
- How to use a tokenizer to prepare input text for an LLM
- The step-by-step process of text generation using a pre-trained LLM
- Caching and compilation techniques for speeding up LLM text generation

In the previous chapter, we discussed the difference between *conventional large language models* (*LLMs*) and *reasoning models*. Also, we introduced several techniques to improve the reasoning capabilities of LLMs. These reasoning techniques are usually applied on top of a conventional (base) LLM.

In this chapter, we will lay the groundwork for the upcoming chapters and load such a conventional LLM on top of which we can apply reasoning techniques in subsequent chapters, as illustrated in figure 1. This conventional LLM is an LLM that has already been pre-trained to generate general texts (but it has not been specifically trained or enhanced for reasoning).

Figure 2.1 A mental model depicting the four main stages of developing a reasoning model. This chapter focuses on stage 1, loading a conventional LLM and implementing the text generation functionality.



In addition to setting up the coding environment and loading a pre-trained LLM, you will learn how to use a *tokenizer* to prepare text input for the model. As illustrated in figure 2.1, you will also implement a text generation function, enabling practical use of the LLM to generate text. This functionality will be used and further improved in later chapters.

2.1 Introduction to LLMs for text generation

In this chapter, we implement all the necessary LLM essentials, from setting up our coding environment and loading a pre-trained LLM to generating text that we will reuse and build upon in this book. In this sense, this chapter can be understood as a setup chapter.

This LLM will be capable of following basic instructions and generating coherent text, as illustrated in figure 2.2.

Figure 2.2 An overview depicting an LLM generating a response (output text) given a user query (input text)

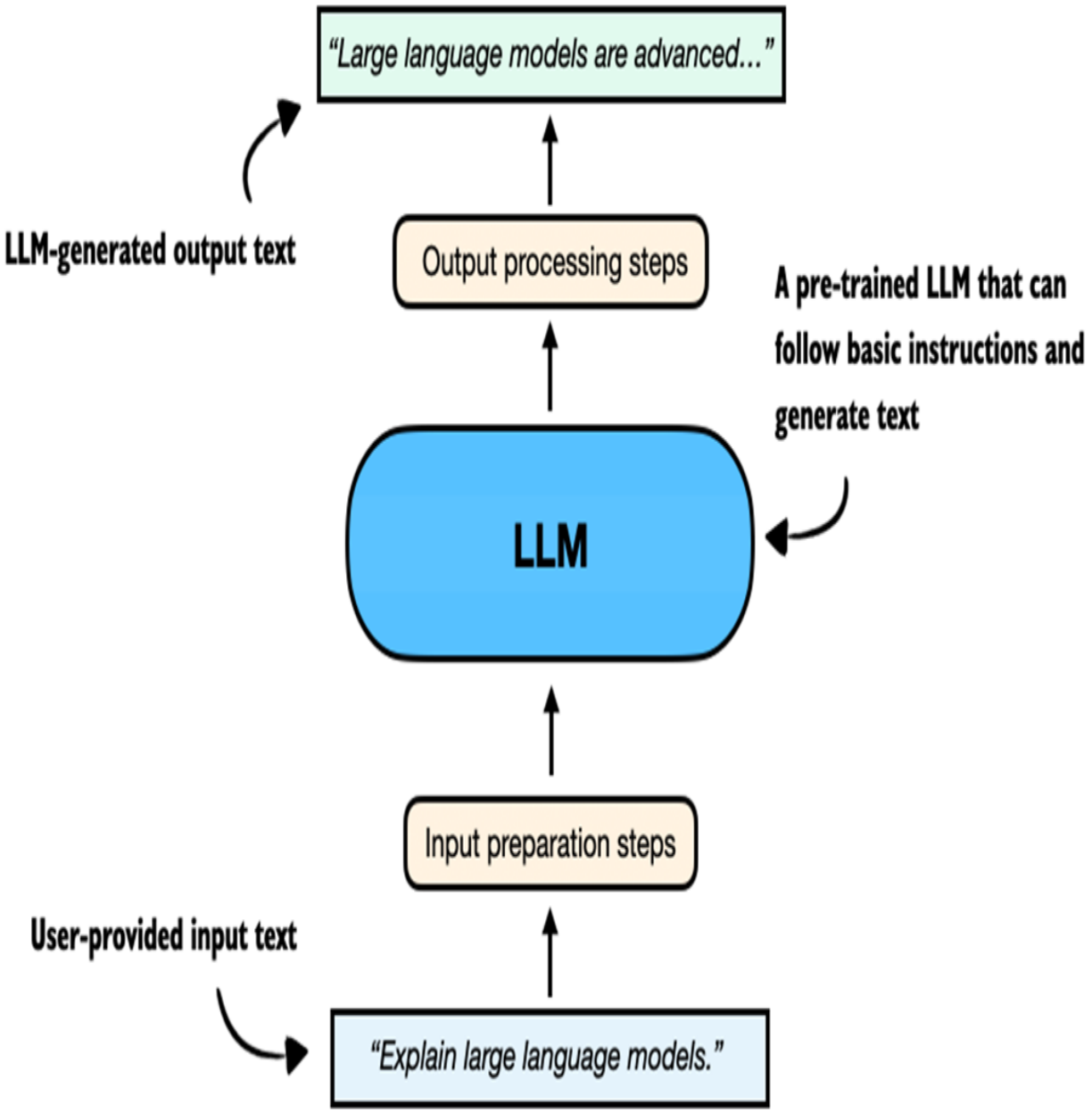


Figure 2.2 summarizes the components of an LLM text generation pipeline, and we will discuss and implement these steps in more detail later in this chapter.

Note

By convention, diagrams involving neural networks such as LLMs are drawn and read vertically from bottom (inputs) to top (outputs). Arrows

indicate the flow of information upward through the model.

If you have not coded an LLM or used LLMs programmatically before, this chapter will teach you how the *text generation* process works. However, in this chapter, we will not go deep into the internals of an LLM, such as the attention mechanism and other architecture components; this is the topic of my other book, *Build a Large Language Model (from Scratch)*. Note that understanding these internals are not required for this book, and, if you are curious, you can learn about them after you finish reading this book.

Before we begin implementing the components shown in figure 2.2, including input preparation, loading the LLM, and generating text, we first need to set up our coding environment. This is the focus of the next section.

2.2 Setting up the coding environment

This section provides instructions and recommendations for setting up your Python coding environment to follow along with the examples in this book. I recommend reading this section in its entirety before deciding which way is for you.

If you are reading this book, you have probably coded in Python before. In this case, the simplest way to install dependencies, if you already have a Python environment set up (with Python 3.10 or newer), is to use Python's package installer (`pip`) in your terminal.

If you have downloaded the code from the publisher's website, use the `requirements.txt` file to install the required Python libraries used throughout this book:

```
pip install -r requirements.txt
```

Alternatively, to install the required packages directly without downloading the `requirements.txt` file, use:

```
pip install -r https://raw.githubusercontent.com/\
rasbt/reasoning-from-scratch/refs/heads/main/requirements.txt
```

Python packages used in this chapter

If you prefer to install only the packages used in this chapter, you can do this with the following command:

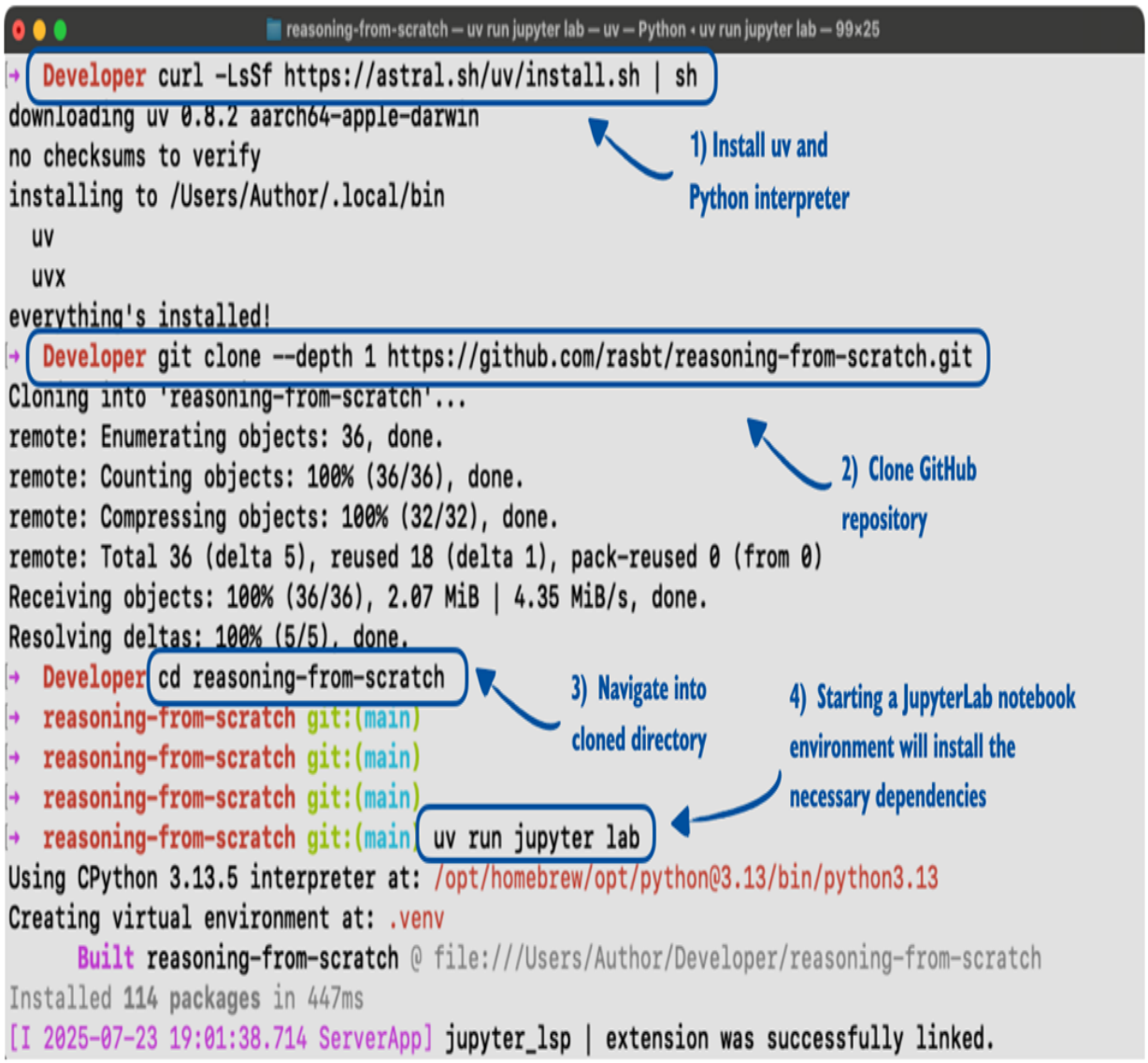
```
pip install torch>=2.7.1 tokenizers>=0.21.2 reasoning-from-scratch
```

- `torch` refers to PyTorch, a widely used deep learning library that provides tools for building and training neural networks.
- `tokenizers` is a library that provides efficient tokenization algorithms, used to prepare input data for LLMs.
- `reasoning-from-scratch` is a custom library that I developed for this book. It includes all the code examples implemented throughout the chapters, along with additional utility functions we will be using.

While `pip` is the canonical way to install Python packages, my preferred way to use Python is via the widely recommended `uv` Python package and project manager instead. It comes with its own Python executable, so it's also a great option if you don't have Python installed on your system, yet.

Figure 2.3 outlines the 4-step process from installing uv to getting ready to execute the code in this chapter, which we will cover in the remainder of this section.

Figure 2.3 Installing and using the `uv` Python package and project manager via the macOS terminal



Note that figure 2.3 steps through the uv installation and usage on a macOS terminal, but uv is supported by Linux and Windows as well.

1) To install uv, run the installation for your OS from the official website: https://docs.astral.sh/uv/getting-started/installation/

2) Next, clone the GitHub repo:

```
git clone --depth 1 https://github.com/rasbt/reasoning-from-scratch.git
```

Here, the `--depth 1` option tells `git` to perform a shallow clone, which means it only downloads the latest version of the code without the full version history. This makes the download faster and uses less space.

If you don't have `git` installed, you can also manually download the source code repository from the publisher's website or by opening this link in your browser: https://github.com/rasbt/reasoning-from-scratch/archive/refs/heads/main.zip (unzip it after downloading).

3) Next, in the terminal, navigate to the `reasoning-from-scratch` folder.

4) Inside the `reasoning-from-scratch` folder, execute:

```
uv run jupyter lab
```

The command above will launch JupyterLab, where you can open a blank Jupyter notebook to type and execute code or open the chapter 2 notebook that contains all the code covered in this chapter.

Tip

Python script files can be executed via `uv run script-name.py`.

The above `uv run...` command also sets up a local virtual environment (usually inside an invisible `.venv/` folder) and installs all dependencies from the `pyproject.toml` file inside the `reasoning-from-scratch` folder automatically. So, the manual installation of code dependencies via the requirements file is not needed. However, if you plan to install additional packages, you can use the following command:

```
uv add packagename
```

The supplementary code repository contains additional installation instructions and details inside the `ch02` subfolder if needed.

2.3 Understanding hardware needs and recommendations

You may have heard that training LLMs is very expensive. For leading LLM companies, it is not uncommon to spend anywhere between 1-10 million Dollars on the small end and >50 million Dollars on the high end in terms of compute costs to train a new base model LLM before even adding any reasoning techniques.

This high price tag would make the development of an LLM unfeasible for me and most readers. So, we are going to use a relatively small (but capable) pre-trained LLM on top of which we implement reasoning techniques.

Note that this smaller LLM is a scaled-down version that otherwise follows the same architecture as contemporary state-of-the-art models. And the reasoning methods that we will apply are the same as those used by larger LLMs. The difference is that the smaller LLM allows us to explore these methods in a budget-friendly way.

As an analogy, imagine you are curious to learn how cars work. If you are new to cars, as a learning exercise, you probably wouldn't start out building an expensive Ferrari right away. Instead, you would, for example, create a smaller car like a Volkswagen Beetle to start with, which still teaches you a lot about how engines and the transmission work. On the contrary, I would even say that working on a smaller car helps you *better* understand how the engine and transmission work because it gets complicated refinements and other details out of the way.

However, while we will use a relatively small model for these educational purposes in this book, the usage, development, and application of the reasoning techniques are still computationally intensive, and later chapters, such as chapters 5-7, will benefit from using a GPU.

If you followed the previous section, you should have PyTorch installed, which you can use to see if your computer has a PyTorch-supported GPU by executing the following PyTorch code in Python:

```
import torch

print(f"PyTorch version {torch.__version__}")
if torch.cuda.is_available():
    print("CUDA GPU")
elif torch.mps.is_available():
```

```
    print("Apple Silicon GPU")
else:
    print("Only CPU")
```

Depending on your machine, the code may return:

```
PyTorch version 2.7.1
Only CPU
```

Using Tensor Cores

If you have a modern NVIDIA GPU (the Volta architecture or newer), you can take advantage of its Tensor Cores, which are specialized units for matrix multiplication that can deliver higher throughput. To enable them, simply execute the following code:

```
torch.set_float32_matmul_precision("high")
```

By default, PyTorch runs in the `"highest"` precision mode, which does not use Tensor Cores. Note that it can have small effects on the results due to differences in floating-point rounding, but based on my tests, it does not appear to alter any results in this book.

Also, if you don't have a GPU or are unsure whether your GPU has Tensor Cores, executing this line is generally safe and should not cause any problems.

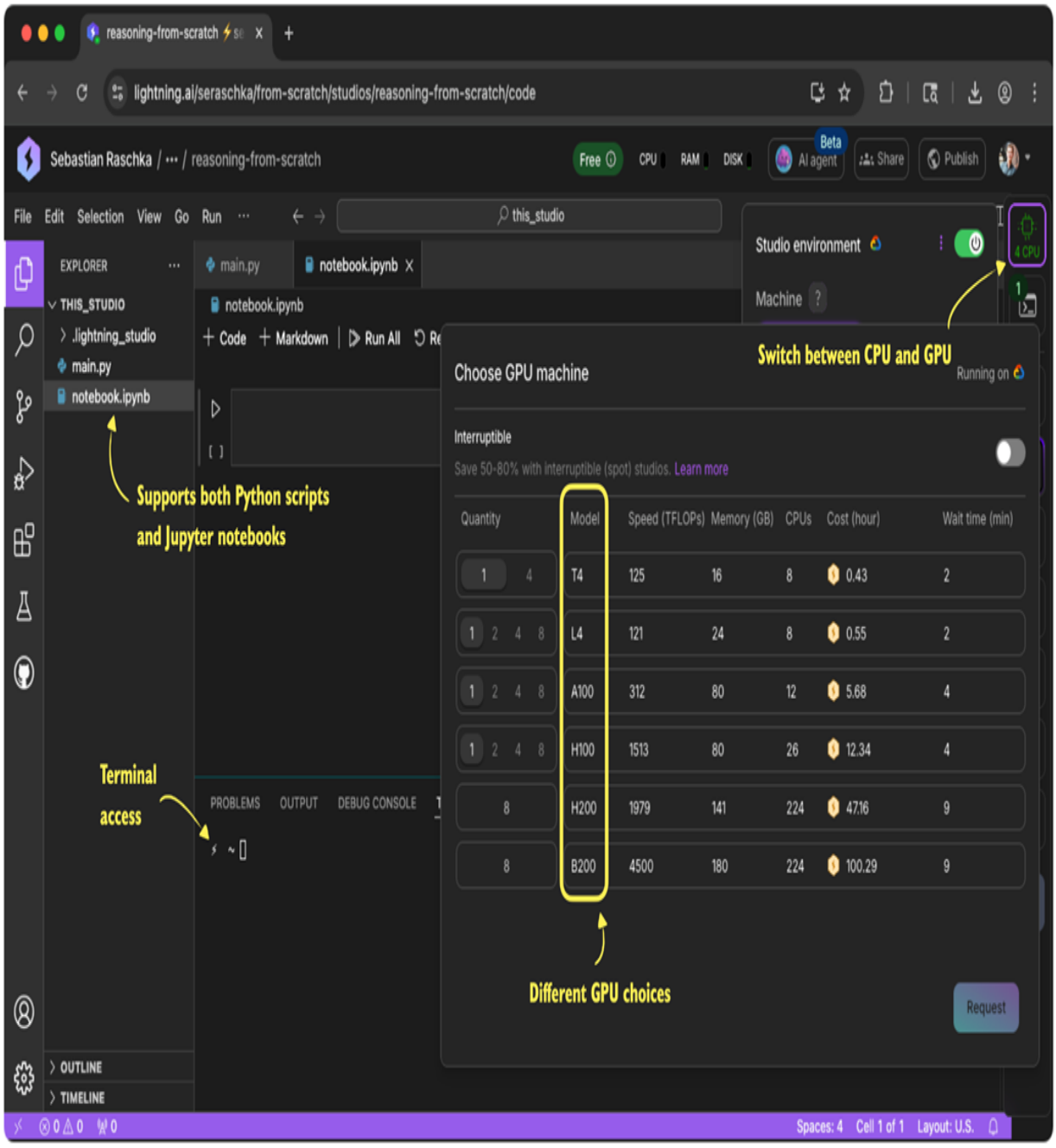
Don't worry if your machine does not have a GPU to run the code. Chapters 2-4 can be executed in a reasonable time on a CPU.

Depending on the chapter, the code will automatically use an NVIDIA GPU if available, otherwise run on the CPU (or Apple Silicon GPU if recommended for a particular section or chapter). However, I will provide more information in the respective sections and chapters.

Like many other AI researchers who work on and with LLMs daily, I don't have a machine with the necessary GPU hardware to train LLMs at home and use cloud resources instead. If you are looking for cloud provider options, my personal preference is Lightning AI Studio (https://lightning.ai/), due to its ease of use and feature support, as shown in figure 2.4.

Alternatively, Google Colab (https://colab.research.google.com/) is another good choice.

Figure 2.4 An overview of the Lightning AI GPU cloud platform in a web browser. The interface supports Python scripts, Jupyter notebooks, terminal access, and lets users switch between CPU and various GPU types based on their compute needs.



As of this writing, Lightning AI also offers users free compute credits after the sign-up and verification process, which can be used for the different GPU choices shown in figure 2.4. (As mentioned before, a GPU is not needed for this chapter; however, if you want to use a GPU, the L4 GPU is more than sufficient for this chapter.

Note

For disclosure, I helped build and launch the Lightning AI platform in 2023 and still hold a small stake. I am not sponsored to recommend it and pay for it myself. I use it because I simply find it the most convenient. It supports multiple types of GPUs, allows easy switching between them and back to CPU to save costs, and lets me pause or resume environments without redoing the setup.

The supplementary code repository contains additional GPU platform recommendations inside the `ch02` subfolder if needed.

Using PyTorch

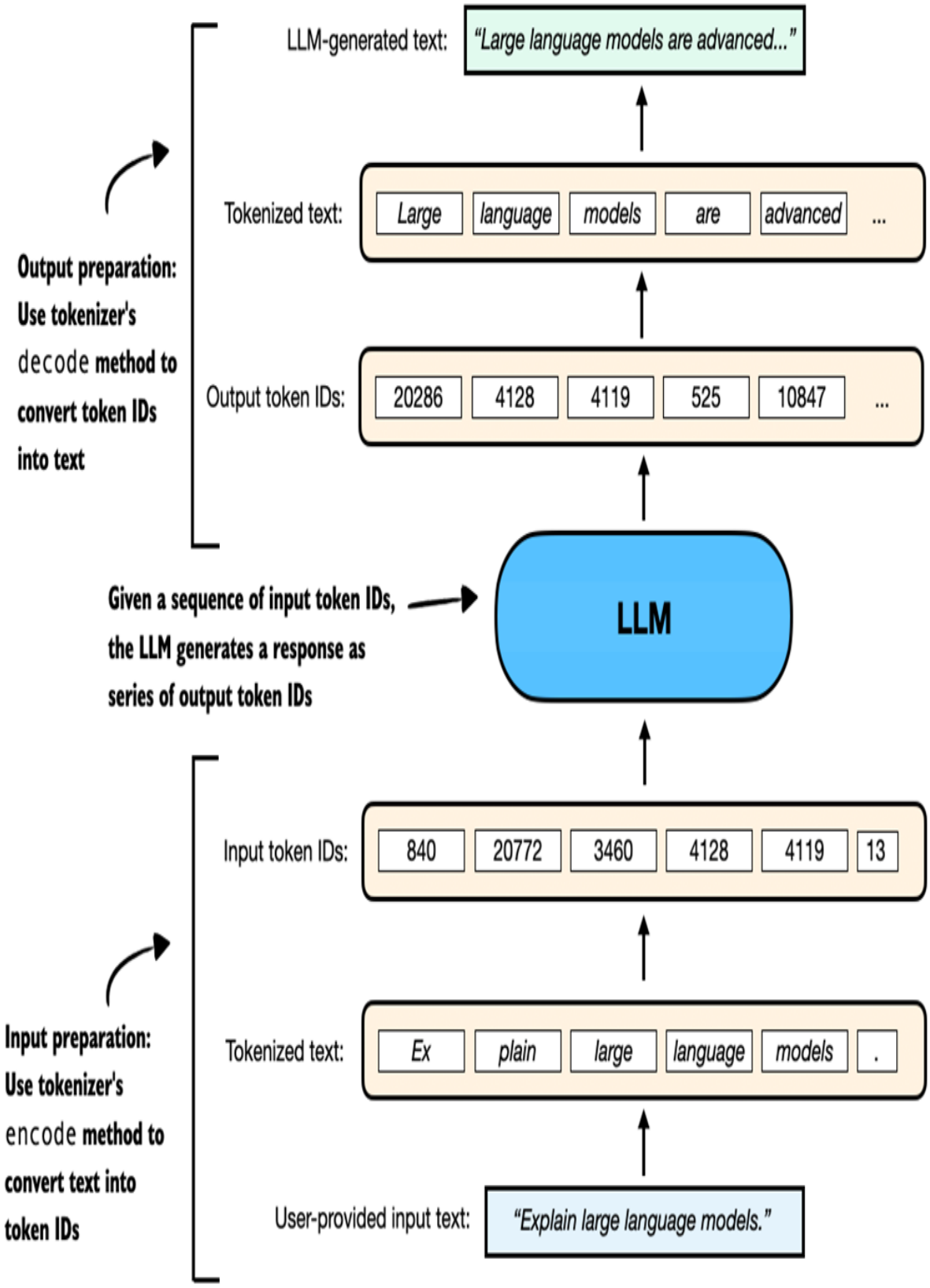
In this section, we imported and used the PyTorch library, which is currently the most widely used general-purpose library. We will use it throughout this book to run and train LLMs, including the reasoning methods we will develop. If you are new to PyTorch, to get the most out of this book, I recommend reading through my *PyTorch in One Hour: From Tensors to Training Neural Networks on Multiple GPUs* tutorial, which is freely available on my website at https://sebastianraschka.com/teaching/pytorch-1h.

2.4 Preparing input texts for LLMs

In this section, we explore how to use a tokenizer to process input and output text for an LLM, as shown in figure 2.5, which expands on the input and output preparation steps shown earlier in figure 2.2 to provide a more detailed view of the tokenization pipeline.

Figure 2.5 A simplified illustration of how an LLM receives input data and generates output. The user-provided text is tokenized into IDs using the tokenizer's `encode` method, which are then

processed by the LLM to generate output token IDs. These are decoded back into human-readable text using the tokenizer's `decode` method.


LLM-generated text:
"Large language models are advanced..."
Tokenized text:
Large
language
models
are
advanced
...
Output token IDs:
20286
4128
4119
525
10847
...
Output preparation: Use tokenizer's decode method to convert token IDs into text
Given a sequence of input token IDs, the LLM generates a response as series of output token IDs
LLM
Input token IDs:
840
20772
3460
4128
4119
13
Tokenized text:
Ex
plain
large
language
models
.
User-provided input text:
"Explain large language models."
Input preparation: Use tokenizer's encode method to convert text into token IDs

To see how this works in practice, we will begin by loading a tokenizer from this book's `reasoning-from-scratch` Python package, which should have been installed according to the instructions in section 2.2.

To download the tokenizer files (corresponding to the *Qwen3* base LLM, which we will introduce in the next section), run:

```
from reasoning_from_scratch.qwen3 import download_qwen3_small
download_qwen3_small(kind="base", tokenizer_only=True,
out_dir="qwen3")
```

This will display a progress bar similar to:

```
tokenizer-base.json: 100% (6 MiB / 6 MiB)
```

The command downloads the `tokenizer-base.json` file (approximately 6 megabytes in size) and saves it in a `qwen3` subdirectory.

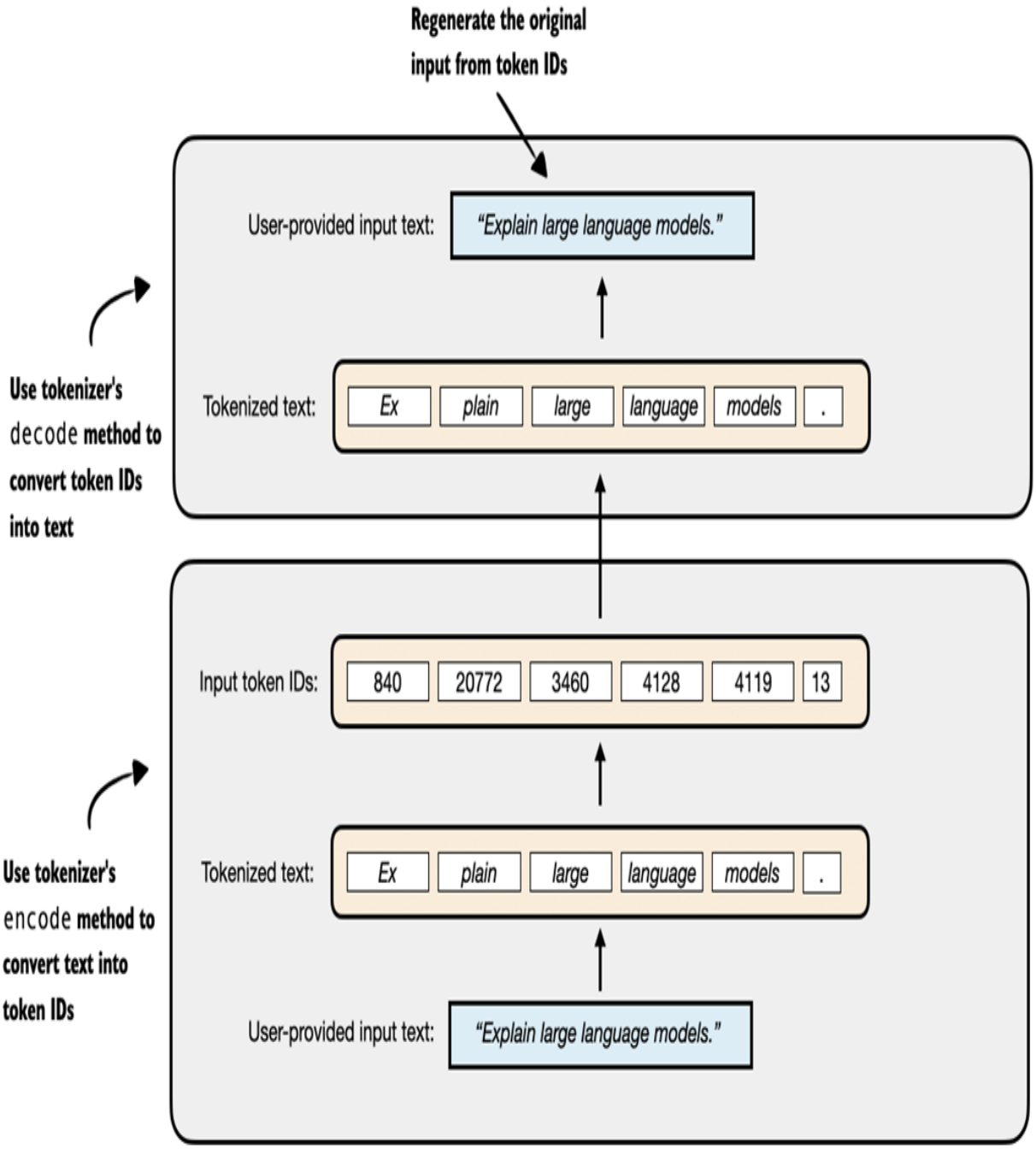
Now, we can load the tokenizer settings from the tokenizer file into the `Qwen3Tokenizer`:

```
from pathlib import Path
from reasoning_from_scratch.qwen3 import Qwen3Tokenizer

tokenizer_path = Path("qwen3") / "tokenizer-base.json"
tokenizer = Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)
```

Since we have not loaded the LLM yet (the central component shown in figure 2.5), we will first do a simpler dry run using just the tokenizer. Specifically, we will do a tokenization round-trip, that is, we will encode a text into *token IDs* and then decode those IDs back into text, as illustrated in figure 2.6.

Figure 2.6 A demonstration of the round-trip tokenization process using a tokenizer. The user-provided input text is first converted into token IDs using the `encode` method, and then accurately reconstructed back into the original text using the `decode` method.



The following code snippet implements the encoding process shown at the bottom of figure 2.6:

```
prompt = "Explain large language models."
input_token_ids_list = tokenizer.encode(prompt)
```

And the following code implements the decoding process, converting the token IDs back into text, shown at the top of figure 2.6:

```
text = tokenizer.decode(input_token_ids_list)
print(text)
```

Based on the printed results, we can see that the tokenizer reconstructed the original input prompt from the token IDs:

```
'Explain large language models.'
```

Before we move on to the LLM, let's take a look at the token IDs that were generated by the `encode` method. The following code prints each token ID and its corresponding decoded string to help illustrate how the tokenizer works:

```
for i in input_token_ids_list:
    print(f"{[i]} --> {tokenizer.decode([i])}")
```

The output is as follows:

```
840 --> Ex
20772 --> plain
3460 -->  large
4128 -->  language
4119 -->  models
13 --> .
```

As shown in the output, the original text is split into six token IDs. Each token represents a word or subword, depending on how the tokenizer segments the input.

For example, "Explain" was split into two separate tokens, "Ex" and "plain". This is because the tokenizer algorithm uses a subword-based method based on *Byte Pair Encoding* (*BPE*). BPE can represent both common and rare words using a mix of full words and subword units. Spaces are also often included in tokens (for example, " large"), which helps the LLM detect word boundaries.

The `Qwen3Tokenizer` has a *vocabulary* of about 151,000 tokens, which is considered relatively large as of this writing (for comparison, the early GPT-

2 has a vocabulary size of approximately 50,000 tokens, and Llama 3 has a vocabulary size of approximately 128,000 tokens).

A larger vocabulary in a language model increases its size and computational cost for each individually generated token, but it also allows more words to be represented as single tokens rather than being split into subword components. This is beneficial because splitting a word (like breaking "Explain" into "Ex" and "plain") results in more input tokens. More tokens lead to longer input sequences, which increases processing time and resource usage. For instance, doubling the number of tokens can roughly double the computational cost of running the model as it needs to generate more tokens to complete the response.

Unfortunately, a detailed coverage and from-scratch implementation of a tokenizer is outside the scope of this book. However, interested readers can find additional resources, including my from-scratch implementation, in the further resources and reading sections in appendix A.

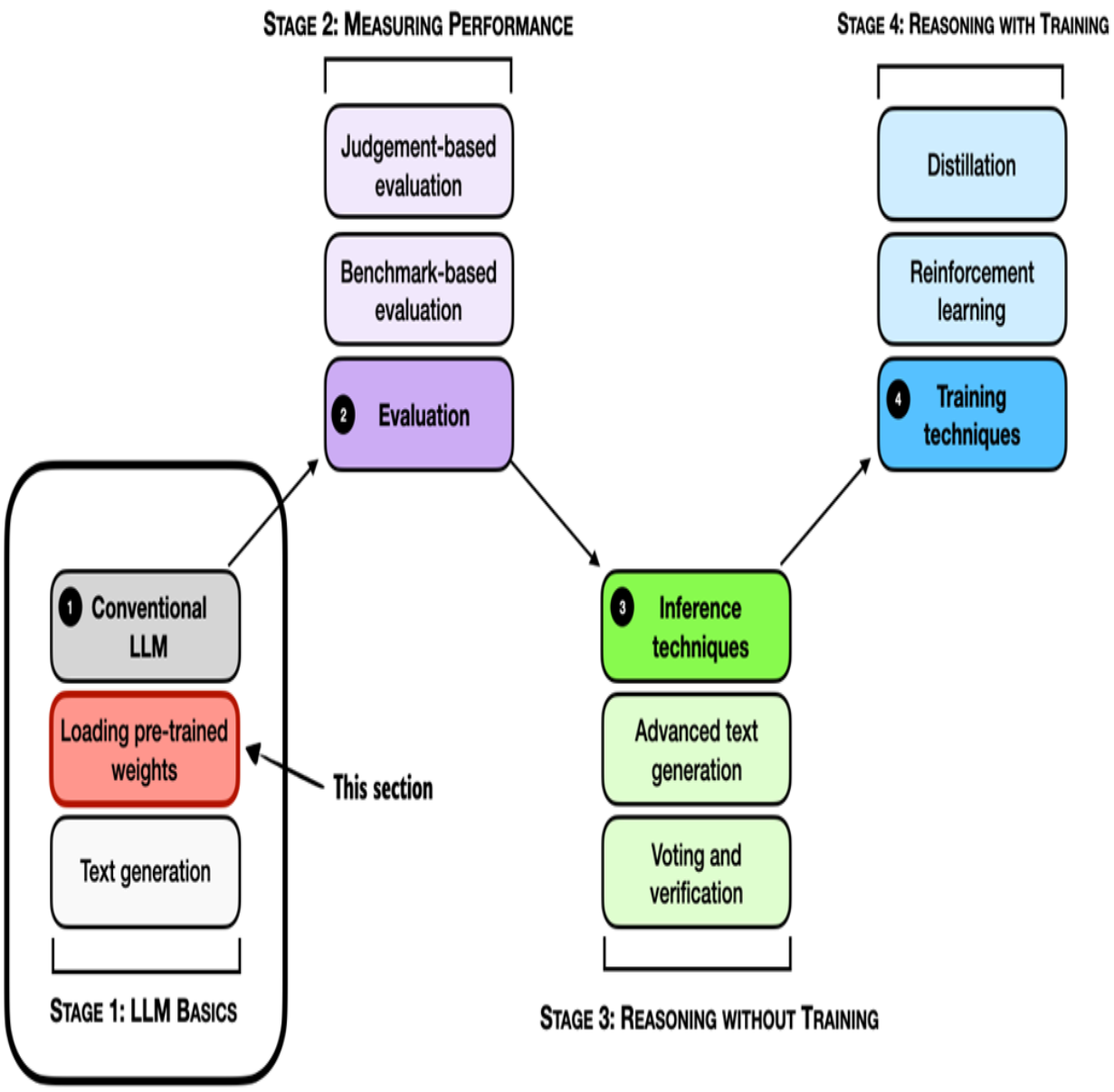
Exercise 2.1: Encoding unknown words

Experiment with the tokenizer to see if and how it handles unknown words. For this, get creative and make up words that don't exist. Also, if you speak multiple languages, try to encode words in a different language than English.

2.5 Loading pre-trained models

In the previous section, we loaded and familiarized ourselves with the tokenizer that prepares the input data for an LLM and converts LLM outputs back into a human-readable text representation. In this section, we will load the LLM itself, as shown in the overview in figure 2.7.

Figure 2.7 An overview of the four key stages in developing a reasoning model in this book. This section focuses on loading pre-trained LLM in Stage 1.



As mentioned in the previous section, this book uses Qwen3 0.6B as a pre-trained base model. In this section, we load its pre-trained weights, as shown in figure 2.7. The "0.6B" in the model name indicates that the model contains approximately 0.6 billion weight parameters.

Why Qwen3? After carefully evaluating several open-weight base models, I chose Qwen3 0.6B for the following reasons:

- For this book, we want a small yet capable open-weight model that can run on consumer hardware.

- The larger variants of the Qwen3 model family are, as of this writing, the leading open-weight models in terms of modeling performance.
- Qwen3 0.6B is more memory-efficient compared to Llama 3 1B and OLMo 2 1B.
- Qwen3 offers both a base model (the focus of our reasoning model development) and an official reasoning variant that we can use as a reference for evaluation purposes.

Note

The canonical spelling of "Qwen3" does not include whitespace, whereas "Llama 3" does.

In line with the spirit of building things "from scratch," this book uses a custom reimplementation of Qwen3 that I wrote in pure PyTorch, which is entirely independent of external LLM libraries. The emphasis of this reimplementation is on code readability and tweakability, in case you want to modify it later for your own experiments. Despite being built from scratch, this implementation remains fully compatible with the original pre-trained Qwen3 model weights.

However, this book does not cover the Qwen3 code implementation in depth. This topic alone would fill an entire separate book, similar to my other book, *Build A Large Language Model (From Scratch)*. Instead, this *Build A Reasoning Model (From Scratch)* book specifically focuses on implementing reasoning methods on top of a base model, in this case, Qwen3.

Note

This reimplemented Qwen3 LLM runs entirely locally, just like any other neural network implemented in PyTorch. There are no server-side components or external API calls involved. All model usage happens on your own machine, and your data stays on your device. If you are concerned about privacy, the setup we are using ensures full control over both the LLM inputs and outputs.

For those interested in additional details about Qwen3, as well as the model code, please see appendix C.

Before we load the model, we can specify the device we are going to use, namely, CPU or GPU. The following code will select the best-available device automatically:

```
def get_device():
    if torch.cuda.is_available():
        device = torch.device("cuda")
        print("Using NVIDIA CUDA GPU")
    elif torch.backends.mps.is_available():
        device = torch.device("mps")
        print("Using Apple Silicon GPU (MPS)")
    elif torch.xpu.is_available():
        device = torch.device("xpu")
        print("Intel GPU")
    else:
        device = torch.device("cpu")
        print("Using CPU")
    return device

device = get_device()
```

While GPUs generally provide substantial speed and performance improvements, it can be helpful to initially run the remaining code in this chapter using the CPU for compatibility and debugging purposes. You can temporarily override the automatic selection by explicitly setting:

```
device = torch.device("cpu")
```

After finishing the chapter and verifying the code works properly on the CPU, remove or comment out the manual override and rerun the code. If your system has a GPU, you should then observe improved performance.

Note

The code in the remainder of this chapter was executed on a Mac Mini with an Apple M4 CPU. Performance comparisons with the Apple Silicon M4 GPU and the NVIDIA H100 GPU are included at the end of the chapter.

However, before we load the model and put it onto the selected `device`, we first need to download the weights for Qwen3 0.6B. These files are required to initialize the pre-trained model correctly:

```
download_qwen3_small(kind="base", tokenizer_only=False,
out_dir="qwen3")
```

The output is as follows:

```
qwen3-0.6B-base.pth: 100% (1433 MiB / 1433 MiB)
âœ“ qwen3/tokenizer-base.json already up-to-date
```

(There is a checkmark in front of the tokenizer because we already downloaded it in the previous section.)

After downloading the model weights via the previous step, we can now instantiate a `Qwen3Model` class into which we load the pre-trained weights via PyTorch's `load_state_dict` method:

```
from reasoning_from_scratch.qwen3 import Qwen3Model,
QWEN_CONFIG_06_B

model_path = Path("qwen3") / "qwen3-0.6B-base.pth"
model = Qwen3Model(QWEN_CONFIG_06_B)  #A
model.load_state_dict(torch.load(model_path))  #B
model.to(device)  #C
```

Note that if the device setting is `"cpu"`, the `model.to(device)` operation will be skipped because the model already sits in CPU memory by default.

After executing the code above, you should see the following output:

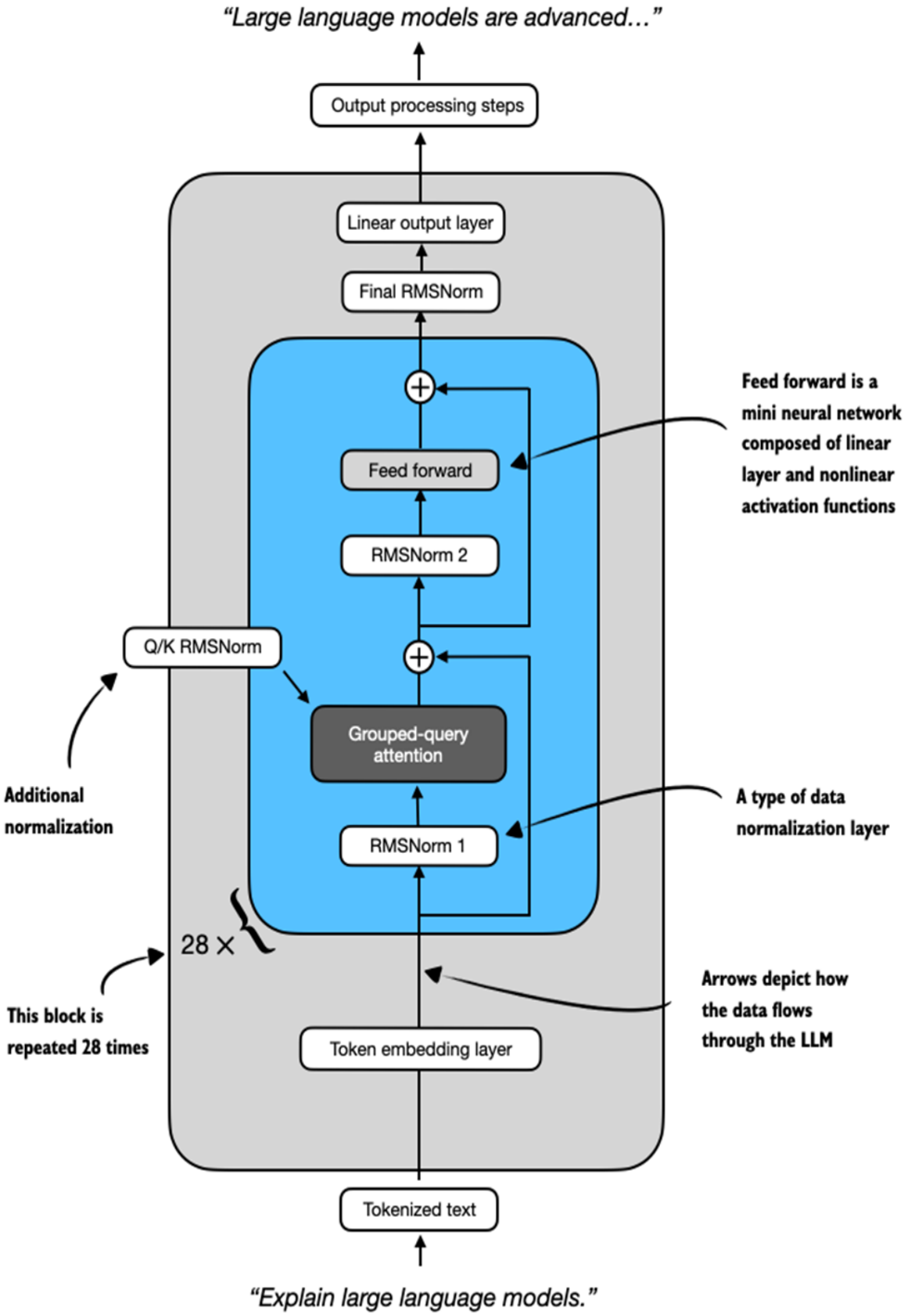
```
Qwen3Model(
  (tok_emb): Embedding(151936, 1024)
  (trf_blocks): ModuleList(
    (0-27): 28 x TransformerBlock(
      (att): GroupedQueryAttention(
        (W_query): Linear(in_features=1024, out_features=2048,
bias=False)
        (W_key): Linear(in_features=1024, out_features=1024,
bias=False)
        (W_value): Linear(in_features=1024, out_features=1024,
bias=False)
        (out_proj): Linear(in_features=2048, out_features=1024,
```

```
bias=False)
        (q_norm): RMSNorm()
        (k_norm): RMSNorm()
      )
      (ff): FeedForward(
        (fc1): Linear(in_features=1024, out_features=3072,
bias=False)
        (fc2): Linear(in_features=1024, out_features=3072,
bias=False)
        (fc3): Linear(in_features=3072, out_features=1024,
bias=False)
      )
      (norm1): RMSNorm()
      (norm2): RMSNorm()
    )
  )
  (final_norm): RMSNorm()
  (out_head): Linear(in_features=1024, out_features=151936,
bias=False)
)
```

This output is a summary of the Qwen3 0.6B base model architecture, as printed by PyTorch. It highlights the model's core components: an embedding layer, a stack of 28 transformer blocks, and a final linear projection head. Each transformer block includes a grouped-query attention mechanism and a multi-layer feedforward network, along with normalization layers throughout.

These components are also illustrated visually in figure 2.8 for readers familiar with LLM architectures. However, a detailed understanding of this architecture is not required for this book. Since we are not modifying the base model itself, but rather building reasoning methods on top of it, you can safely treat the architecture as a black box for now. However, interested readers can optionally find more information on these components in appendix C.

Figure 2.8 Overview of the Qwen3 0.6B model architecture. Input text is tokenized and passed through an embedding layer, followed by 28 repeated transformer blocks. Each block contains grouped-query attention, feedforward layers, and RMS normalization. The model ends with a final normalization and linear output layer. Arrows show the data flow through the model.

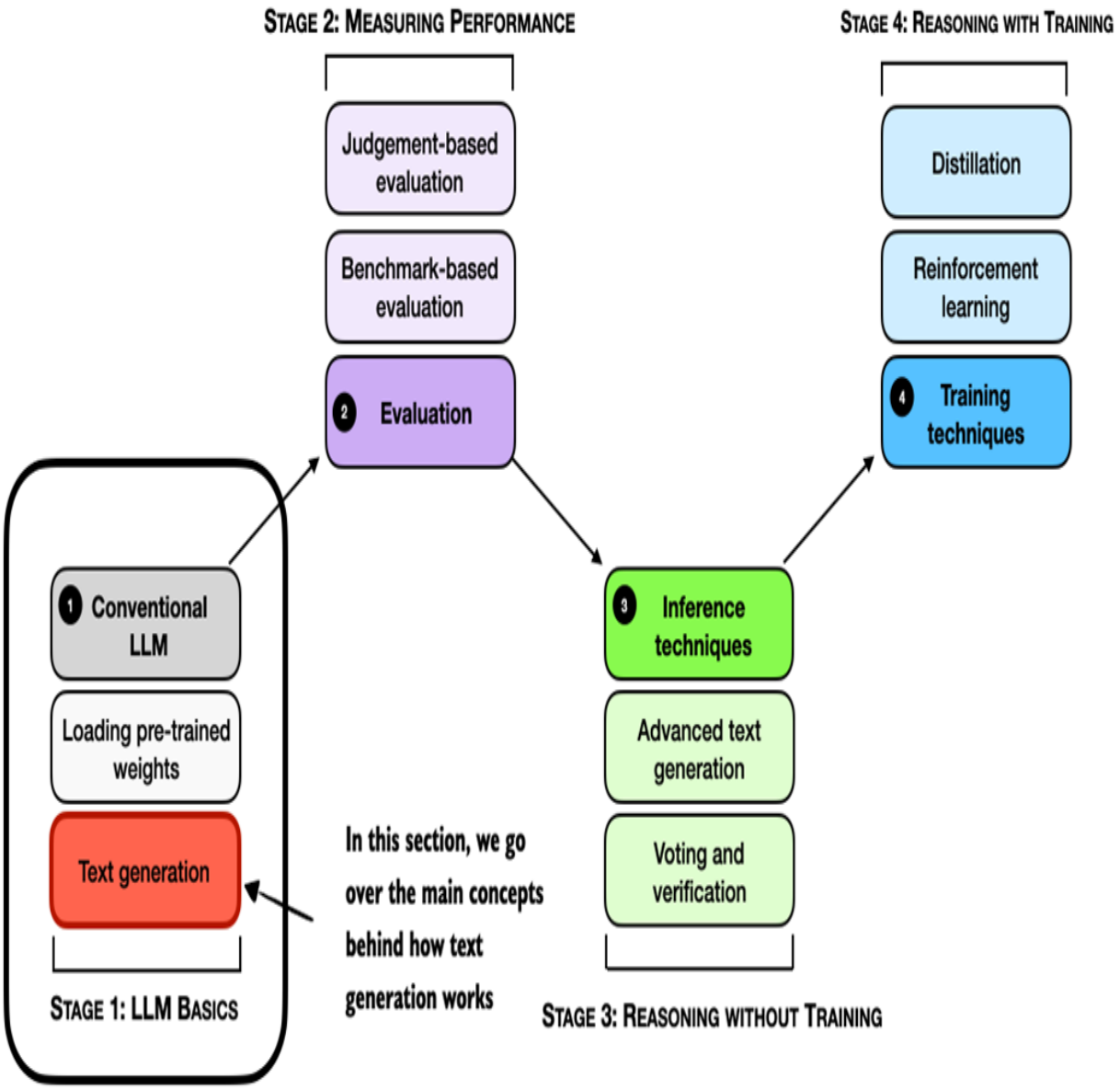

"Large language models are advanced..."
Output processing steps
Linear output layer
Final RMSNorm
Feed forward
RMSNorm 2
Q/K RMSNorm
Grouped-query attention
RMSNorm 1
28 ×
Token embedding layer
Tokenized text
"Explain large language models."
Feed forward is a mini neural network composed of linear layer and nonlinear activation functions
Additional normalization
A type of data normalization layer
This block is repeated 28 times
Arrows depict how the data flows through the LLM

The key takeaway from this section is that we have now loaded a pre-trained model, with its architecture shown in figure 2.8, that should be capable of generating coherent text. In the next section, we will code a text generation function that feeds tokenized data into the model and returns the response in a human-readable format.

2.6 Understanding the sequential LLM text generation process

After loading a pre-trained LLM, our goal is to write a function that leverages the LLM to generate text. This function forms the foundation for reasoning-improving methods that we will implement later in the book, as shown in figure 2.9.

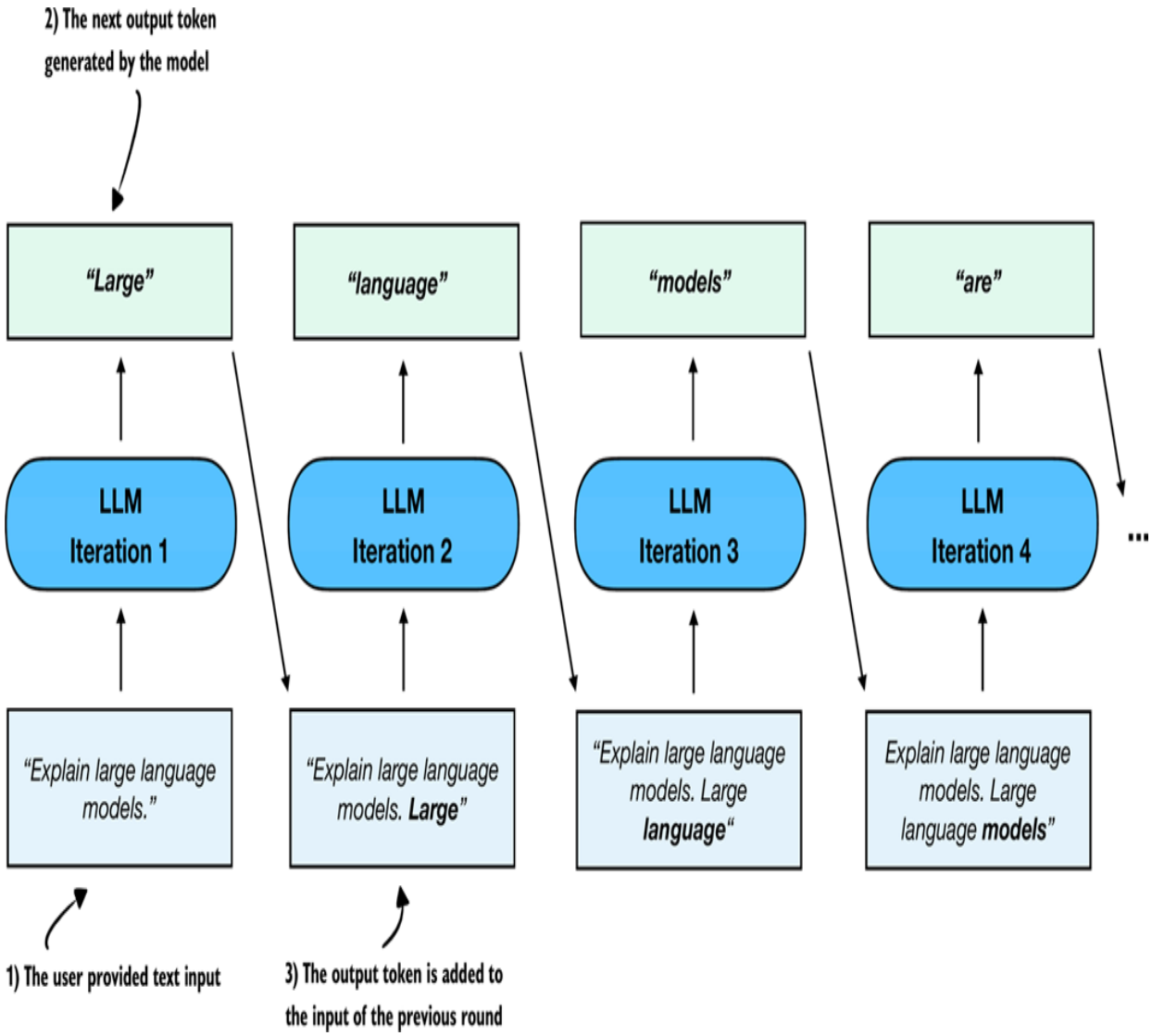
Figure 2.9 An overview of the four key stages in developing a reasoning model in this book. This section explains the main concept behind text generation in LLMs, which allows us to implement a text generation function for using the pre-trained LLM in the remainder of this chapter.



However, before we get to implement this text generation function that we will use in this and upcoming chapters (as shown in figure 2.9), let's go over the basic concepts behind text generation in LLMs.

You may already know that text generation in LLMs is a sequential process where LLMs generate one word at a time. This is often also called *autoregressive* text generation and is shown in figure 2.10

Figure 2.10 An illustration of the sequential (autoregressive) text generation in LLMs. At each iteration, the model generates the next token based on the input and previously generated tokens, which are cumulatively fed back into the model to produce coherent output.

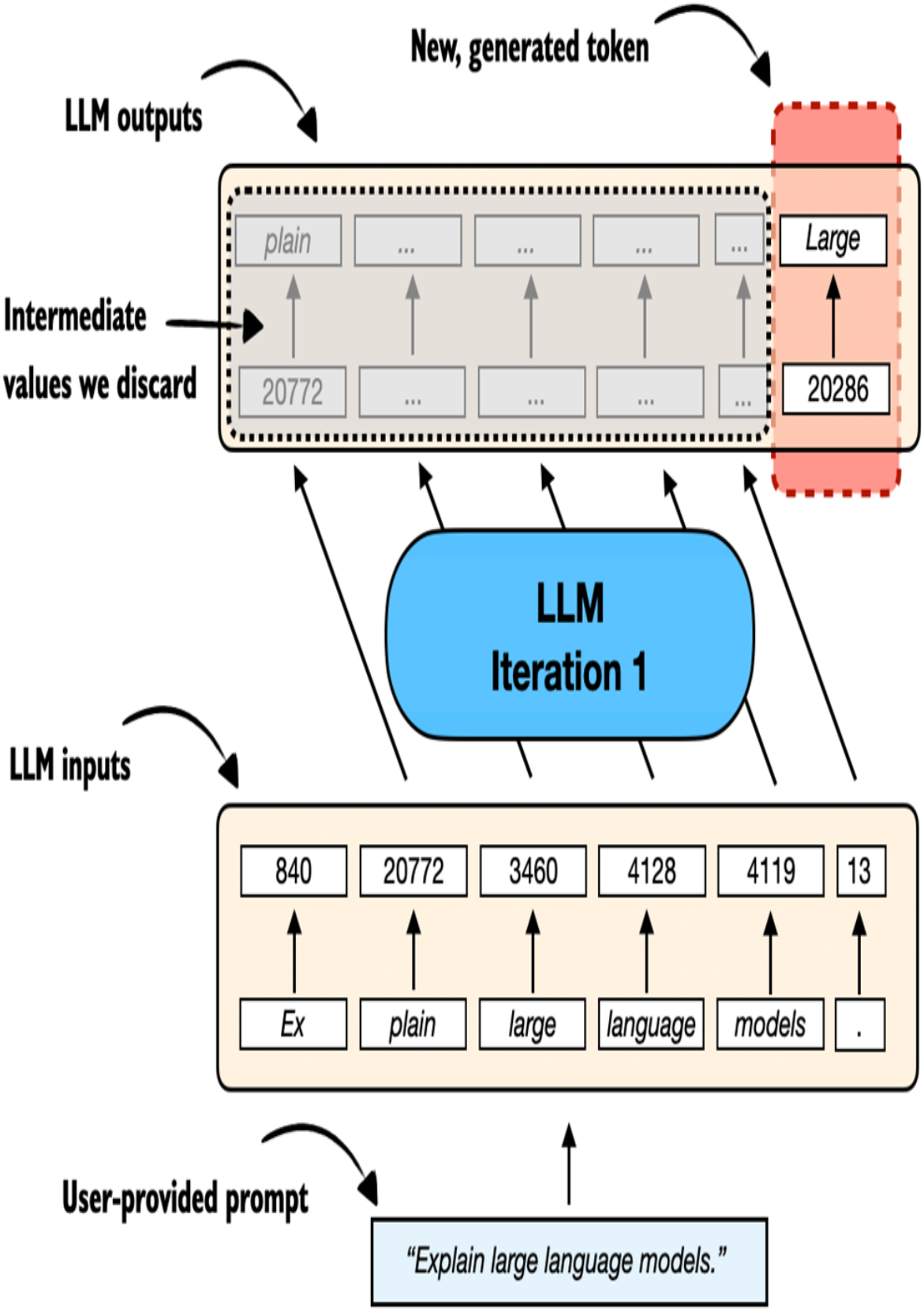


Note that the sequential text generation process shown in figure 2.10 is a broad overview. The figure shows one generated output token (top row) at each step, when feeding it with an input prompt. This is done for simplicity to explain the main concept behind LLM-based text generation.

Now, if we look at one of these iterations more closely, an LLM generates one output token for each input token. This means that if we have six input tokens, the LLM returns six output tokens, as illustrated in figure 2.11. However, it is important to note that we only care about the last generated token in each iteration.

Figure 2.11 A closer look at a single iteration of the autoregressive text generation process. The LLM generates an output sequence that mirrors the input but is shifted one position to the right.

At each iteration, the model predicts the next token in the sequence, The LLM effectively learns to continue the input prompt one token at a time.


New, generated token
LLM outputs
plain
...
...
...
...
Large
Intermediate
values we discard
20772
...
...
...
...
20286
LLM
Iteration 1
LLM inputs
840
20772
3460
4128
4119
13
Ex
plain
large
language
models
.
User-provided prompt
"Explain large language models."

Before implementing a text-generation function that uses the concept shown in figure 2.11 for each iteration to implement the autoregressive text generation process shown in figure 2.10, let's take a look at a code example to illustrate figure 2.11 further by reusing the "Explain large language models." example prompt from section 2.4:

```
prompt = "Explain large language models."
input_token_ids_list = tokenizer.encode(prompt)
print(f"Number of input tokens: {len(input_token_ids_list)}")

input_tensor = torch.tensor(input_token_ids_list)  #A
input_tensor_fmt = input_tensor.unsqueeze(0)  #B
input_tensor_fmt = input_tensor_fmt.to(device)

output_tensor = model(input_tensor_fmt)  #C
output_tensor_fmt = output_tensor.squeeze(0)  #D
print(f"Formatted Output tensor shape:
{output_tensor_fmt.shape}")
```

Squeezing and unsqueezing tensors

The `.squeeze()` and `.unsqueeze()` operations in PyTorch are used to change the shape of a tensor by removing or adding dimensions of size 1. This is often useful for reshaping a tensor to match what a model expects. For example, a model might expect input tensors with two dimensions (e.g., rows and columns) so it can process batches of inputs (see appendix E). But if the input is just a row vector, we can use `.unsqueeze(0)` to add an extra dimension and make it compatible:

```
example = torch.tensor([1, 2, 3])
print(example)
print(example.unsqueeze(0))
```

This returns:

```
tensor([1, 2, 3])
tensor([[1, 2, 3]])
```

Here, `.unsqueeze(0)` adds a new dimension at position 0, turning a 1D tensor into a 2D tensor with shape `(1, 3)`. Conversely, `.squeeze(0)` removes a dimension of size 1 from position 0:

```
example = torch.tensor([[1, 2, 3]])
print(example)
print(example.squeeze(0))
```

This returns:

```
tensor([[1, 2, 3]])
tensor([1, 2, 3])
```

This is useful when you want to remove extra dimensions that are not needed.

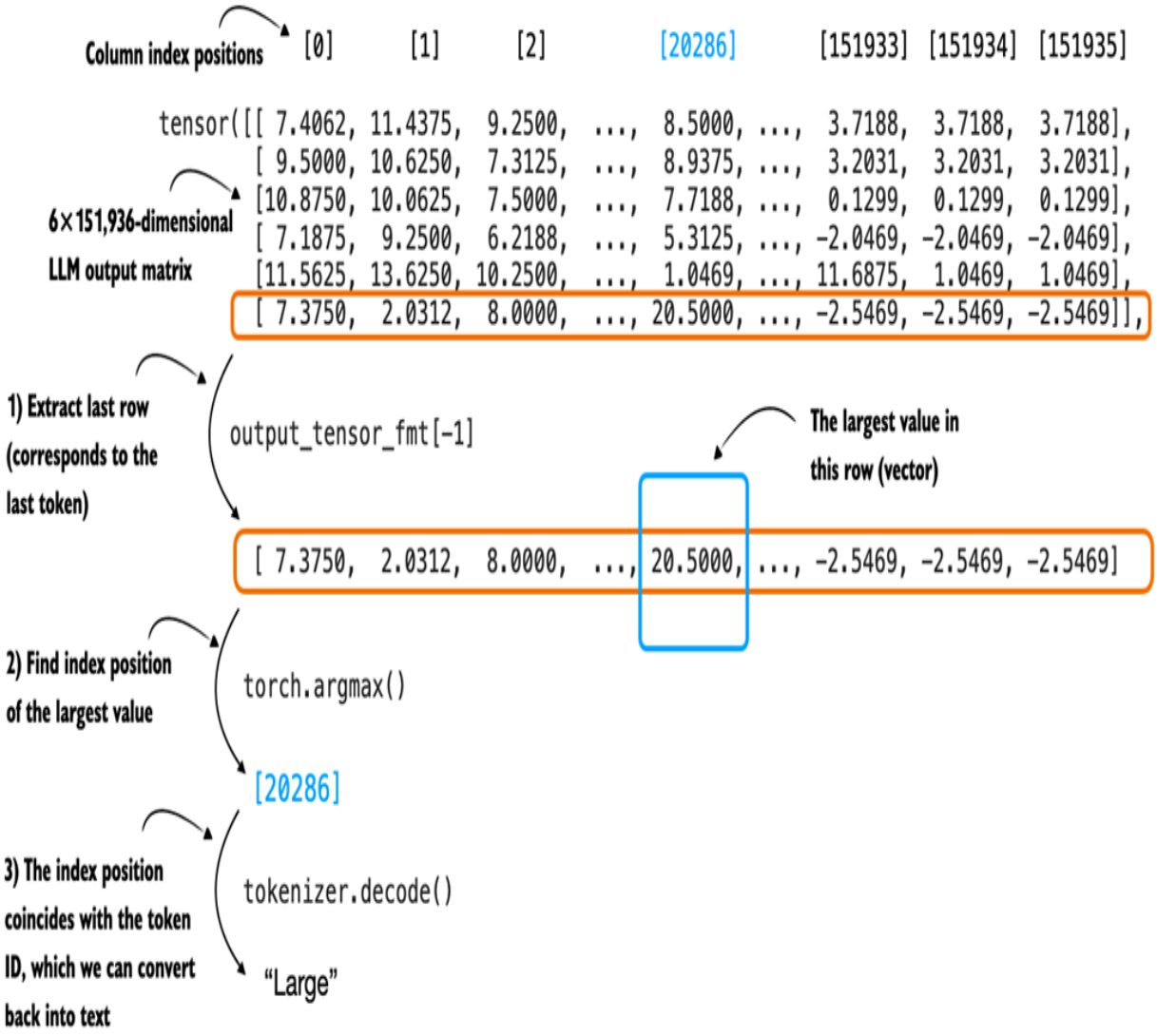
The output from the previous code example is follows:

```
Number of input tokens: 6
Formatted Output tensor shape: torch.Size([6, 151936])
```

As we can see, we feed six input tokens into the model, which returns a 6×151,936-dimensional matrix. The 6 in this matrix corresponds to the six input tokens. The second dimension, 151,936, corresponds to the vocabulary size that the model supports. For instance, each of the six tokens is represented by a vector with 151,936 values. We can think of the values in these vectors as scores for each possible word in the vocabulary, where the highest score corresponds to the most likely word or subword (in the 151,936-entry vocabulary) to be chosen as the generated token.

So, to get the next generated word, we extract the last row of this 6×151,936-dimensional matrix, find the token ID corresponding to the largest score value in this row, and convert this token ID back into text via the tokenizer, as illustrated in figure 2.12.

Figure 2.12 A closer look at how the raw scores output by an LLM, in a single text generation iteration, are converted into a token ID and its corresponding text representation.



Let's see how we can convert the LLM output matrix into the generated text token (shown in figure 2.12) in code:

Note that LLMs are trained with a next-word prediction task, and as shown in figure 2.11, we are only interested in the last token, which we can obtain via the `[-1]` index:

```
last_token = output_tensor_fmt[-1].detach()
print(last_token)
```

Here, `.detach()` separates the tensor from the part of the system that tracks how the model learns. In simple terms, it lets us take the last token from the model's output and use it for the next step without keeping extra information

we don't need during generation. This saves memory and can make things run faster.

This prints the 151,936 values corresponding to the last token:

```
tensor([ 7.3750,  2.0312,  8.0000,  ..., -2.5469, -2.5469,
-2.5469],
       dtype=torch.bfloat16)
```

Then, we can use the argmax function to obtain the position with the largest value score (value) in this tensor:

```
print(last_token.argmax(dim=-1, keepdim=True))
```

The result is:

```
tensor([20286])
```

This returned integer value is the position of the largest value in this vector, and it also corresponds to the token ID of the generated token (`last_token`), which we can translate back into text via the tokenizer:

```
print(tokenizer.decode([20286]))
```

This prints the generated token:

```
 Large
```

Max versus argmax

The `torch.max()` and `torch.argmax()` functions in PyTorch are used to find the largest value in a tensor and the index of that value. For example:

```
example = torch.tensor([-2, 1, 3, 1])
print(torch.max(example))
print(torch.argmax(example))
```

This returns:

```
tensor(3)
tensor(2)
```

The maximum value is 3, and it first appears at index 2.

We can also use `keepdim=True` with `torch.argmax()` to keep the output shape consistent by retaining the reduced dimension:

```
print(torch.argmax(example, keepdim=True))
```

This returns:

```
tensor([2])
```

Here, `keepdim=True` keeps the result as a 1D tensor with the same number of dimensions as the input, which can be helpful for keeping the shape required by the tokenizer and for concatenation later on in our text generation function.

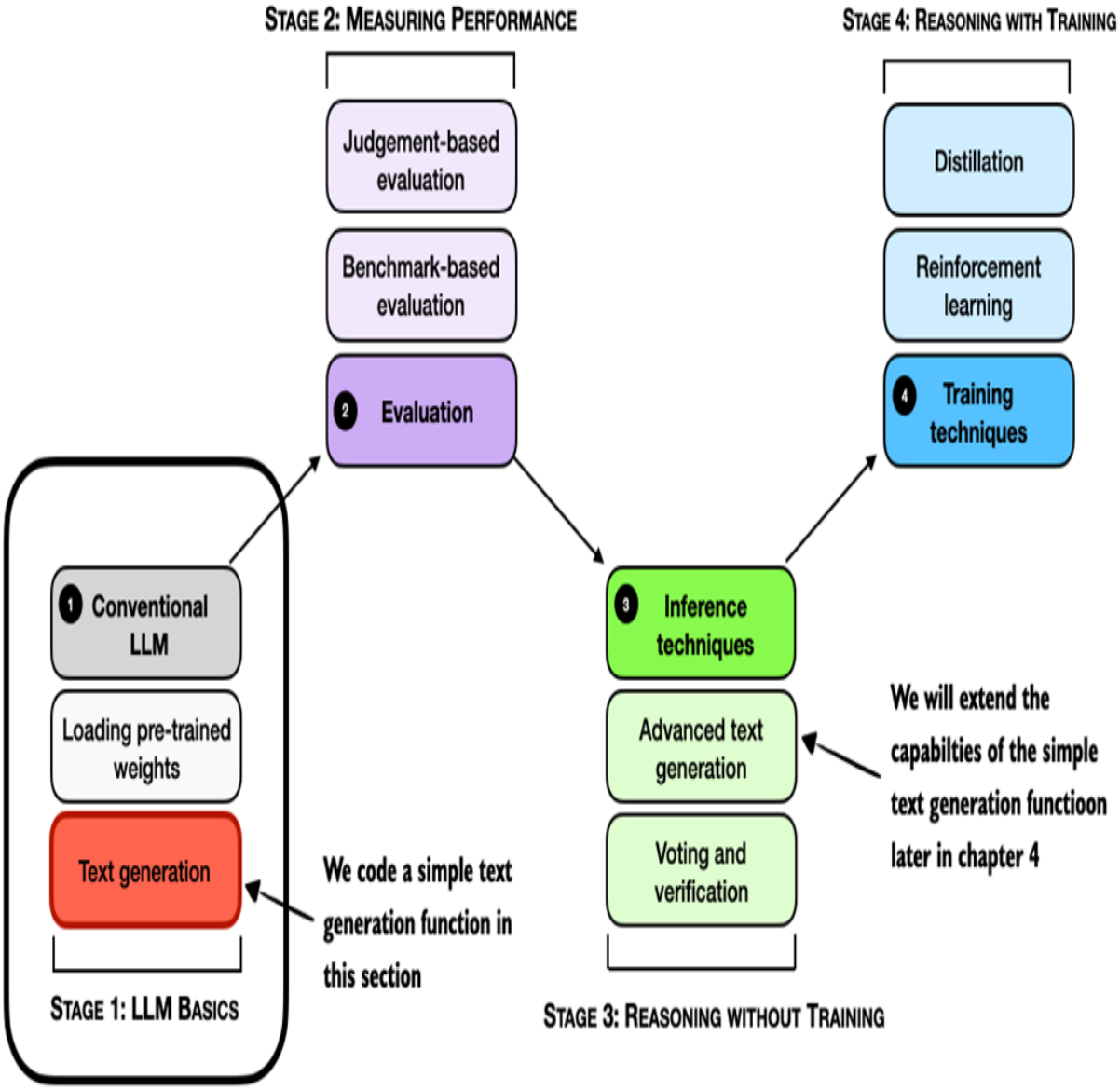
To recap, figure 2.10 illustrated the iterative (autoregressive) text generation process in an LLM. Then, figure 2.11 zooms in on one of the iterations in this process. Figure 2.12 then further zoomed into this one iteration and shows how the score matrix (output by an LLM), gets converted into a token ID (and its corresponding text representation).

While we have seen how to use the LLM to generate a single token, in the next section, we will put these concepts to action and implement a function that applies this concept sequentially to generate coherent output text.

2.7 Coding a minimal text generation function

The previous section explained a single iteration in the basic, sequential text generation process in LLMs. In this section, building on that concept, we will implement a text generation function that uses the pre-trained LLM to generate coherent text following a user prompt, as illustrated in Figure 2.13 in the chapter overview.

Figure 2.13 An overview of the four key stages in developing a reasoning model in this book. In this section we implement a text generation function for the pre-trained LLM.

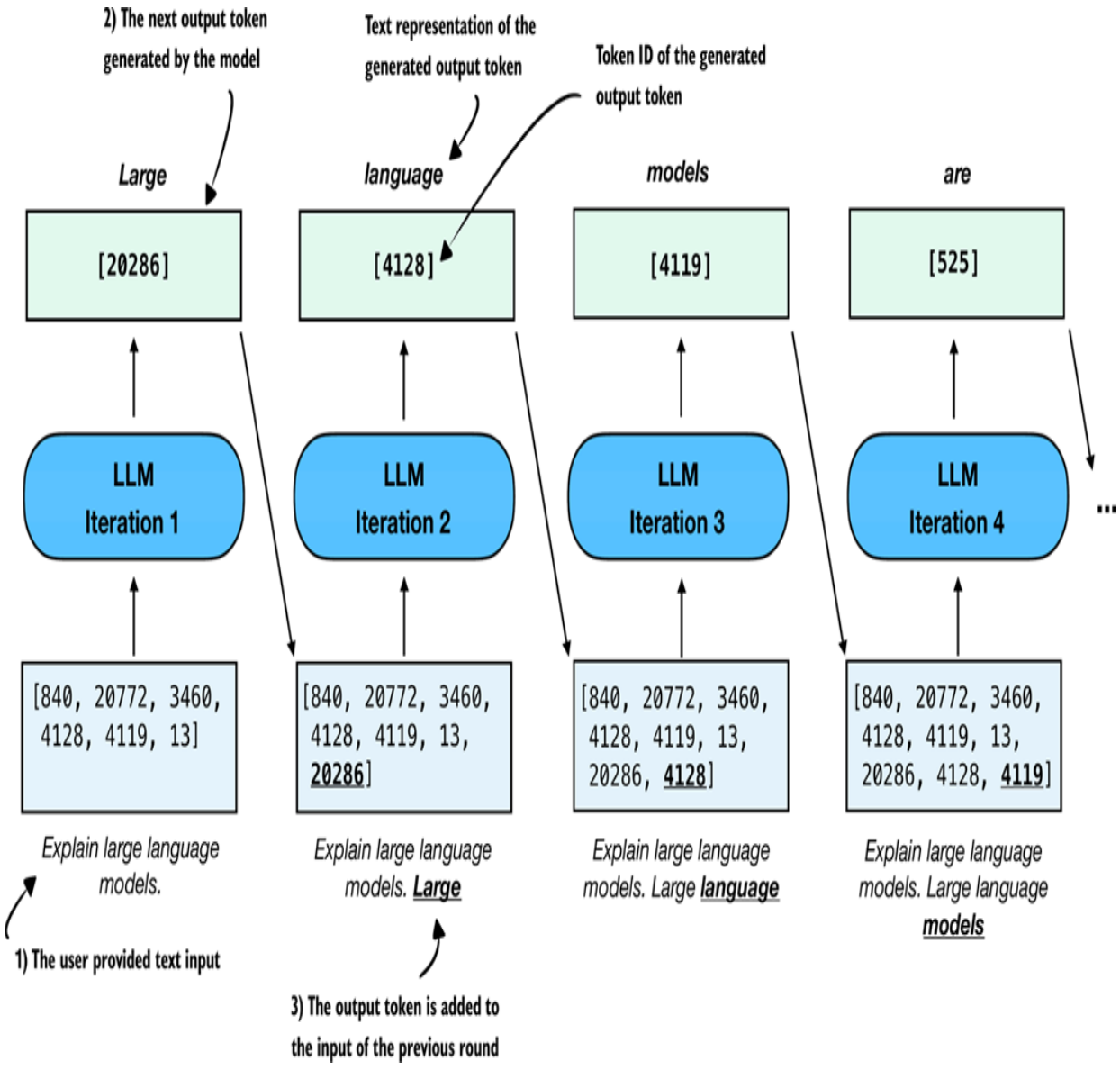


This text generation function, mentioned in figure 2.13, works by first converting the input prompt into token IDs that the model can process. The model then predicts the next most likely token, appends it to the sequence, and reprocesses the extended sequence to generate the next token. This iterative process continues until a stopping condition is met, and the generated token IDs are then decoded back into text.

Figure 2.14 shows this process step by step, with both the generated token IDs and their corresponding text at each stage. (This figure is similar to

figure 2.10 shown at the beginning of the previous section, except it shows the generated token ID alongside their text representation.)

Figure 2.14 An illustration of sequential (autoregressive) text generation in large language models (LLMs), with token IDs shown explicitly. At each iteration, the model generates the next token based on the original input and all previously generated tokens. The predicted token is added to the sequence in both its textual and token ID form.



The `generate_text_basic` function in listing 2.1 below implements the sequential text generation process (figure 2.14) using the `argmax` function introduced in the previous section:

Listing 2.1 A basic text generation function

```
@torch.inference_mode()
#A
def generate_text_basic(
    model,
    token_ids,
    max_new_tokens,
    eos_token_id=None
):
    input_length = token_ids.shape[1]
    model.eval()
#B

    for _ in range(max_new_tokens):
        out = model(token_ids)[:, -1]
#C
        next_token = torch.argmax(out, dim=-1, keepdim=True)

        if (eos_token_id is not None
#D
                and torch.all(next_token == eos_token_id)):
            break

        token_ids = torch.cat(
#E
            [token_ids, next_token], dim=1)
#E
    return token_ids[:, input_length:]
#F
```

In essence, the `generate_text_basic` function listing 2.1 applies the `argmax`-based token ID extraction via a for-loop for a user-specified number of iterations (`max_new_tokens`). It returns the generated token IDs, similar to what's shown in figure 2.14, which we can then convert back into text.

Let's use the function to generate a 100-token response to a simple `"Explain large language models in a single sentence."` prompt to make sure that the `Qwen3Model` and `generate_text_basic` function work (we get to the reasoning task examples in later chapters).

Please note that the following code will be slow and can take 1-3 minutes to complete, depending on your computer (we will speed it up in later

sections):

```
prompt = "Explain large language models in a single sentence."
input_token_ids_tensor = torch.tensor(
    tokenizer.encode(prompt),
    device=device                              #A
    ).unsqueeze(0)

max_new_tokens = 100                          #B
output_token_ids_tensor = generate_text_basic(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=max_new_tokens,
)
output_text = tokenizer.decode(
    output_token_ids_tensor.squeeze(0).tolist()     #C
)
print(output_text)
```

The generated output text is as follows:

```
 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
writing
articles, and even creating creative content.<|endoftext|>Human
language
is a complex and dynamic system that has evolved over millions
of
years to enable effective communication and social interaction.
It is
composed of a vast array of symbols, including letters, numbers,
and
words, which are used to convey meaning and express thoughts and
ideas. The evolution of language has
```

Note that the output above was generated on a CPU. Depending on the device (e.g., CPU versus GPU), the exact wording may vary slightly due to differences in floating-point behavior on different hardware.

As we can see based on the output above, the model follows the instruction quite well by producing a single, clear sentence in response to the prompt.

However, it continues generating additional, off-topic text after the special token `<|endoftext|>`. This token is used during training to mark the end of a document and separate different samples.

Tip

The leading whitespace in `" Large"` (the first output word) appears because the model continued the text based on the input prompt but we sliced off the original prompt with `token_ids[:, input_length:]` in the return line in listing 2.1. If this leading whitespace bothers you, you can remove it via `token_ids[:, input_length:].lstrip()` or `output_text.lstrip()`.

When using the model for inference (that is, generating outputs based on input), we typically want it to stop as soon as it produces the special token `<|endoftext|>`. This token is represented by the ID 151643, which we can confirm using:

```
print(tokenizer.encode("<|endoftext|>"))
```

For convenience, this token ID is also saved via the `tokenizer.eos_token_id` attribute. We can pass this ID to the `generate_text_basic` function to signal when generation should stop:

```
output_token_ids_tensor = generate_text_basic(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=max_new_tokens,
    eos_token_id=tokenizer.eos_token_id  #A
)
output_text = tokenizer.decode(
    output_token_ids_tensor.squeeze(0).tolist()
)
print(output_text)
```

The output looks like this:

```
 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
```

```
writing
articles, and even creating creative content.
```

If we compare the response to the previous response, we can see that the text generation stopped once the end-of-sequence token was encountered.

You may have noticed that generating the response is relatively slow and might take several seconds up to multiple minutes, depending on the hardware.

Exercise 2.2: Streaming token generation

Write a modified version of the `generate_text_basic` function that returns each token as it is generated and prints it, which is also known as *streaming* token generation.

The goal of this exercise is to understand how to implement token-by-token text generation, a technique often used in real-time applications like chatbots and interactive assistants.

Tip 1: Use `yield` instead of `return` to turn the function into a generator.

Tip 2: Then, outside the function, decode each token using a tokenizer and print it as it's generated (`for token in generate_text_basic_stream(...):...`) to simulate streaming output.

Before we wrap up and learn how to speed up this function substantially, let's implement a simple utility function that measures the runtime of the text generation process:

```
def generate_stats(output_token_ids, tokenizer, start_time,
end_time):
    total_time = end_time - start_time
    print(f"Time: {total_time:.2f} sec")
    print(f"{int(output_token_ids.numel() / total_time)}
tokens/sec")

    for name, backend in (("CUDA", getattr(torch, "cuda",
None)),
                          ("XPU", getattr(torch, "xpu", None))):
        if backend is not None and backend.is_available():
```

```
            max_mem_bytes = backend.max_memory_allocated()
            max_mem_gb = max_mem_bytes / (1024 ** 3)
            print(f"Max {name} memory allocated:
{max_mem_gb:.2f} GB")
            backend.reset_peak_memory_stats()

    output_text =
tokenizer.decode(output_token_ids.squeeze(0).tolist())
    print(f"\n{output_text}")
```

The `generate_stats` function above will calculate the total runtime, given a start and end time stamp, the generation speed in terms of tokens per second (tokens/sec), and the GPU memory used. Note that the GPU memory usage is currently only computed for CUDA-supported GPUs, as PyTorch lacks similar utility functions for CPUs and Apple Silicon GPUs.

To apply the `generate_stats` function, we obtain a `start_time` and `end_time` stamp immediately before and after running the `generate_text_basic` function via Python's `time` module:

```
import time

start_time = time.time()
output_token_ids_tensor = generate_text_basic(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=max_new_tokens,
    eos_token_id=tokenizer.eos_token_id
)
end_time = time.time()
generate_stats(output_token_ids_tensor, tokenizer, start_time,
end_time)
```

The output, on a Mac Mini M4 CPU, is as follows:

```
Time: 7.94 sec
5 tokens/sec
 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
writing
articles, and even creating creative content.
```

At 5 tokens per second, the generation speed is relatively slow. In the next section, we will implement a caching technique that speeds up the generation process 5-6 fold.

Text generation and inference terminology

When reading LLM literature or software documentation, you will inevitably stumble upon the term *inference*, often used in place of *text generation*. In this context, inference comes from neural network jargon and refers to using a trained model to make predictions, such as generating the next tokens from a prompt. This is different from inference in statistics, which typically means drawing conclusions about a population from data. So when we call the *generate_text_basic* function, we may be performing inference in the neural network sense.

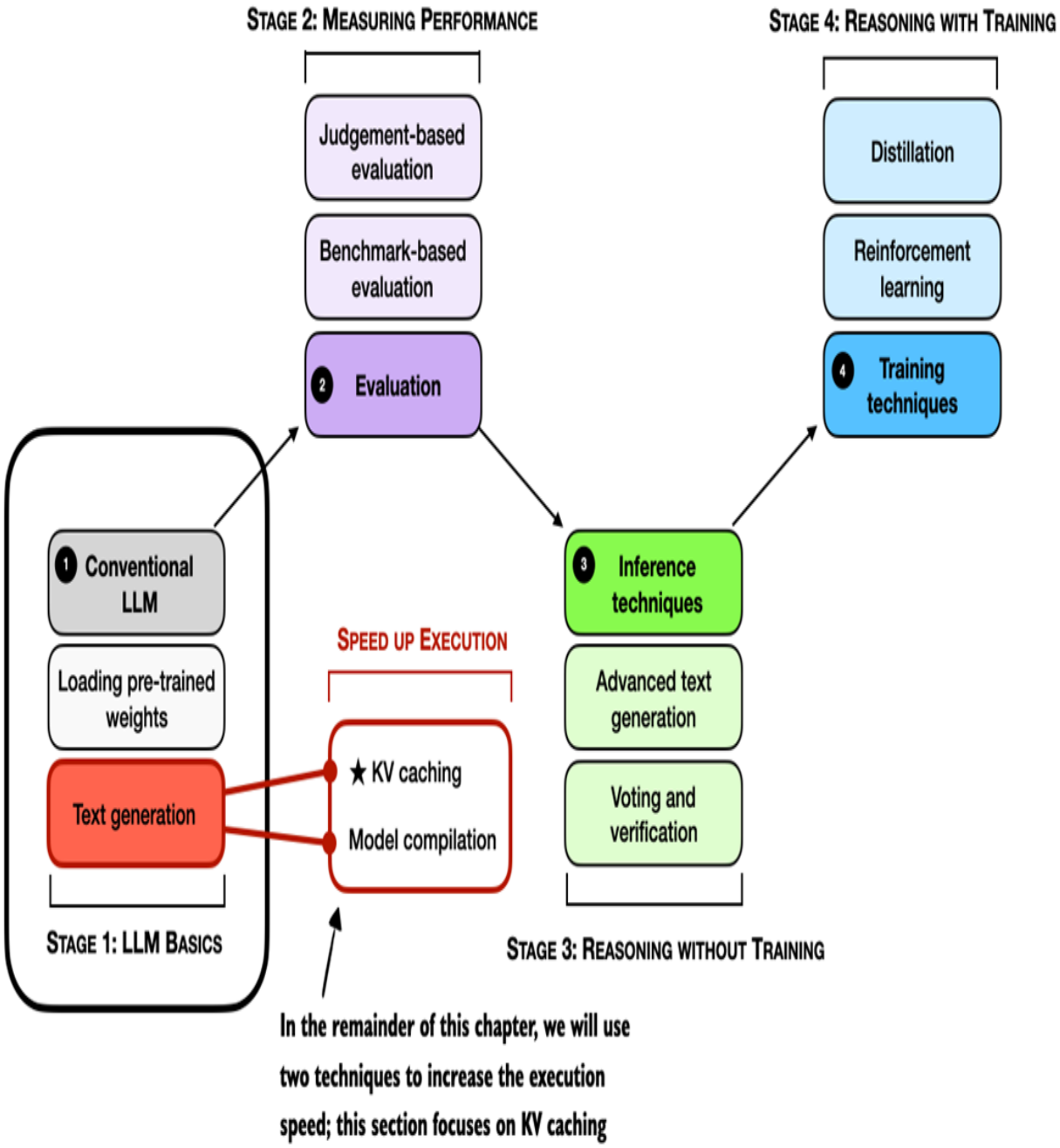
2.8 Faster inference via KV caching

So now that we have a basic text generation function in place, we can turn our attention to what happens when we actually run it in practice. As you may have noticed, the text generation in the previous section can be a bit slow. That slowdown points us to a key concern: performance during inference.

When running inference with LLMs, which in this context means generating text from a prompt, runtime performance (efficiency) quickly becomes important, especially for long sequences. While the code in this book emphasizes clarity over speed, real-world systems often use engineering tricks to make inference more efficient.

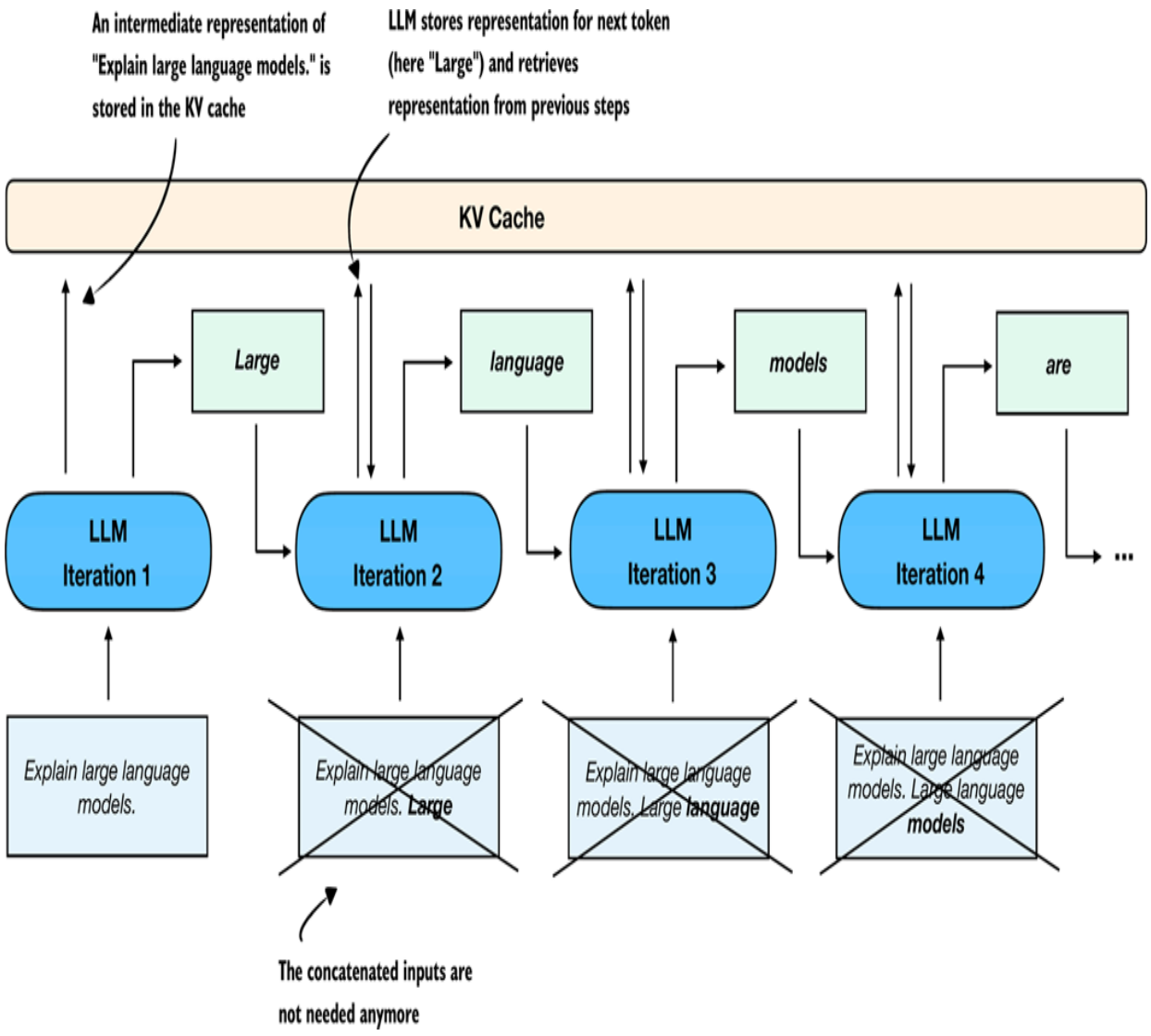
In the remaining two sections, we will cover two fundamental techniques, KV caching and model compilation, as shown in the overview in figure 2.15, to speed up the text generation.

Figure 2.15 An overview of the four key stages in developing a reasoning model in this book. This section builds on pre-trained LLM and the basic text generation function we coded earlier and applies KV caching to speed up execution.



As shown in figure 2.15, One engineering trick that increases the text generation speed is *KV caching*, where KV refers to the keys and values used in the model's attention mechanism. If you are not familiar with these terms, that's okay. The key idea is that we can cache certain intermediate values and reuse them at each step of text generation, as shown in figure 2.16, which helps speed up inference.

Figure 2.16 Illustration of how a KV cache improves efficiency during autoregressive text generation. Instead of reprocessing the entire input sequence at each step, the KV cache stores intermediate representations so that the LLM can reuse them to generate the next token. This eliminates the need to concatenate the generated token with prior inputs in each subsequent iteration.



The key idea of KV caching, as shown in figure 2.16, is to store intermediate values computed in each iteration in a cache. Previously, each new token generated by the network was concatenated to the entire input sequence and fed back into the model repeatedly (indicated by crossed-out boxes in the diagram). This approach was inefficient because all tokens, except the newly generated one, remain identical in subsequent iterations. By using a KV

cache, we avoid redundant computation and instead directly retrieve stored intermediate representations.

As mentioned earlier, the non-reasoning focused LLM details like KV caching, which we used to improve the token generation speed, are outside the scope of this book, and they are not required for the topics covered later in this book. However, interested readers can find more information on the mechanics of KV caching in my freely available article: *Understanding and Coding the KV Cache in LLMs from Scratch* (https://magazine.sebastianraschka.com/p/coding-the-kv-cache-in-llms).

Below is a modified version of the `generate_text_basic` function that incorporates a KV cache, which is almost identical to the basic text generation function in listing 2.1, except for the KV cache-related change highlighted via the comments:

Listing 2.2 A basic text generation function with KV cache

```
from reasoning_from_scratch.qwen3 import KVCache

@torch.inference_mode()
def generate_text_basic_cache(
    model,
    token_ids,
    max_new_tokens,
    eos_token_id=None
):

    input_length = token_ids.shape[1]
    model.eval()
    cache = KVCache(n_layers=model.cfg["n_layers"])         #A
    model.reset_kv_cache()
    out = model(token_ids, cache=cache)[:, -1]    #B

    for _ in range(max_new_tokens):
        next_token = torch.argmax(out, dim=-1, keepdim=True)

        if (eos_token_id is not None
                and torch.all(next_token == eos_token_id)):
            break

        token_ids = torch.cat([token_ids, next_token], dim=1)
        out = model(next_token, cache=cache)[:, -1]         #C
```

```
    return token_ids[:, input_length:]
```

The `generate_text_basic_cache` function in listing 2.2 differs only slightly from the `generate_text_basic` function in listing 2.1. The main difference is the introduction of a `KVCache` object.

During the first iteration, the model is given the full input token sequence as before, using `model(token_ids, cache=cache)`. Behind the scenes, the KV cache stores intermediate values for all these input tokens.

In the following iterations, we no longer need to pass the entire sequence. Instead, we only provide the `next_token` to the model using `model(next_token, cache=cache)`. The model then retrieves the necessary context from the previously stored KV cache.

Let's time this function to see whether it provides any performance benefits:

```
start_time = time.time()
output_token_ids_tensor = generate_text_basic_cache(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=max_new_tokens,
    eos_token_id=tokenizer.eos_token_id,
)
end_time = time.time()
generate_stats(output_token_ids_tensor, tokenizer, start_time,
end_time)
```

The output is:

```
Time: 1.40 sec
29 tokens/sec

 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
writing
articles, and even creating creative content.
```

As we can see, this approach is significantly faster, generating 29 tokens per second compared to just 5 tokens per second previously (measured on a Mac Mini M4 CPU).

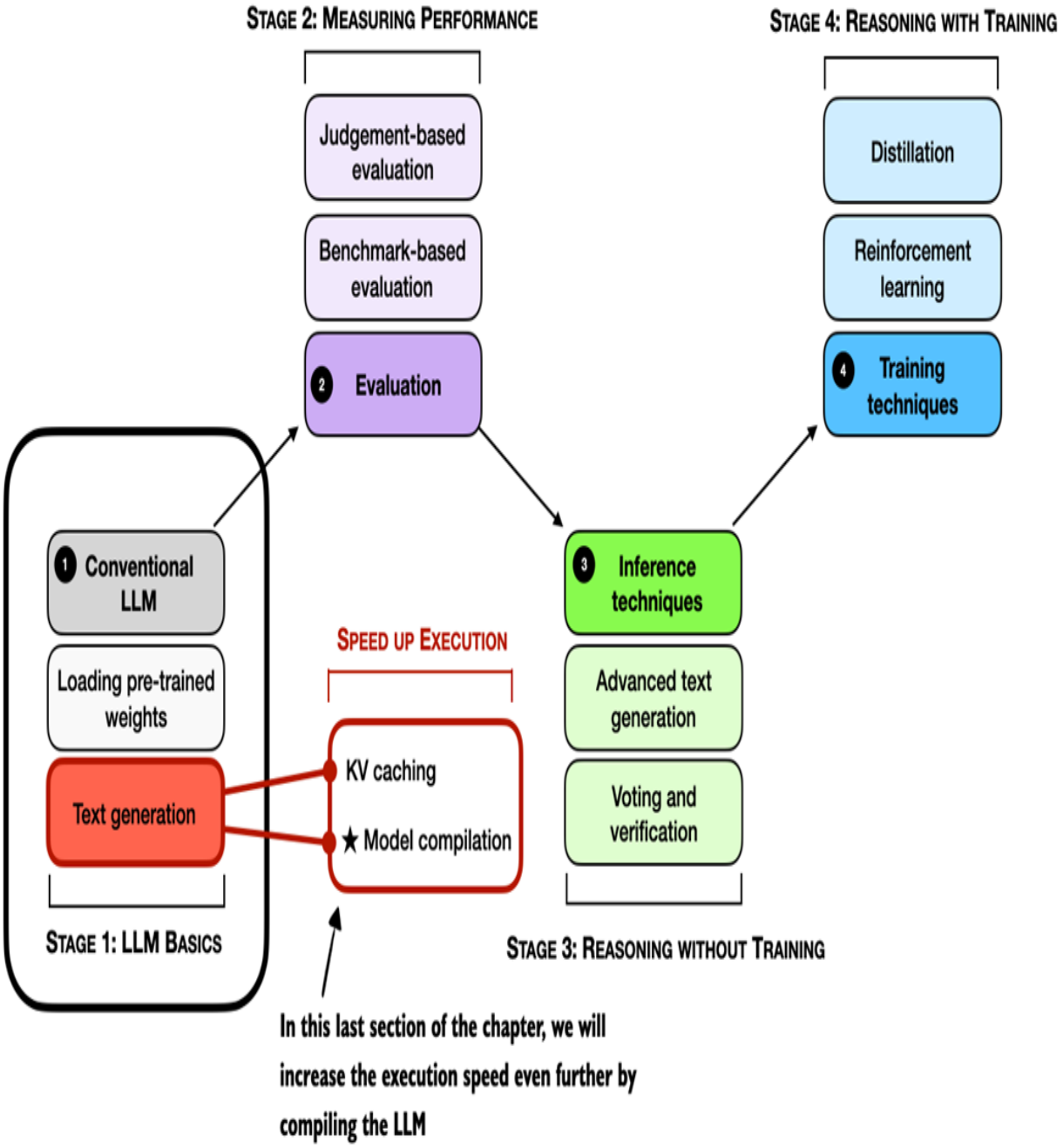
Importantly, we also see that the generated text is the same as before, which is an important sanity check to ensure that the KV cache is implemented and used correctly.

In the next section, we will learn about another technique we can use to further improve the generation speed, which will come in handy when we evaluate the model in the upcoming chapters. Faster generation allows us to run more evaluations in less time and makes it easier to compare different models or settings efficiently.

2.9 Faster inference via PyTorch model compilation

In the previous section, we covered KV caching as a technique to improve runtime efficiency as shown in the overview in figure 2.17.

Figure 2.17 An overview of the four key stages in developing a reasoning model in this book. This section builds on pre-trained LLM and the basic text generation function we coded earlier, including KV caching, and adds model compilation to speed up the execution speed even further.



As shown in figure 2.17, in this remaining section of this chapter, we will apply another technique that can substantially speed up model inference: model compilation using `torch.compile`. This feature allows the model to be compiled ahead of time, which reduces overhead and improves runtime performance during text generation.

At the time of writing, however, `torch.compile` is not well supported on MPS devices (Apple Silicon GPUs). Attempting to use it on such hardware will result in an `InductorError` for the `Qwen3Model`.

To maintain compatibility across devices, we check the hardware type and apply `torch.compile` only when it is supported:

```
if device.type == "mps":
    print("`torch.compile` is not supported"
          f" for the {model.__class__.__name__} model"
          " on MPS (Apple Silicon) as of this writing."
    )
    model_compiled = model  #A
else:
    major, minor = map(int, torch.__version__.split(".")[:2])
    if (major, minor) >= (2, 8):
        # This avoids retriggering model recompilations
        # in PyTorch 2.8 and newer
        # if the model contains code like self.pos = self.pos +
1
        torch._dynamo.config.allow_unspec_int_on_nn_module =
True

    model_compiled = torch.compile(model)
```

We assign the compiled model to a new variable so that the code in this chapter continues to function properly.

It is worth noting that the first execution using the compiled model may be slower than usual due to the initial compilation and optimization steps. To better measure the performance improvement, we will repeat the text generation process multiple times.

To begin, we will test this using the non-cached version of the generation function. The code is similar to what we used before except that we run it three times in a row. The code execution may take a few minutes to finish, depending on the system:

```
for i in range(3): #A
    start_time = time.time()
    output_token_ids_tensor = generate_text_basic(
        model=model_compiled,
        token_ids=input_token_ids_tensor,
```

```
        max_new_tokens=max_new_tokens,
        eos_token_id=tokenizer.eos_token_id
    )
    end_time = time.time()

    if i == 0:  #B
        print("Warm-up run")  #B
    else:
        print(f"Timed run {i}:")
    generate_stats(output_token_ids_tensor, tokenizer,
start_time, end_time)

    print(f"\n{30*'-'}\n")
```

The output is as follows:

```
Warm-up run
Time: 11.68 sec
3 tokens/sec

 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
writing
articles, and even creating creative content.

------------------------------

Timed run 1:
Time: 6.78 sec
6 tokens/sec
Output text:

 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
writing
articles, and even creating creative content.

------------------------------

Timed run 2:
Time: 6.80 sec
6 tokens/sec
```

```
Output text:

 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
writing
articles, and even creating creative content.

------------------------------
```

As we can see from the results above, the compiled model achieves a slight improvement in speed, with around 6 tokens per second compared to the previous 5 tokens per second.

Next, let's see how the KV cache version performs in comparison, using the same code as before except for swapping `generate_text_basic` with `generate_text_basic_cache`:

```
for i in range(3):
    start_time = time.time()
    output_token_ids_tensor = generate_text_basic_cache(
        model=model_compiled,
        token_ids=input_token_ids_tensor,
        max_new_tokens=max_new_tokens,
        eos_token_id=tokenizer.eos_token_id
    )
    end_time = time.time()

    if i == 0:
        print("Warm-up run")
        generate_stats(
        output_token_ids_tensor, tokenizer, start_time, end_time
    )
    else:
        print(f"Timed run {i}:")
        generate_stats(output_token_ids, tokenizer, start_time,
end_time)

    print(f"\n{30*'-'}\n")
```

The output is as follows:

```
Warm-up run
Time: 8.07 sec
5 tokens/sec

 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
writing
articles, and even creating creative content.

------------------------------

Timed run 1:
Time: 0.60 sec
68 tokens/sec
Output text:

 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
writing
articles, and even creating creative content.

------------------------------

Timed run 2:
Time: 0.60 sec
68 tokens/sec
Output text:

 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
writing
articles, and even creating creative content.

------------------------------
```

As we can see based on the outputs above, the model generation speed improved from 29 tokens per second for the uncompiled model with KV

cache to 68 tokens per second when the same model is compiled (on a Mac Mini M4 CPU), which is more than a 2-fold speed-up.

Exercise 2.3: Rerun code on non-CPU devices

If you have access to a GPU, rerun the code in this chapter on a GPU device and compare the runtimes to the CPU runtimes.

In case you are curious, how the different model configurations compare on an Apple Silicon GPU and a high-end NVIDIA GPU, see table 2.1.

Table 2.1 Token generation speeds and GPU memory usage for different model configurations on different hardware

Mode	Hardware	Tokens/sec	GPU memory
Regular	Mac Mini M4 CPU	5	-
Regular compiled	Mac Mini M4 CPU	6	-
KV cache	Mac Mini M4 CPU	28	-
KV cache compiled	Mac Mini M4 CPU	68	-
Regular	Mac Mini M4 GPU	17	-
Regular compiled	Mac Mini M4 GPU	InductorError	-
KV cache	Mac Mini M4 GPU	18	-

KV cache compiled	Mac Mini M4 GPU	InductorError	-
Regular	NVIDIA H100 GPU	51	1.55 GB
Regular compiled	NVIDIA H100 GPU	164	1.81 GB
KV cache	NVIDIA H100 GPU	48	1.52 GB
KV cache compiled	NVIDIA H100 GPU	141	1.81 GB

As shown in the table above, the NVIDIA GPU delivers the best performance, which is expected. However, the CPU also performs remarkably well when using both a KV cache and a compiled model. It's worth noting that this is a relatively small model, and I optimized the KV-cache implementation for CPUs to ensure accessibility for most readers. With a larger model or GPU-optimized code, the performance gap in favor of the NVIDIA GPU would likely be more pronounced.

Also, keep in mind that performance can vary with longer input sequences since the cost of the LLM-internal attention mechanism scales quadratically with the input length.

All examples were run using a single prompt (i.e., a batch size of 1). For readers interested in how performance scales with multiple inputs, batched inference is discussed in appendix E.

2.10 Summary

- Using LLMs to generate text involves multiple key steps:
    - Setting up the coding environment to run LLM code and install necessary dependencies.

    - Loading a pre-trained base LLM (such as Qwen3 0.6B), which will be extended with reasoning capabilities in later chapters.
    - Initializing and using a tokenizer, which converts text input into token IDs and decodes output back to human-readable form.
- Text generation in LLMs follows a sequential (autoregressive) process, where the model generates one token at a time by predicting the next most likely token.
- The speed and efficiency of text generation can be improved through:
    - KV caching, which stores intermediate states to avoid recomputing previously encountered input tokens at each step.
    - Model compilation using torch.compile, which optimizes runtime performance.
- This chapter lays the technical foundation for reasoning capabilities in upcoming chapters by implementing a functional, efficient text generation pipeline using a pre-trained base LLM.

OceanofPDF.com

3 Evaluating reasoning models

This chapter covers

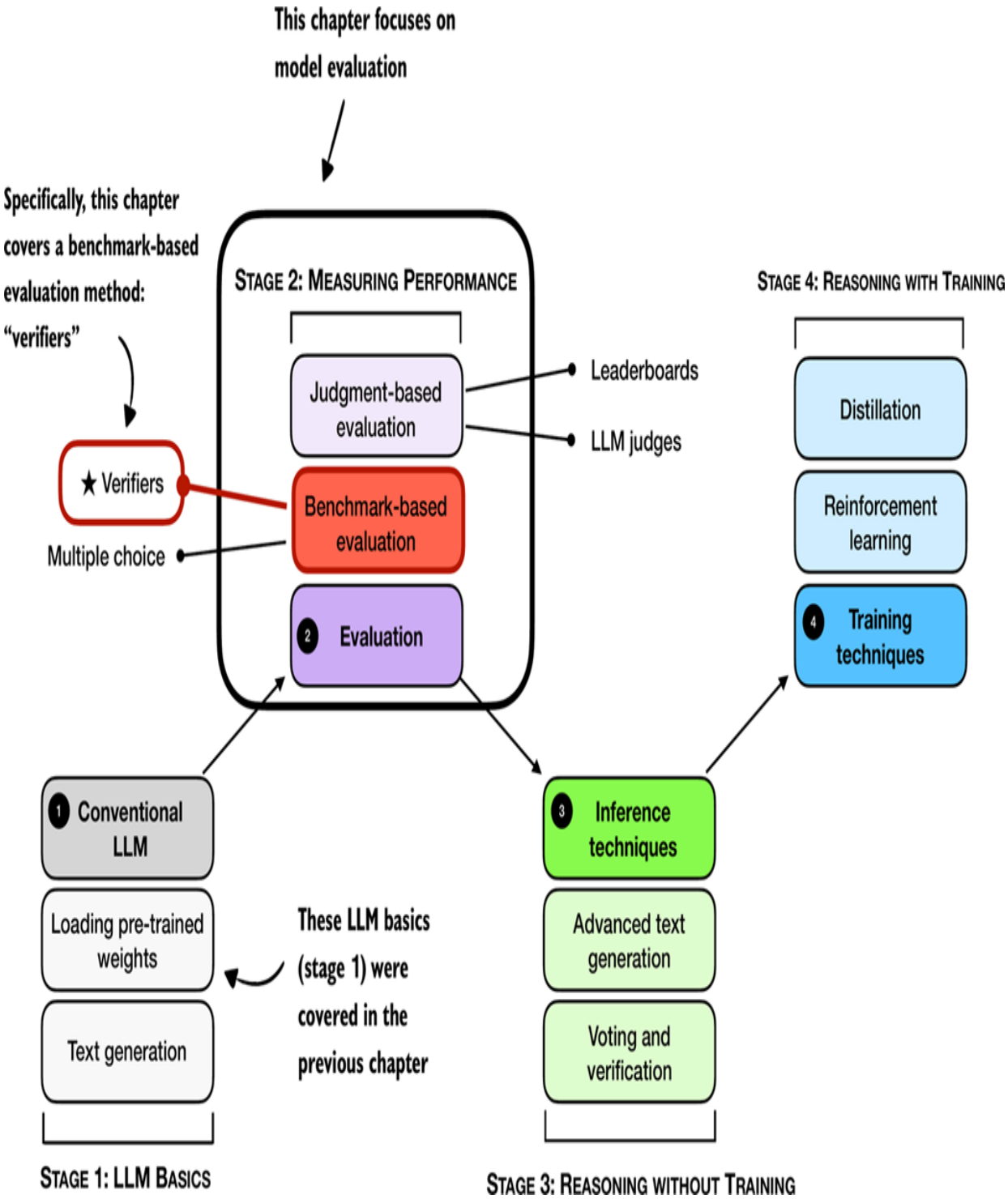
- Extracting final answers reliably from an LLM response
- Verifying answer correctness by comparing an LLM's output to the reference solution using a symbolic math solver
- Running a full evaluation pipeline by loading a pre-trained model, generating outputs, and grading them against a dataset

Evaluation is what lets us distinguish between LLMs that merely sound convincing and those that can solve problems correctly. LLM evaluation techniques span a broad range of approaches, from measuring task accuracy to making sure that LLMs adhere to specific safety standards.

In this chapter, we focus on implementing a *verification*-based method that checks whether an LLM can solve math problems accurately by comparing its own answers against reference solutions using a calculator-like implementation.

This verifier is particularly useful because it not only evaluates performance on math tasks but also introduces the principle of *verifiable rewards*, which is the foundation of the reinforcement learning approach to reasoning models that we will implement later in chapter 5. (Interested readers can find additional evaluation methods in appendix F.)

Figure 3.1 A mental model of the topics covered in this book. This chapter covers evaluation methods (stage 2), with a special focus on implementing verifiers.



3.1 Building a math verifier

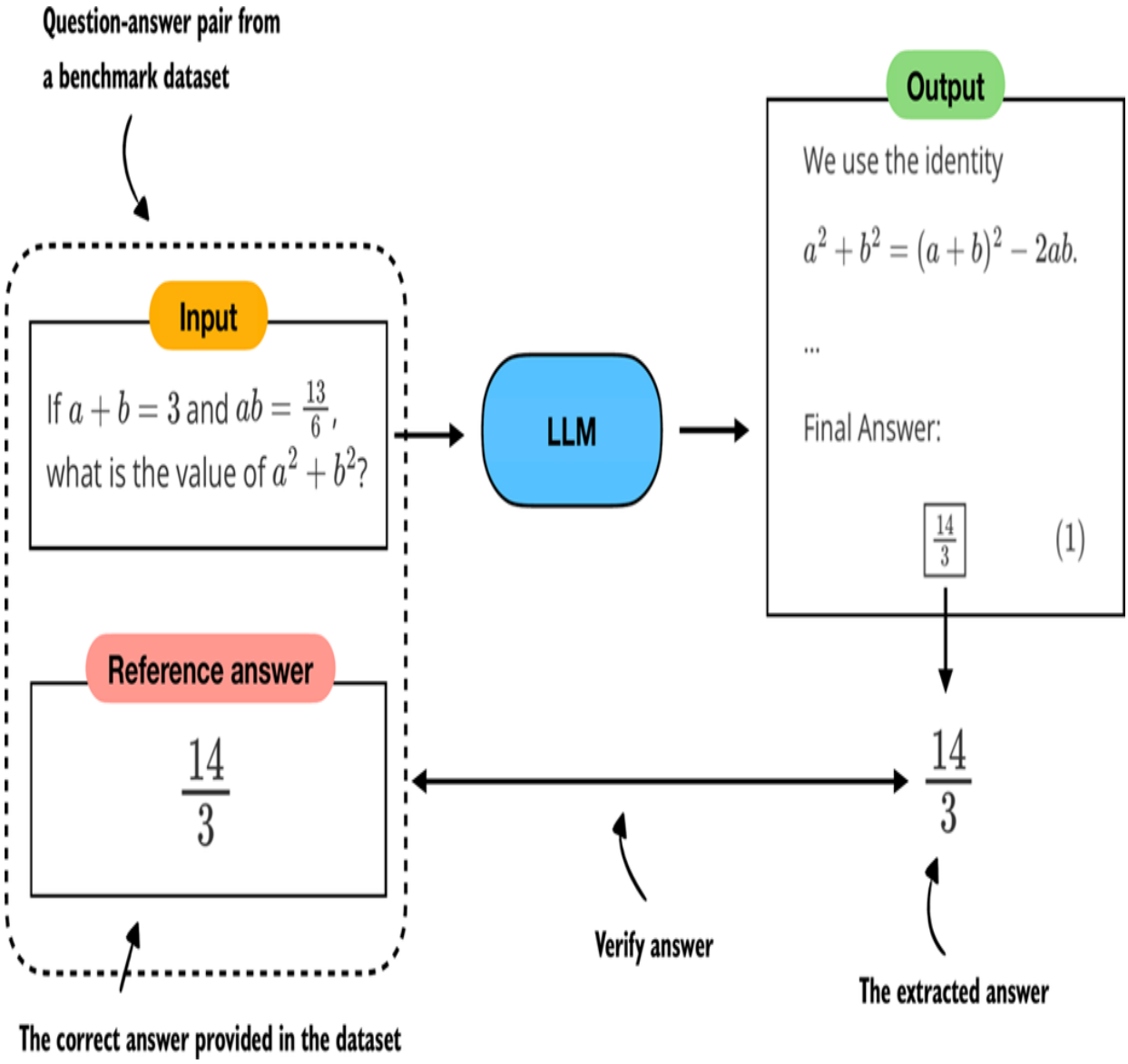
There are four common ways of evaluating trained LLMs in practice: *multiple choice*, *verifiers*, *leaderboards*, and *LLM judges*, as shown in figure 3.1. These methods are widely used across research papers, technical reports, marketing materials, and model cards, and results often draw from more than one category.

As figure 3.1 illustrates, these evaluation approaches can be grouped into two broader types: *benchmark-based evaluation* and *judgment-based evaluation*. All four evaluation methods are useful in different contexts, but verifiers are especially relevant for reasoning models.

Math problems provide a natural example: depending on the problem complexity, math problems benefit from step-by-step reasoning to solve, yet evaluation is straightforward because the final answer can be checked against a correct answer. In this setting, the verifier approach provides a simple and reliable way to measure whether a model's reasoning steps lead to the correct outcome.

In this chapter, we focus on verifiers as a benchmark-based approach for measuring answer correctness in math problems, as illustrated in figure 3.2.

Figure 3.2 Evaluating an LLM with a verification-based method in free-form question answering. The model generates a free-form answer (which may include multiple steps) and a final boxed answer, which is extracted and compared against the correct answer from the dataset.



Verifiers compare the extracted answer with the reference solution, as shown in figure 3.2, often by relying on external tools such as code interpreters or calculator programs.

While our immediate focus is evaluation, verifiers will reappear later in this book. They not only serve as a way to measure performance but also provide the feedback signal used in reinforcement learning methods for training reasoning models, which we will explore in chapter 5.

Tip

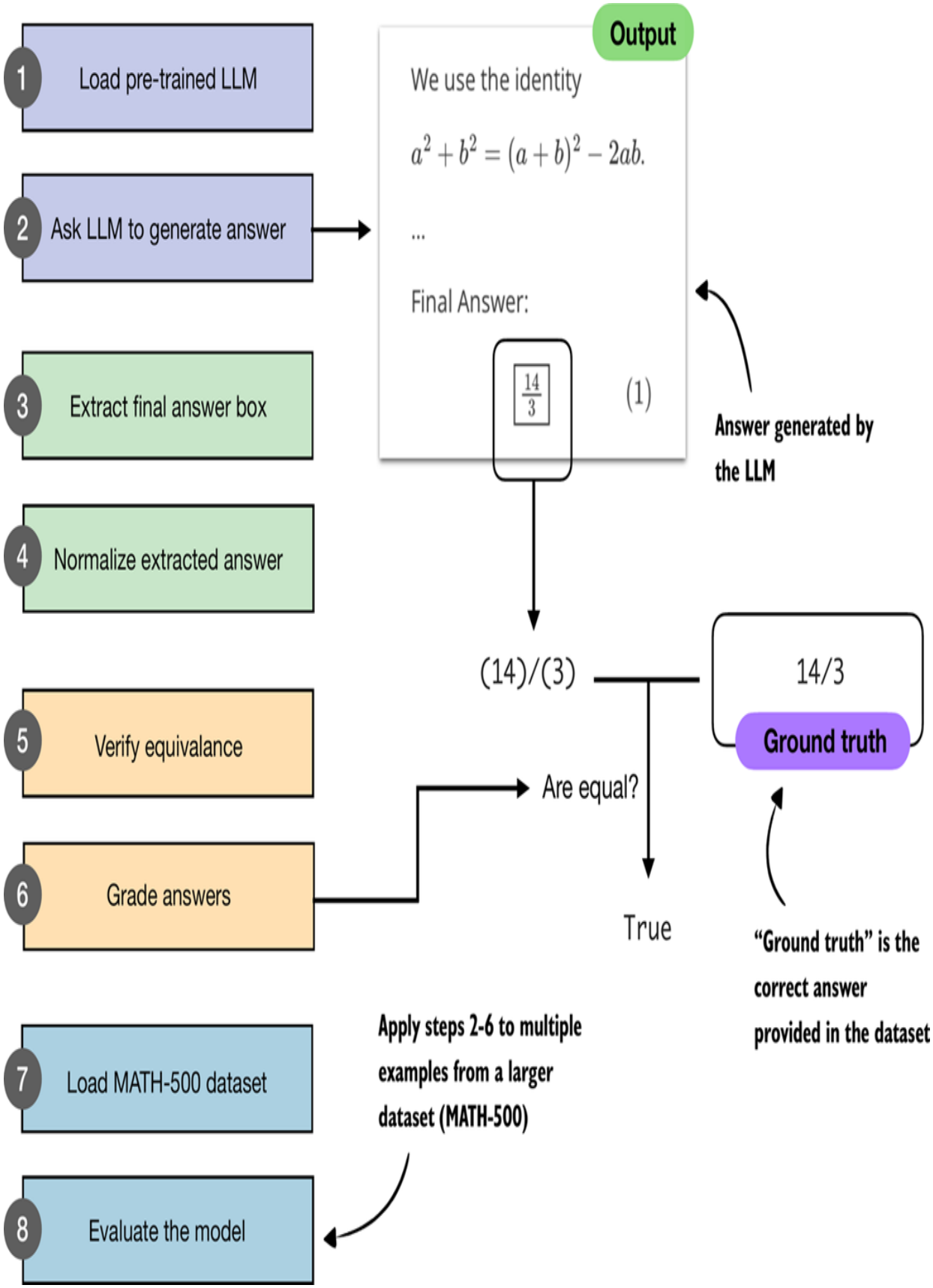
For readers interested in going further, appendix F covers other evaluation methods such as multiple-choice benchmarks, preference-based leaderboards, and LLM-as-a-judge approaches.

The downside is that verifier methods can only be applied to domains that can be easily (and ideally deterministically) verified, such as math and code. Also, this approach can introduce additional complexity and dependencies, and it may shift part of the evaluation burden from the model itself to the external tool.

However, because math problem solving can be generated in unlimited variations programmatically and benefits from step-by-step reasoning, this task has become a cornerstone of reasoning model evaluation and development.

In the remainder of this chapter, we will build a math verifier step by step, following the 8 steps shown in figure 3.3.

Figure 3.3 A step-by-step workflow for building and applying a math verifier. Starting with a pre-trained LLM, we generate answers, extract and normalize them, and then compare them against the ground-truth solutions. Verified answers are then graded, and the process is repeated across a dataset (MATH-500) to evaluate overall model performance.


1 Load pre-trained LLM
2 Ask LLM to generate answer
3 Extract final answer box
4 Normalize extracted answer
5 Verify equivalance
6 Grade answers
7 Load MATH-500 dataset
8 Evaluate the model
Output
We use the identity
$a^2 + b^2 = (a + b)^2 - 2ab.$
...
Final Answer:
$\frac{14}{3}$
(1)
Answer generated by the LLM
(14)/(3)
14/3
Ground truth
Are equal?
True
"Ground truth" is the correct answer provided in the dataset
Apply steps 2-6 to multiple examples from a larger dataset (MATH-500)

The next section will start with steps 1 and 2 shown in figure 3.3, namely, loading the pre-trained LLM introduced in the previous chapter and setting it up to generate answers.

3.2 Loading a pre-trained model to generate text

In this section, we begin implementing the verifier by following steps 1 and 2 of the workflow in figure 3.3. Specifically, we will load the pre-trained LLM introduced in the previous chapter and configure it to generate answers. This provides the foundation for the later steps, where we will extract, normalize, and verify these answers.

Specifically, we use the same pre-trained base model that we used in chapter 2. However, once you have completed this chapter, you can rerun the notebook after changing `WHICH_MODEL = "base"` to `WHICH_MODEL = "reasoning"` in listing 3.1 to evaluate an already trained reasoning model on the same dataset.

Listing 3.1 Loading a pre-trained model

```
from pathlib import Path
import torch
from reasoning_from_scratch.ch02 import get_device
from reasoning_from_scratch.qwen3 import (
    download_qwen3_small, Qwen3Tokenizer,
    Qwen3Model, QWEN_CONFIG_06_B
)

device = get_device()
torch.set_float32_matmul_precision("high")  #A

# device = "cpu"  #B

WHICH_MODEL = "base"  #C

if WHICH_MODEL == "base":
    download_qwen3_small(
        kind="base", tokenizer_only=False, out_dir="qwen3"
    )
    tokenizer_path = Path("qwen3") / "tokenizer-base.json"
    model_path = Path("qwen3") / "qwen3-0.6B-base.pth"
    tokenizer =
```

```
Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)

elif WHICH_MODEL == "reasoning":
    download_qwen3_small(
        kind="reasoning", tokenizer_only=False, out_dir="qwen3"
    )
    tokenizer_path = Path("qwen3") / "tokenizer-reasoning.json"
    model_path = Path("qwen3") / "qwen3-0.6B-reasoning.pth"
    tokenizer = Qwen3Tokenizer(
        tokenizer_file_path=tokenizer_path,
        apply_chat_template=True,
        add_generation_prompt=True,
        add_thinking=True,
    )

else:
    raise ValueError(f"Invalid choice: WHICH_MODEL=
{WHICH_MODEL}")

model = Qwen3Model(QWEN_CONFIG_06_B)
model.load_state_dict(torch.load(model_path))
model.to(device)

USE_COMPILE = False  #D
if USE_COMPILE:
  torch._dynamo.config.allow_unspec_int_on_nn_module = True
  model = torch.compile(model)
```

By default, listing 3.1 loads the base model, just as in chapter 2. An optional variant is the reasoning model, which the Qwen3 team trained on top of the base model using reasoning-specific methods. We will cover these training methods in chapter 5. Here, the reasoning model, which can be loaded by setting `WHICH_MODEL = "reasoning"` in listing 3.1, is included as an option so that we can later compare its evaluation results with those of the base model.

Now that we have loaded the model, we can use the text generation function from chapter 2 to generate text. However, instead of the `generate_text_basic_stream` function introduced in the main chapter, we use the slightly modified `generate_text_basic_stream_cache` version from exercise 2.2 (see the solution in appendix B for the source code) as it prints the tokens as soon as they are generated, which can be useful for

debugging purposes (so it doesn't appear the LLM is stuck when generating a longer response). The usage of this function is shown in listing 3.2.

Listing 3.2 Generating model outputs

```
from reasoning_from_scratch.ch02_ex import (
    generate_text_basic_stream_cache
)

prompt = (  #A
    r"If $a+b=3$ and $ab=\tfrac{13}{6}$, "
    r"what is the value of $a^2+b^2$?"
)

input_token_ids_tensor = torch.tensor(  #B
    tokenizer.encode(prompt),
    device=device
).unsqueeze(0)  #C

all_token_ids = []

for token in generate_text_basic_stream_cache(  #D
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=2048,
    eos_token_id=tokenizer.eos_token_id
):
    token_id = token.squeeze(0)  #E
    decoded_id = tokenizer.decode(token_id.tolist())
    print(  #F
        decoded_id,
        end="",
        flush=True
    )
    all_token_ids.append(token_id)

#G
all_tokens = tokenizer.decode(all_token_ids)
```

In listing 3.2, we start by encoding a simple math problem into token IDs that the model can process. The model then generates tokens one by one in a streaming fashion, which we print immediately as they appear so we can read the output while it's being generated. At the same time, we collect the generated tokens into a list so that we can later decode them into the

complete final answer string. This pattern of both streaming and collecting tokens is handy because it lets us monitor the generation live while still storing the full answer text (`all_tokens`) that we can process later.

The response, generated by the code in listing 3.2, is as follows:

```
To find the value of \( a^2 + b^2 \) given that \( a + b = 3 \)
and \( ab = \frac{13}{6} \), we can use the following algebraic
identity:

\[
a^2 + b^2 = (a + b)^2 - 2ab
\]

**Step 1:** Substitute the given values into the equation.

\[
a^2 + b^2 = (3)^2 - 2 \left( \frac{13}{6} \right)
\]

[...]  #A

**Final Answer:**

\[
\boxed{\dfrac{14}{3}}
\]
```

As we can see, based on this answer, even though it is a base model, it provides a reasoning model-like explanation. This is likely because the Qwen3 team included chain-of-thought data during the pre-training stages, as stated in their technical report. However, even though the model has some reasoning-model-like behavior, adding additional reasoning methods can further improve these capabilities. (Note that the response may differ depending on whether you executed the code on a CPU, CUDA, or MPS device.)

Furthermore, if you are unfamiliar with the LaTeX syntax that is commonly used for mathematics, the response above can be very hard to decipher. If this is the case, you can use IPython's `Latex` class to render it, as shown below:

```
from IPython.display import Latex, display
display(Latex(all_tokens))
```

Executing the code above in a code notebook will render the response as shown in figure 3.4.

Figure 3.4 Rendered response with step-by-step calculations and the final boxed answer.

"**" symbols, used for bold font in Markdown syntax, are not part of the Latex syntax and are thus not rendered

Step 1: Substitute the given values into the equation.

$$a^2 + b^2 = (3)^2 - 2\left(\frac{13}{6}\right)$$

Step 2: Calculate $(3)^2$.

$$(3)^2 = 9$$

Step 3: Calculate $2 \times \frac{13}{6}$.

$$2 \times \frac{13}{6} = \frac{26}{6} = \frac{13}{3}$$

Latex code is properly rendered

Step 4: Subtract the second result from the first.

$$a^2 + b^2 = 9 - \frac{13}{3}$$

Step 5: Convert 9 to a fraction with a denominator of 3 to perform the subtraction.

$$9 = \frac{27}{3}$$

$$a^2 + b^2 = \frac{27}{3} - \frac{13}{3} = \frac{14}{3}$$

Final Answer:

$$\boxed{\frac{14}{3}}$$

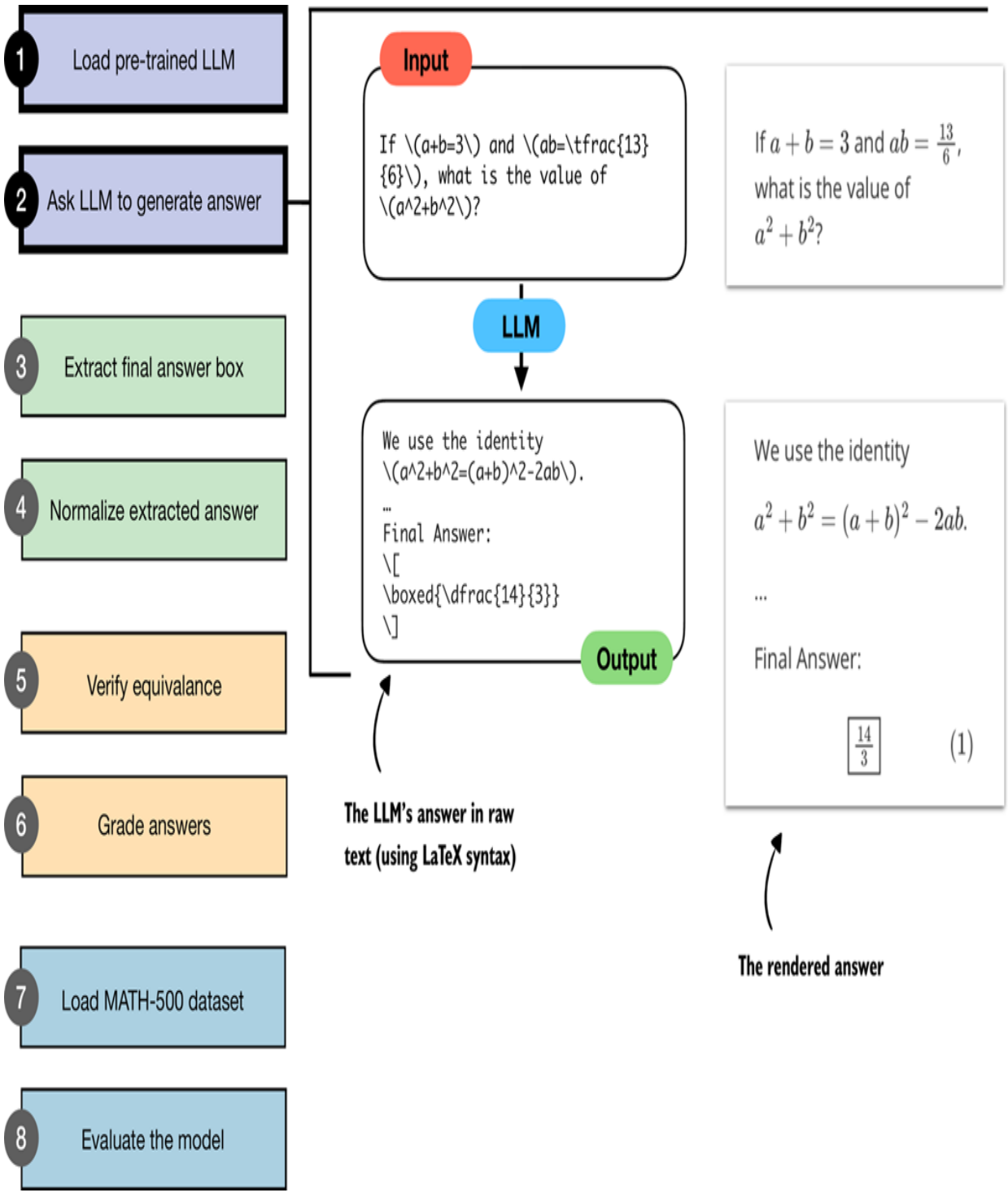
The final answer is given in an answer box

Note that the final answer given in figure 3.4, 14/3 is indeed the correct answer to this problem.

3.3 Implementing a wrapper for easier text generation

In the previous section, we loaded the pre-trained LLM and set up the text generation functionality (as illustrated in figure 3.5), which are the first two steps of the evaluation process covered in the remainder of this chapter.

Figure 3.5 Illustration of steps 1 and 2 from the verifier workflow. A pre-trained LLM is loaded and prompted with a math problem, producing an output in raw LaTeX syntax. The answer is also shown in the rendered and more readable form.



For additional convenience in later sections, we create a wrapper (listing 3.3) for the text generation function so that we only have to pass in the model, tokenizer, and prompt, along with some additional settings instead of repeating the tokenization and input preparation steps each time:

Listing 3.3 A wrapper for streamed text generation

```
def generate_text_stream_concat(
    model, tokenizer, prompt, device, max_new_tokens,
    verbose=False,
):
    input_ids = torch.tensor(                    #A
        tokenizer.encode(prompt), device=device  #A
        ).unsqueeze(0)                           #A

    generated_ids = []
    for token in generate_text_basic_stream_cache(  #B
        model=model,                                #B
        token_ids=input_ids,                        #B
        max_new_tokens=max_new_tokens,              #B
        eos_token_id=tokenizer.eos_token_id,        #B
    ):                                              #B
        next_token_id = token.squeeze(0)
        generated_ids.append(next_token_id.item())


        if verbose: #C
            print(
                tokenizer.decode(next_token_id.tolist()),
                end="",
                flush=True
            )

    return tokenizer.decode(generated_ids)  #D
```

This wrapper function in listing 3.3 handles the full cycle of text generation: it tokenizes the input prompt, streams new tokens from the model, and then decodes the results into a final string. And, as mentioned before, the optional verbose flag allows us to see tokens as they are generated in real time. The function can be used as follows:

```
generated_text = generate_text_stream_concat(
    model, tokenizer, prompt, device,
    max_new_tokens=2048,
    verbose=True  #A
)
```

This prints the exact same response as in section 3.2:

```
[...]  #A
```

```
**Final Answer:**

\[
\boxed{\dfrac{14}{3}}
\]
```

3.4 Extracting the final answer box

Now that we have the model loaded and ready, we can get to the chapter-specific and interesting parts: evaluating the model.

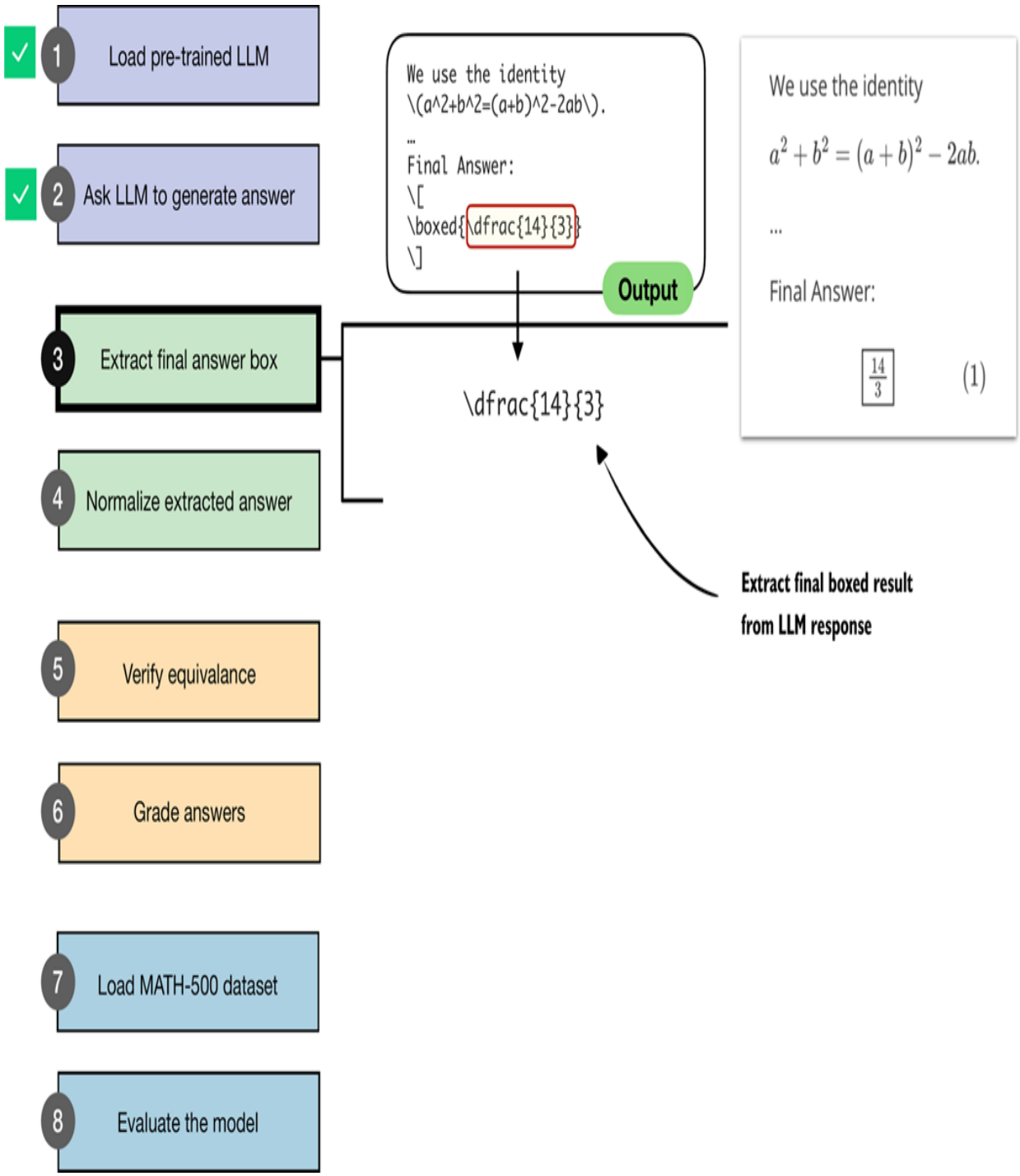
In the previous section, we saw that the model returned the final answer in an answer box (written as `r"\boxed{\dfrac{14}{3}}"` in raw text), even though we hadn't specifically asked for this format.

The reason the model answered in this specific format is likely because the model has seen examples from benchmark datasets (including MATH-500) that were similarly formatted during pretraining. As a general rule, it is fair to assume that any information available on the internet when a model was trained has been part of the training data.

Although it was not necessary here, when we evaluate the model in the MATH-500 dataset later on, we will add a specific prompt that instructs the model to return answers in this boxed form, as it is a common convention that makes the evaluation more consistent across different models and makes data extraction easier.

We will now write code that performs this extraction of the boxed answer content as illustrated in figure 3.6.

Figure 3.6 An illustration of how the boxed result from the LLM output is extracted.



Specifically, this section implements step 3 shown in figure 3.6. The next section will implement the normalization method for step 4.

Since your model may produce slightly different responses (depending on your hardware) than those shown above, we will work with a hard-coded

answer for the time being (pretending that this answer was generated by the model). In later sections, we will revisit the model and have it generate answers for the tasks in the MATH-500 dataset.

```
model_answer = (
r"""... some explanation...
**Final Answer:**

\[                      #A
\boxed{\dfrac{14}{3}}  #A
\]                      #A
""")
```

Note

We are using a raw string (`r"""..."""` instead of a regular string `"""..."""`). Raw strings make it easier to handle the `\` characters, which would otherwise be treated as escape sequences and require doubling each backslash.

Next, let's define a function in listing 3.4 to extract the answer box from the `model_answer`.

Listing 3.4 Extracting answer boxes

```
def get_last_boxed(text):
    boxed_start_idx = text.rfind(r"\boxed")  #A
    if boxed_start_idx == -1:
        return None

    current_idx = boxed_start_idx + len(r"\boxed")  #B

    #C
    while current_idx < len(text) and
text[current_idx].isspace():
        current_idx += 1

    #D
    if current_idx >= len(text) or text[current_idx] != "{":
        return None

    current_idx += 1
    brace_depth = 1
    content_start_idx = current_idx
```

```
    #E
    while current_idx < len(text) and brace_depth > 0:
        char = text[current_idx]
        if char == "{":
            brace_depth += 1
        elif char == "}":
            brace_depth -= 1
        current_idx += 1


    if brace_depth != 0:  #F
        return None


    return text[content_start_idx:current_idx-1]  #G
```

The get_last_box helper utility function in listing 3.4 extracts out the content of the last `\boxed{...}` expression from a model's output. More specifically, it scans for the final `\boxed`, skips over whitespace, checks for braces, and handles any nesting so that we capture the intended answer string.

While it may look a bit tedious, having this parser in place will pay off when we run evaluations on datasets like MATH-500, where extracting the correct final answer is the first step toward measuring a model's reasoning ability. (MATH-500 is a curated collection of 500 problems that is widely used as a reasoning model benchmark dataset, which we will use later in this chapter.)

Now, let's test it on the model answer:

```
extracted_answer = get_last_boxed(model_answer)
print(extracted_answer)
```

The output of this function call is `"\dfrac{14}{3}"`, which is the boxed answer we wanted to extract.

Rendering math formulas

We can render math formulas via the `Latex` class we introduced earlier. Alternatively, for single math formulas that are not accompanied by answer text, we can also use the simpler Math class:

```
from IPython.display import Math
display(Math(r"\dfrac{14}{3}"))
```

This renders the fraction as 14/3

While the previous `get_last_boxed` function correctly extracted the text, we will make the answer extraction a bit more robust to account for cases where a final answer box is either missing or incomplete via the `extract_final_candidate` function in listing 3.5:

Listing 3.5 Extracting the final answer candidate

```
import re

RE_NUMBER = re.compile(
    r"-?(?:\d+/\d+|\d+(?:\.\d+)?(?:[eE][+-]?\d+)?)"
)

def extract_final_candidate(text, fallback="number_then_full"):

    result = ""  #A

    if text:  #B
        boxed = get_last_boxed(text.strip())
        if boxed:
            result = boxed.strip().strip("$ ")

        #C
        elif fallback in ("number_then_full", "number_only"):
            m = RE_NUMBER.findall(text)
            if m:
                result = m[-1]  #D
            elif fallback == "number_then_full":

                result = text  #E
    return result
```

The `extract_final_candidate` function in listing 3.5 provides `fallback` settings in case no boxed answer can be found, which are as follows:

- `"number_then_full"` (default): pick the last simple number, else the whole text;
- `"number_only"`: pick the last simple number, else return an empty string `""`;

- `"none"`: extract only boxed content, else return empty string `""`.

For the fallback setting, the code in listing 3.5 uses *regular expressions* (regex for short) via Python's `re` library. Regexes are a way to search for patterns in text. In our case, the regex pattern is designed to recognize numbers, including fractions, decimals, and scientific notation. While the regex syntax looks intimidating, you don't need to worry about the exact syntax here. What matters is that this gives us a reliable tool to extract the last numeric candidate from the model's output when no boxed answer is available.

Let's try it on our model answer:

```
print(extract_final_candidate(model_answer))
```

This correctly returns `"\dfrac{14}{3}"`. Next, let's try some additional examples. First, another boxed candidate:

```
print(extract_final_candidate(r"\boxed{ 14/3. }"))
```

This correctly returns `"14/3."`, stripping the extra whitespace but not the punctuation. However, the punctuation character will be handled correctly by the equality check we implement later.

Next, let's try a candidate without a box, which should trigger the fallback setting, and see what happens:

```
print(extract_final_candidate("abc < > 14/3 abc"))
```

Thanks to the default fallback setting, it will find the last number in the answer and also correctly return `"14/3"`.

In this section, we defined utility functions to extract the LLM's answer from within its answer text context. This brings us one step closer to achieving the overall goal of verifying whether this answer is indeed correct. In the next section, we will normalize the response into a more general, canonical form before we implement the checking functionality.

Why not use an LLM for the answer extraction?

We could use an LLM itself to extract the boxed answer. However, this would introduce unnecessary complexity and potential errors. Extraction is a simple, mechanical task: we just need to locate the last boxed expression or, if that is missing, fall back to a number or the raw text.

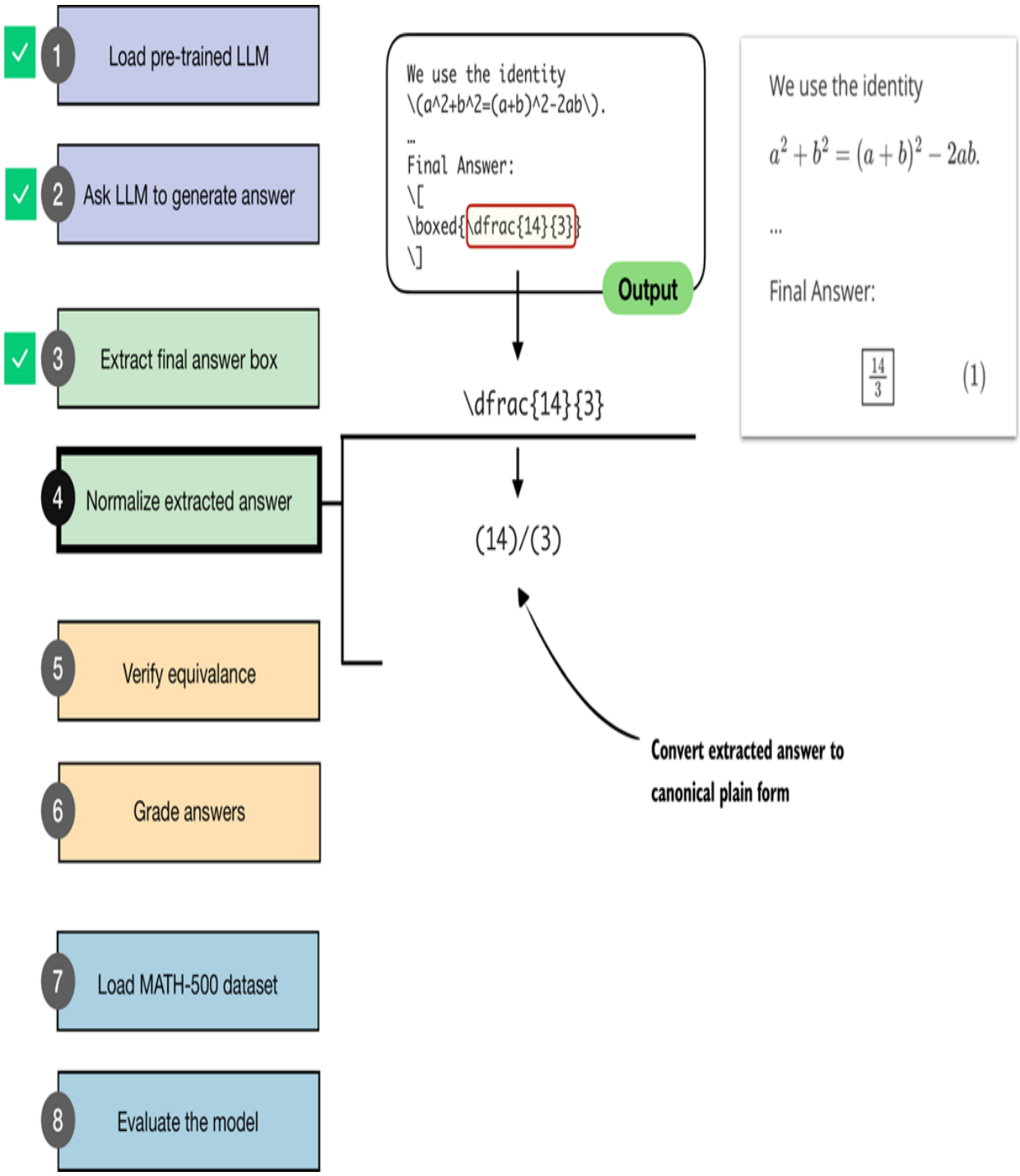
Regular expressions may look complicated at first, but in the end, we have a small, reusable utility function that is cheap to execute and handles the extraction deterministically and reproducibly, without depending on the variability of another model's output.

3.5 Normalizing the extracted answer

Previously, we extracted the boxed answer `"\dfrac{14}{3}"` from the model's response. However, models may print the same value in many ways, such as `"\frac{14}{3}"`, `"14/3"`, `"$14/3$"`, or `"(14)/(3)"`. To implement and use a robust checking system that can check whether the answer is correct, we first need a consistent method of comparing results.

In this section, we implement a normalization pass (step 4 in figure 3.7) that strips formatting and standardizes the answer.

Figure 3.7 An illustration of how the boxed result from the LLM output is extracted and converted into a canonical plain form. This normalized answer is then later used for verification against the correct answer.



The normalization step shown in figure 3.7 is implemented via the `normalize_text` function in listing 3.6.

Listing 3.6 Normalizing extracted answers

```
LATEX_FIXES = [  #A
    (r"\\left\s*", ""),
    (r"\\right\s*", ""),
    (r"\\,|\\!|\\;|\\:", ""),
    (r"\\cdot", "*"),
    (r"\u00B7|\u00D7", "*"),
    (r"\\^\\circ", ""),
    (r"\\dfrac", r"\\frac"),
    (r"\\tfrac", r"\\frac"),
    (r"°", ""),
]

RE_SPECIAL = re.compile(r"<\|[^>]+?\|>")  #B

def normalize_text(text):
    if not text:
        return ""
    text = RE_SPECIAL.sub("", text).strip()

    text = re.sub(r"\^\s*\{\s*\\circ\s*\}", "", text)  #C
    text = re.sub(r"\^\s*\\circ", "", text)            #C
    text = text.replace("°", "")                       #C

    match = re.match(r"^\\text\{(?P<x>.+?)\}$", text)  #D
    if match:
        text = match.group("x")

    text = re.sub(r"\\\(|\\\)|\\\[|\\\]", "", text)  #E


    for pat, rep in LATEX_FIXES:  #F
        text = re.sub(pat, rep, text)

    #G
    text = text.replace("\\%", "%").replace("$",
"").replace("%", "")
    text = re.sub(
        r"\\sqrt\s*\{([^}]*)\}",
        lambda match: f"sqrt({match.group(1)})",
        text,
    )
    text = re.sub(
        r"\\sqrt\s+([^\\\s{}]+)",
        lambda match: f"sqrt({match.group(1)})",
        text,
    )

    #H
```

```
    text = re.sub(
        r"\\frac\s*\{([^{}]+)\}\s*\{([^{}]+)\}",
        lambda match: f"({match.group(1)})/({match.group(2)})",
        text,
    )
    text = re.sub(
        r"\\frac\s+([^\s{}]+)\s+([^\s{}]+)",
        lambda match: f"({match.group(1)})/({match.group(2)})",
        text,
    )

    #I
    text = text.replace("^", "**")
    text = re.sub(
        r"(?<=\d)\s+(\d+/\d+)",
        lambda match: "+" + match.group(1),
        text,
    )

    #J
    text = re.sub(
        r"(?<=\d),(?=\d\d\d(\D|$))",
        "",
        text,
    )

    return text.replace("{", "").replace("}",
"").strip().lower()
```

The `normalize_text` function in listing 3.6 takes an extracted answer string and rewrites it into a standardized format that we can reliably compare against reference solutions. It first strips away special tokens and unnecessary LaTeX clutter, such as `\left`, `\right`, or degree symbols. It then unwraps cases like `\text{...}`, removes inline math markers, and rewrites common structures into a calculator-style form. For example, it turns `\sqrt{a}` into `sqrt(a)` and `\frac{a}{b}` into `(a)/(b)`. Finally, it normalizes exponents, mixed numbers, and thousands separators and cleans up braces and casing. In short, the function transforms differently formatted LaTeX outputs into a clean, standardized string representation.

Let's now try the `normalize_text` function on our model answer:

```
print(normalize_text(extract_final_candidate(model_answer)))
```

As a result, instead of printing the answer with LaTeX formatting (`r"\dfrac{14}{3}"`), it returns the answer in a standardized, LaTeX-free form:

```
"(14)/(3)"
```

Next, let's try a differently formatted answer:

```
print(normalize_text(r"\text{\[\frac{14}{3}\]}"))
```

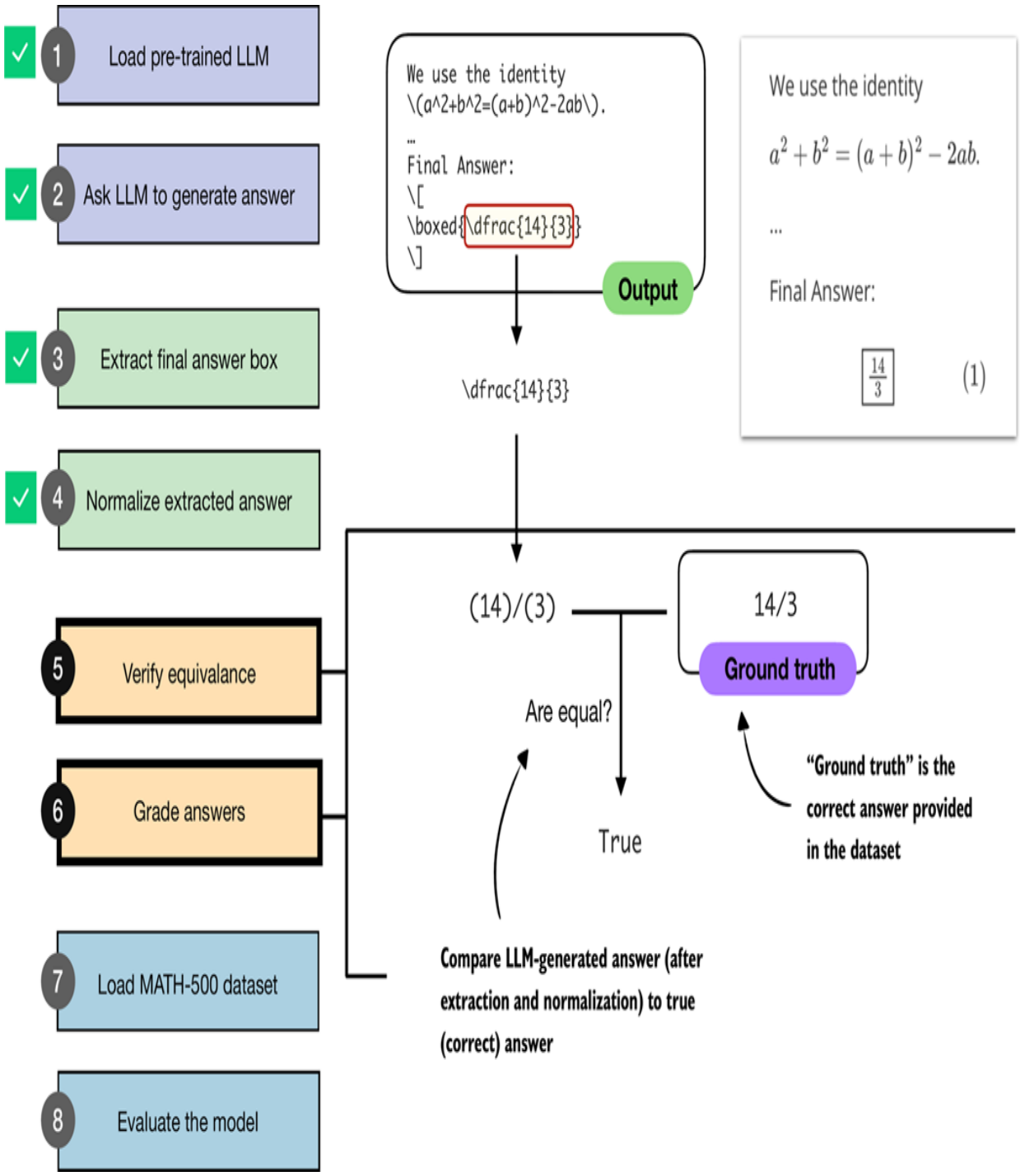
This also returns, `"(14)/(3)"`, as intended.

We now have a robust method to extract answer texts from an LLM's response. The next task, covered in the next section, is to implement a function to compare the LLM answer to a correct reference answer.

3.6 Verifying mathematical equivalence

So far, in this chapter, we implemented steps to ask an LLM to generate an answer, extract the relevant portion, and normalize it. The next step, as illustrated in figure 3.8, is to compare the extracted answer to a correct reference answer, which is, in technical contexts, referred to as *ground truth*.

Figure 3.8 An illustration of how an LLM-generated answer is checked against the correct reference answer (ground truth). The final boxed answer is extracted and normalized, then compared to the correct answer provided in the dataset. If both match, the response is graded as correct.



Note that if we want to implement the equality check shown in figure 3.8, a direct comparison using Python's == operator is not sufficient, since expressions like `"14/3"` and `"(14)/(3)"` would not match, and equivalent but unnormalized fractions such as `"(28)/(6)"` and `"(14)/(3)"` would also be treated as unequal.

As part of our equality check, we implement an additional intermediate step: parsing the extracted and normalized answer using a symbolic math engine.

For this, we use the SymPy open-source math library (https://sympy.org), which has been developed and tested for two decades and has become a staple of scientific computing in Python. The parsing function is implemented in listing 3.7.

Note

If you haven't installed the dependencies in chapter 2, you can manually install SymPy via `uv pip install sympy` (or `uv add sympy`).

Listing 3.7 SymPy parser for mathematical equality check

```
from sympy.parsing import sympy_parser as spp
from sympy.core.sympify import SympifyError
from tokenize import TokenError

def sympy_parser(expr):
    try:
        return spp.parse_expr(
            expr,
            transformations=(
                *spp.standard_transformations,  #A
                #B
                spp.implicit_multiplication_application,
            ),

            evaluate=True,  #C
        )
    except (SympifyError, SyntaxError, TypeError, IndexError,
TokenError):
        return None
```

The `sympy_parser` function in listing 3.7 takes an input expression, such as the normalized answers we extract from the LLM response, and converts it into a SymPy object that can be reliably compared for mathematical equivalence. To do so, it applies SymPy's standard parsing rules, supports implicit multiplication like (2y instead of 2*y), and also simplifies basic arithmetic (so 2+3 becomes 5).

Note

The `sympy_parser` takes into account what looks like an excessive amount of error cases, but these are all errors that I encountered when evaluating the model on all 500 MATH-500 problems, as the model does not always generate perfectly formatted outputs.

Let's see it in action and apply it to the normalized answer candidate:

```
print(sympy_parser(normalize_text(
    extract_final_candidate(model_answer)
)))
```

This returns the fraction `14/3`. Next, let's try an unnormalized fraction:

```
print(sympy_parser("28/6"))
```

Similarly, this returns `14/3`.

Using the `sympy_parser`, we can now implement the equality check function in listing 3.8:

Listing 3.8 Equality check function using SymPy

```
from sympy import simplify

def equality_check(expr_gtruth, expr_pred):
    if expr_gtruth == expr_pred:  #A
        return True

    #B
    gtruth, pred = sympy_parser(expr_gtruth),
sympy_parser(expr_pred)

    if gtruth is not None and pred is not None:  #C
        try:
            return simplify(gtruth - pred) == 0  #D
        except (SympifyError, TypeError):
            pass

    return False
```

The `equality_check` function in listing 3.8 determines whether a model's answer matches the ground-truth solution. It first looks for an exact string match, which is the simplest case. If the strings differ, it parses both expressions into SymPy objects (via the `sympy_parser` function we implemented in listing 3.7) and checks whether their difference simplifies to zero. This allows us to recognize answers that may look different on the surface (for example, 14/3 and 28/6) but are mathematically the same.

Let's try the equality checker from listing 3.8 on an example:

```
print(equality_check(
    normalize_text("13/4."),
    normalize_text(r"(13)/(4)")
))
```

As intended, this ignores the formatting and returns `True`. Next, let's try a more challenging example and see whether the symbolic math parser recognizes that 0.5 is the same as 1/2:

```
print(equality_check(
    normalize_text("0.5"),
    normalize_text(r"(1)/(2)")
))
```

This also returns `True`. Now, let's try a negative example:

```
print(equality_check(
    normalize_text("14/3"),
    normalize_text("15/3")
))
```

This returns `False` since the expressions are different.

So far, so good. Based on the encouraging results above, we may conclude that we now have a robust equality checker that we can use to evaluate the LLM on a math benchmark dataset. However, to make sure that it's ready for prime time, let's try one more example:

```
print(equality_check(
    normalize_text("(14/3, 2/3)"),
    normalize_text("(14/3, 4/6)")
))
```

In this case, we are comparing two tuples. Since 2/3 and 4/6 are mathematically equivalent, we would expect the result to be `True`. Instead, the function returns `False`, because it currently only handles simple expressions, not tuples. We will address this limitation in the next section.

3.7 Grading answers

Now, we will build upon the mathematical equality checking function from the previous section to implement a robust grading function that can also handle tuple-like expressions, such as correctly comparing expressions like `"(14/3, 2/3)"` and `"(14/3, 4/6)"`.

First, we implement a Python helper function that splits such tuple-like expressions into individual subparts via listing 3.9.

Listing 3.9 Helper function to split tuple-like expressions

```
def split_into_parts(text):
    result = [text]

    if text:  #A
        if (
            len(text) >= 2
            and text[0] in "([" and text[-1] in ")]"
            and "," in text[1:-1]
        ):
            items = [p.strip() for p in text[1:-1].split(",")]
#B
            if all(items):
                result = items
    else:  #C
        result = []

    return result
```

The `split_into_parts` function in listing 3.9 helps us handle answers with multiple components. If the input looks like a tuple or list, such as `(a, b)` or `[a, b]`, it splits the content on commas and returns the individual pieces. (If the string is empty, it simply returns an empty list.) In essence, this function

breaks down multi-part answers into smaller parts that can be checked one by one.

Before we implement the grading function next, let's take the `split_into_parts` for a test drive and try it on the tuple-like expression from earlier:

```
split_into_parts(normalize_text(r"(14/3, 2/3)"))
```

This returns `['14/3', '2/3']`, as desired.

Now, we can implement the `grade_answer` function (listing 3.10), which splits tuple-like expressions (if present) into subparts, and then uses the `equality_check` function from the previous section to compare a generated answer to a reference (ground truth) answer.

Listing 3.10 Function to grade predicted answers against ground truth

```
def grade_answer(pred_text, gt_text):
    result = False   #A
    if pred_text is not None and gt_text is not None:  #B
        gt_parts = split_into_parts(
            normalize_text(gt_text)
        )
        pred_parts = split_into_parts(
            normalize_text(pred_text)
        )

        if (gt_parts and pred_parts                #C
           and len(gt_parts) == len(pred_parts)):  #C
            result = all(
                equality_check(gt, pred)
                for gt, pred in zip(gt_parts, pred_parts)
            )  #D

    return result  #E
```

The implementation of the `grade_answer` function in listing 3.10 first assumes the prediction is incorrect (`False`) and only continues if both prediction and ground truth are non-empty. It then normalizes each side and splits them into subparts (for example, breaking `"(14/3, 2/3)"` into `["14/3", "2/3"]`). If the number of subparts matches, it compares them one

by one using equality_check. The result is returned as correct (`True`) only if all pairs match mathematically.

We can think of the `grade_answer` function as an advanced version of the `equality_check` function from the previous section. The `grade_answer` function can split tuple-like expressions and normalize the answers before applying the `equality_check` function.

On simple expressions, it works similarly to the `equality_check`, returning `True` if two expressions are mathematically equivalent:

```
grade_answer("14/3", r"\frac{14}{3}")
```

In addition, as described above, it now also returns `True` in case of two mathematically equivalent tuple-like expressions:

```
grade_answer(r"(14/3, 2/3)", "(14/3, 4/6)")
```

To check the `grade_answer` function more comprehensively, the code in listing 3.11 contains more diverse test cases.

Listing 3.11 Test cases and demo function to test the grader

```
tests = [  #A
        ("check_1", "3/4", r"\frac{3}{4}", True),
        ("check_2", "(3)/(4)", r"3/4", True),
        ("check_3", r"\frac{\sqrt{8}}{2}", "sqrt(2)", True),
        ("check_4", r"\( \frac{1}{2} + \frac{1}{6} \)", "2/3",
True),
        ("check_5", "(1, 2)", r"(1,2)", True),
        ("check_6", "(2, 1)", "(1, 2)", False),
        ("check_7", "(1, 2, 3)", "(1, 2)", False),
        ("check_8", "0.5", "1/2", True),
        ("check_9", "0.3333333333", "1/3", False),
        ("check_10", "1,234/2", "617", True),
        ("check_11", r"\text{2/3}", "2/3", True),
        ("check_12", "50%", "1/2", False),
        ("check_13", r"2\cdot 3/4", "3/2", True),
        ("check_14", r"90^\circ", "90", True),
        ("check_15", r"\left(\frac{3}{4}\right)", "3/4", True),
    ]
```

```
def run_demos_table(tests):
    header = ("Test", "Expect", "Got", "Status")
    rows = []
    for name, pred, gtruth, expect in tests:
        got = grade_answer(pred, gtruth)  #B
        status = "PASS" if got == expect else "FAIL"
        rows.append((name, str(expect), str(got), status))

    data = [header] + rows

    col_widths = [  #C
        max(len(row[i]) for row in data)
        for i in range(len(header))
    ]

    for row in data:  #D
        line = " | ".join(
            row[i].ljust(col_widths[i])
            for i in range(len(header))
        )
        print(line)

    passed = sum(r[3] == "PASS" for r in rows)  #E
    print(f"\nPassed {passed}/{len(rows)}")     #E
```

The code in listing 3.11 is a simple test suite that takes in a selection of `tests` to check whether the `grade_answer` function works as intended. The `tests` list contains tuples that cover a selection of fractions, LaTeX notations, tuple inputs, decimals, percentages, and other tricky formats.

The `run_demos_table` function then runs each test by calling `grade_answer`, collects the outcomes, and organizes the results into a formatted table.

Calling the `run_demos_table(test)` function in listing 3.11 prints the following:

```
Test     | Expect | Got   | Status
check_1  | True   | True  | PASS
check_2  | True   | True  | PASS
check_3  | True   | True  | PASS
check_4  | True   | True  | PASS
check_5  | True   | True  | PASS
check_6  | False  | False | PASS
check_7  | False  | False | PASS
```

```
check_8  | True   | True  | PASS
check_9  | False  | False | PASS
check_10 | True   | True  | PASS
check_11 | True   | True  | PASS
check_12 | False  | False | PASS
check_13 | True   | True  | PASS
check_14 | True   | True  | PASS
check_15 | True   | True  | PASS

Passed 15/15
```

As we can see based on the `PASS` results above, the `grade_answer` function is relatively robust and capable of handling a variety of differently formatted expressions.

Exercise 3.1: Adding more test cases

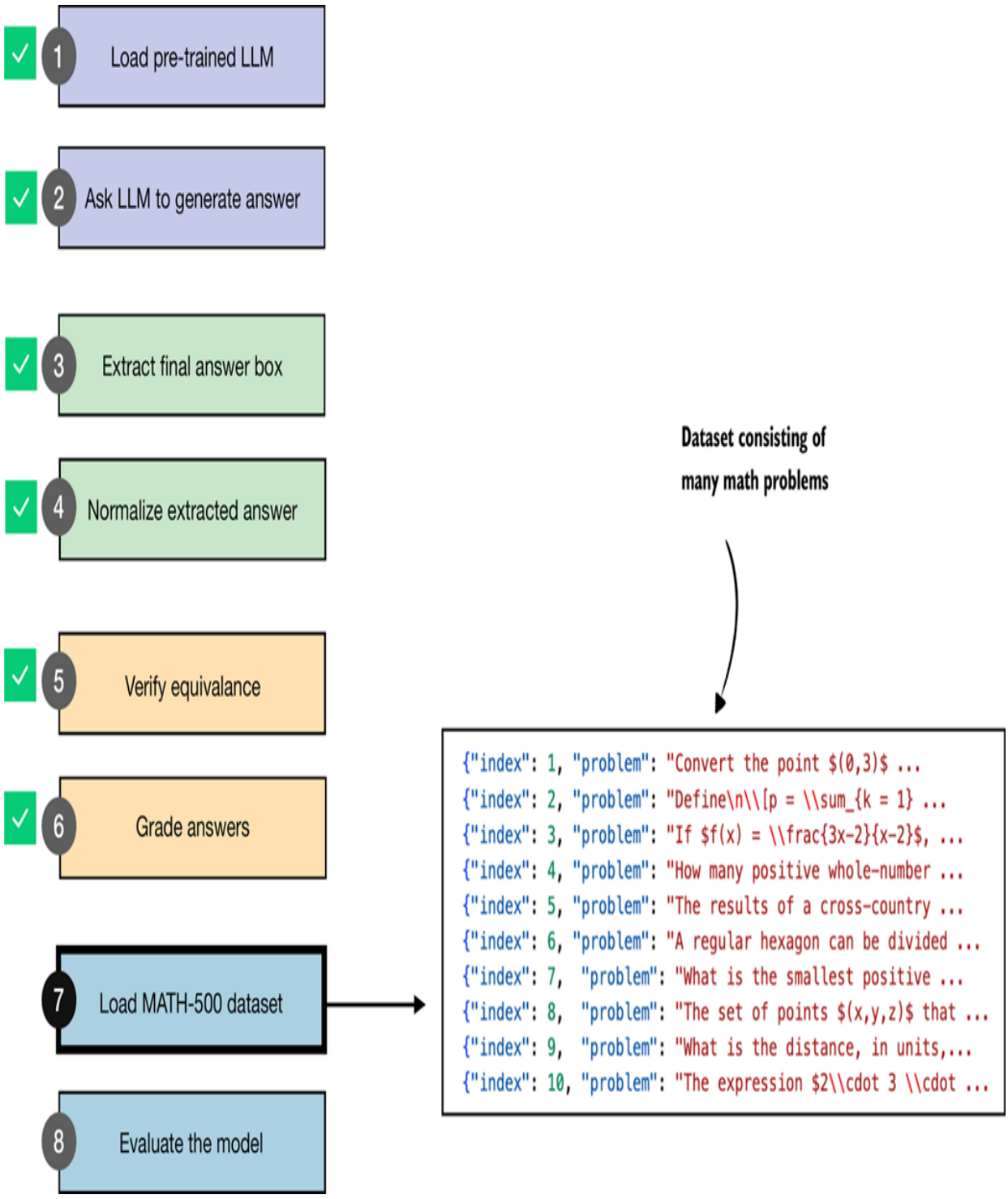
Try to think of additional test cases, ideally challenging ones, and add them to the `run_demos_table()` function. Can you find cases where the check fails incorrectly?

With the `grade_function` implemented, we now have the building blocks in place to evaluate the LLM. In the next section, we will load a math dataset on which we will evaluate the LLM.

3.8 Loading the evaluation dataset

As we have seen in the chapter so far, implementing a robust verification pipeline can be a tedious task. Fortunately, we now have all the pieces in place, from answer extraction to grading, and are ready to evaluate the LLM on a benchmark dataset. For this, as illustrated in figure 3.9, we will use the MATH-500 dataset (https://huggingface.co/datasets/HuggingFaceH4/MATH-500), a widely used benchmark for reasoning models. It is a curated collection of 500 problems sampled from the original MATH dataset.

Figure 3.9 Loading the evaluation dataset. After completing steps 2–6 on individual problems (generate, extract, normalize, verify, and grade answers) in the previous sections, the two remaining steps are to load the full dataset (step 7) and apply the same procedure across all problems to evaluate the model (step 8).



We will load the MATH-500 dataset (step 7 in figure 3.9) using the following code:

```
import json
from urllib.request import urlopen
```

```
local_path = Path("math500_test.json")
url = (
    "https://raw.githubusercontent.com/rasbt/reasoning-from-
scratch/"
    "main/ch03/01_main-chapter-code/math500_test.json"
)

if local_path.exists():
    with local_path.open("r", encoding="utf-8") as f:
        math_data = json.load(f)
else:
    with urlopen(url) as f:
        math_data = json.load(f)

print("Number of entries:", len(math_data))
```

This prints:

```
Number of entries: 500
```

Loading the dataset from Hugging Face Model Hub

The MATH-500 dataset split was originally proposed in the PRM800K repository (https://github.com/openai/prm800k/tree/main?tab=readme-ov-file#math-splits) and is also available on the Hugging Face Hub (https://huggingface.co/datasets/HuggingFaceH4/MATH-500). However, in this book, we load a copy from the code repository to ensure reproducibility in case the external sources change.

If you prefer to download the dataset directly from Hugging Face, you can use the following code. Note that this requires the `datasets` library, which can be installed via `pip install datasets` or `uv add datasets`:

```
from datasets import load_dataset
dset = load_dataset("HuggingFaceH4/MATH-500", split="test")
```

(You do not need to run this here; it is included for reference only.)

Before we jump to the next section to implement the model evaluation pipeline, let's take a closer look at the structure of the dataset by printing its first entry (we use the built-in `pprint` library for nicer formatting):

```
from pprint import pprint
pprint(math_data[0])
```

This produces the following output:

```
{'answer': '\\left( 3, \\frac{\\pi}{2} \\right)',
 'level': 2,
 'problem': 'Convert the point $(0,3)$ in rectangular
coordinates to polar '
            'coordinates.  Enter your answer in the form
$(r,\\theta),$ where '
            '$r > 0$ and $0 \\le \\theta < 2 \\pi.$',
 'solution': 'We have that $r = \\sqrt{0^2 + 3^2} = 3.$  Also,
if we draw the '
             'line connecting the origin and $(0,3),$ this line
makes an angle '
             'of $\\frac{\\pi}{2}$ with the positive $x$-
axis.\n'
             '\n'
             '[asy]\n'
             'unitsize(0.8 cm);\n'
             '\n'
             'draw((-0.5,0)--(3.5,0));\n'
             'draw((0,-0.5)--(0,3.5));\n'
             'draw(arc((0,0),3,0,90),red,Arrow(6));\n'
             '\n'
             'dot((0,3), red);\n'
             'label("$(0,3)$", (0,3), W);\n'
             'dot((3,0), red);\n'
             '[/asy]\n'
             '\n'
             'Therefore, the polar coordinates are
$\\boxed{\\left( 3, '
             '\\frac{\\pi}{2} \\right)}.$',
 'subject': 'Precalculus',
 'unique_id': 'test/precalculus/807.json'}
```

As we can see, the dataset entry is formatted as a Python dictionary with keys and values. The relevant keys are

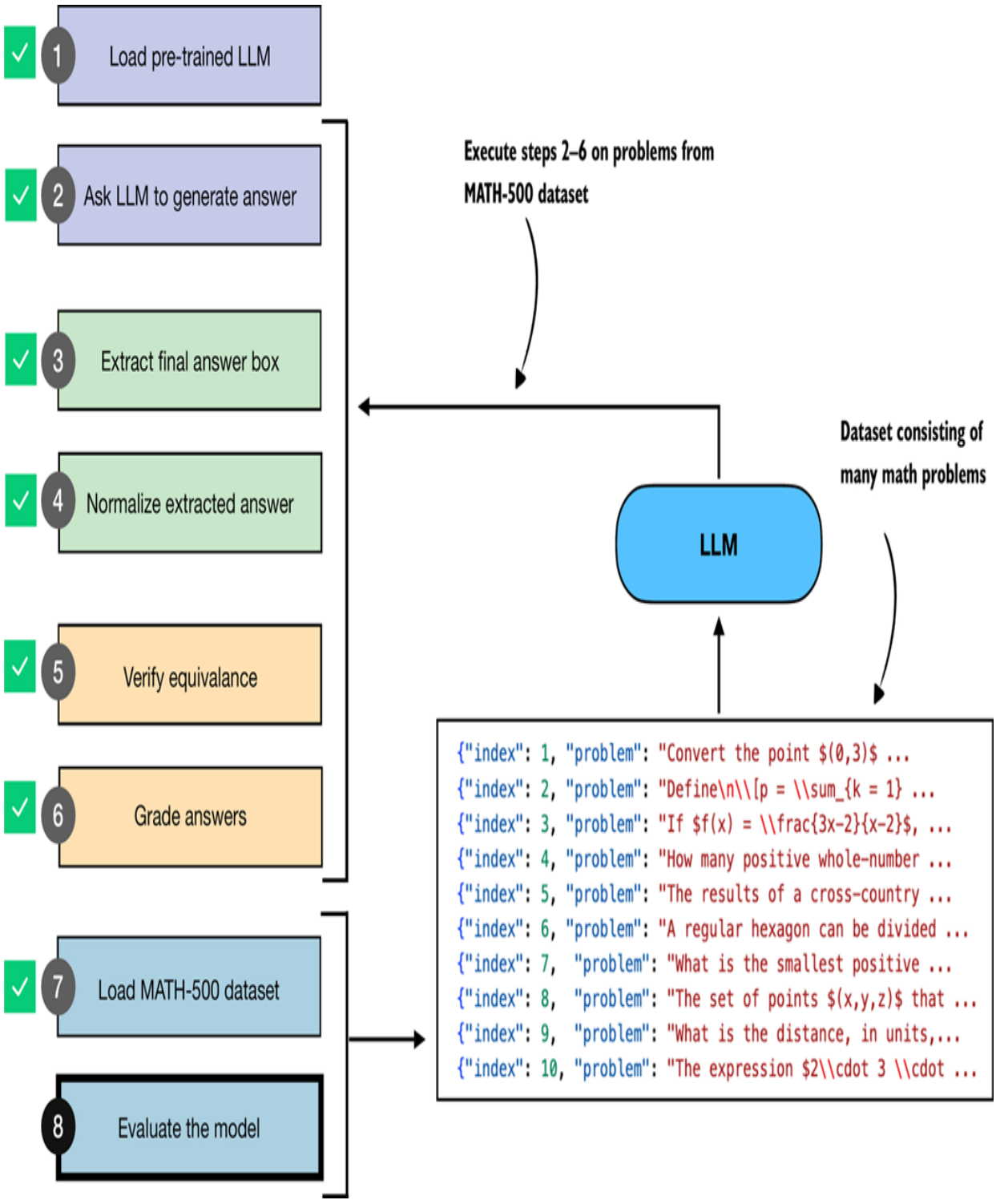
- `"problem"`: the math question or problem for the LLM to solve,
- `"answer"`: the correct (ground truth) answer we want to compare the LLM answer against,
- `"solution"`: a worked-out, step-by-step explanation of the problem (not used in this chapter, but useful for training or analysis).

Now that we have a pre-trained LLM, evaluation functions, and a benchmark dataset to work with, we can implement the model evaluation.

3.9 Evaluating the model

In this section, we put the LLM text generation and evaluation tools from steps 2–6 in figure 3.10 into practice and apply them to the MATH-500 dataset (step 8 in figure 3.10), which we loaded in the previous section.

Figure 3.10 The complete evaluation pipeline on the MATH-500 dataset. After loading the dataset (step 7), steps 2–6 are applied systematically across all problems to obtain the final model evaluation (step 8).



As you may recall from section 3.4 (*Extracting the final answer box*), our answer checking pipeline expects that the model returns the answer in boxed form, which is a common convention when evaluating reasoning models on

math problems. To increase the likelihood that the model adheres to this format, we can format the prompt as shown in listing 3.12:

Listing 3.12 Function to render a prompt template for math evaluation

```
def render_prompt(prompt):
    template = (
        "You are a helpful math assistant.\n"
        "Answer the question and write the final result on a new
line as:\n"
        "\\boxed{ANSWER}\n\n"
        f"Question:\n{prompt}\n\nAnswer:"
    )
    return template
```

Let's now apply the prompt template from listing 3.12 to the example prompt we introduced earlier in this chapter (section 3.2). For convenience, we redefine the example prompt here:

```
prompt = (
    r"If $a+b=3$ and $ab=\tfrac{13}{6}$, "
    r"what is the value of $a^2+b^2$?"
)
prompt_fmt = render_prompt(prompt)
print(prompt_fmt)
```

The formatted prompt is now as follows:

```
You are a helpful math assistant.
Answer the question and write the final result on a new line as:
\boxed{ANSWER}

Question:
If $a+b=3$ and $ab=\tfrac{13}{6}$, what is the value of
$a^2+b^2$?

Answer:
```

Next, we pass the prompt to the text generation wrapper function we defined in listing 3.3 in section 3.3 to recap the text generation process before we construct the model evaluation function:

```
generated_text = generate_text_stream_concat(
    model, tokenizer, prompt_fmt, device,
    max_new_tokens=2048,
    verbose=True
)
```

Using this prompt example, the model responds with a relatively brief answer: `"\boxed{10}"`. (Note that the generated response may differ depending on whether you executed the code on a CPU, CUDA, or MPS device.)

While brevity can speed up generation by reducing the number of tokens, the response is incorrect. In contrast, in section 3.3, without a prompt template, the model produced a longer response, which led to the correct answer, 14/3.

However, whether a prompt template is well-suited for a given model and task ideally needs to be determined on a larger set of examples before we can draw any conclusions, for instance, the MATH-500 dataset we will evaluate the model on later in this section.

Prompt template choices

The prompt template in listing 3.12 is used here to demonstrate how a model evaluation pipeline can be implemented with answers that are automatically checked for correctness. The chosen template encourages short outputs, which lets you work through this chapter efficiently on a first read. Afterward, I recommend revisiting the chapter with alternative settings to optimize accuracy on the reasoning model variant.

As it turns out, using no prompt template boosts the base model performance by 50%, but it reduces the accuracy of the reasoning model by 40%.

Additionally, we may also experiment with alternative prompt templates. For instance, the common standard prompt for the MATH-500 benchmark is the following variant that swaps "Question:" with "Problem:" in listing 3.12. This seemingly minor change improves the base model’s accuracy by approximately 20%, likely because it better matches the memorized training data (assuming the MATH-500 test set was included in the training corpus).

However, while the base model benefits from this change, the accuracy of the reasoning model variant drops by 30%.

Next, before we implement the final model evaluation function, let us test our model evaluation pipeline end to end on a smaller example via the demo function in listing 3.13:

Listing 3.13 Demo function to run the evaluation pipeline

```
def mini_eval_demo(model, tokenizer, device):
    ex = {  #A
        "problem": "Compute 1/2 + 1/6.",
        "answer": "2/3"
    }
    prompt = render_prompt(ex["problem"])     #B
    gen_text = generate_text_stream_concat(   #C
        model, tokenizer, prompt, device,     #C
        max_new_tokens=64,                    #C
    )                                         #C
    pred_answer = extract_final_candidate(gen_text)  #D
    is_correct = grade_answer(                       #E
        pred_answer, ex["answer"]                    #E
    )                                                #E

    print(f"Device: {device}")
    print(f"Prediction: {pred_answer}")
    print(f"Ground truth: {ex['answer']}")
    print(f"Correct: {is_correct}")
```

The `mini_eval_demo` function in listing 3.13 combines all the aspects we have covered so far in this chapter:

1. Applying a prompt template
2. Feeding the formatted prompt to the LLM to generate an answer
3. Extracting and normalizing the answer
4. Grading the answer

This `mini_eval_demo` function essentially connects the evaluation components together into a small function that we can use to test the code before coding the final evaluation pipeline for the MATH-500 dataset. The code starts from a toy example (`ex`), renders the problem into the prompt

template (`prompt`), and streams a response from the model (`generate_text_stream_concat`). It then parses the model output to a final candidate answer (`pred_answer`) and grades it against the ground truth with `grade_answer`. Lastly, it prints the results for us to evaluate.

Calling the `mini_eval_demo(model, tokenizer, device)` function results in the following output:

```
Device: mps
Prediction: 1/3
Ground truth: 2/3
Correct: False
```

We can see that the generated answer (`"1/3"`) was correctly extracted, but it doesn't match the correct answer (`"2/3"`), and hence the check returns False. (Note that the results may differ depending on whether you execute the code on a CPU, CUDA, or MPS device.)

Now that we have tested our workflow on a simpler example, let's implement it to run on the MATH-500 dataset.

Listing 3.14 End-to-end model evaluation pipeline for MATH-500 dataset

```
import time

def evaluate_math500_stream(
    model,
    tokenizer,
    device,
    math_data,
    out_path=None,
    max_new_tokens=512,
    verbose=False,
):

    if out_path is None:
        dev_name = str(device).replace(":", "-")   #A
        out_path = Path(f"math500_{WHICH_MODEL}-
{dev_name}.jsonl")

    num_examples = len(math_data)
    num_correct = 0
    print(f"MATH-500: 0/{num_examples}", end="\r", flush=True)
```

```
    start_time = time.time()

    with open(out_path, "w", encoding="utf-8") as f:   #B
        for i, row in enumerate(math_data, start=1):
            prompt = render_prompt(row["problem"])     #C
            gen_text = generate_text_stream_concat(    #D
                model, tokenizer, prompt, device,      #D
                max_new_tokens=max_new_tokens,         #D
                verbose=verbose,                       #D
            )

            extracted = extract_final_candidate(       #E
                gen_text                               #E
            )                                          #E
            is_correct = grade_answer(                 #F
                extracted, row["answer"]               #F
            )                                          #F
            num_correct += int(is_correct)

            record = {                                 #G
                "index": i,                            #G
                "problem": row["problem"],             #G
                "gtruth_answer": row["answer"],        #G
                "generated_text": gen_text,            #G
                "extracted": extracted,                #G
                "correct": bool(is_correct),           #G
            }                                          #G
            f.write(json.dumps(record, ensure_ascii=False) +
"\n")

            if verbose:                                #H
                print(
                    f"\n\n{'='*50}\nMATH-500:
{i}/{num_examples}\n"
                    f"{'='*50}\nExtracted: {extracted}\n"
                    f"Expected:  {row['answer']}\n"
                    f"Correct so far: {num_correct}\n{'-'*50}"
                )
            else:
                print(
                    f"MATH-500: {i}/{num_examples}",
                    end="\r", flush=True
                )

    #I
    seconds_elapsed = time.time() - start_time
    acc = num_correct / num_examples if num_examples else 0.0
```

```
    print(f"\nAccuracy: {acc*100:.1f}%
({num_correct}/{num_examples})")
    print(f"Total time: {seconds_elapsed/60:.1f} min")
    print(f"Logs written to: {out_path}")
    return num_correct, num_examples, acc
```

The `evaluate_math500_stream` function in listing 3.14 uses the same main steps as the smaller demo function from listing 3.13: for each problem, it renders the prompt, streams a model response, extracts the answer candidate, and grades it against the reference answer.

In addition to iterating over a dataset with multiple entries, it adds some additional bells and whistles. For instance, it saves the generated responses to a JSON file in a Python dictionary-like format for record keeping and closer inspection.

Let's now run this function on a subset, the first 10 examples of MATH-500, which takes about 0.7 minutes on a Mac Mini with an M4 chip. (Evaluating the reasoning model variant takes about 7 min as it generates longer responses.)

```
print("Model:", WHICH_MODEL)
num_correct, num_examples, acc = evaluate_math500_stream(
    model, tokenizer, device,
    math_data=math_data[:10],  #A
    max_new_tokens=2048,
    verbose=False              #B
)
```

In the code example above, we set `max_new_tokens` to a generous 2048, since the reasoning model variant, per design, tends to generate much longer responses, and we don't want to cut it off prematurely. This, however, leads to much longer evaluation times, where it may appear that the generation is stuck. Optionally, you could set `verbose=True` to see the response being generated live, token by token.

The result of running the `evaluate_math500_stream` function is as follows:

```
Model: base
Device: mps
MATH-500: 10/10
```

```
Accuracy: 20.0% (2/10)
Total time: 0.7 min
Logs written to: math500_base-mps.jsonl
```

(Note that the results may differ depending on whether you execute the code on a CPU, CUDA, or MPS device.)

As we can see, the model achieves a relatively low accuracy of 20%. We can open the `math500_base-mps.jsonl` file in a text editor to analyze the results, together with the generated response. For instance, we find that the answers, in all cases, have been successfully extracted, but they are plain wrong, which indicates that the model does not have very strong math problem-solving capabilities (yet). This is expected since it's merely a base model.

Loading the .jsonl file programmatically

The `.jsonl` file suffix is a convention used for JSON files with one data entry per row. You can view it in your favorite text editor. Optionally, we can load the .jsonl file created during the evaluation in Python using the following code:

```
dev_name = str(device).replace(":", "-")
local_path = f"math500_{WHICH_MODEL}-{dev_name}.jsonl"
results = []
with open(local_path, "r") as f:
    for line in f:
        if line.strip():
            results.append(json.loads(line))
```

The reasoning model variant, which you can enable by setting `WHICH_MODEL = "reasoning"` in listing 3.1 in section 3.2, performs much better and achieves a 90% accuracy on the same 10-sample subset and 50.8% on the complete 500-sample dataset, as shown in table 3.1.

Table 3.1 MATH-500 task accuracy on different devices

Mode	Device	Accuracy	MATH-500 size
Base	CPU	30%	10

Base	CUDA	30%	10
Base	MPS	20%	10
Reasoning	CPU	90%	10
Reasoning	CUDA	90%	10
Reasoning	MPS	80%	10
Base	CUDA	15.3%	500
Reasoning	CUDA	50.8%	500

As shown in table 3.1, the reasoning variant, with its longer responses, has a drastically improved accuracy, but also increases the compute intensity and answer generation time substantially (from 0.7 min for the base model to 7 min for the reasoning model on a Mac Mini with M4 chip on the 10-sample subset, and from 13.3 min to 185.4 min on an H100), which highlights one of the trade-offs of using reasoning models.

Tip

The code repository contains a bonus script (https://github.com/rasbt/reasoning-from-scratch/blob/main/ch03/02_math500-verifier-scripts/evaluate_math500_batched.py) that runs the code in this chapter in batched mode. This means it processes multiple examples per forward pass to accelerate the evaluation while requiring more RAM. With a batch size of 128, this reduces the runtime of the base model, when evaluating all 500 samples, from 13.3 min to 3.3 min on an H100 GPU. Similarly, it reduces the runtime of the reasoning model from 185.4 min to 14.6 min. Note that the H100 is used as an example, and the script is compatible with other GPUs as well.

Exercise 3.2: Calculating the average response length

Try to modify the code in this chapter to also report the average response length in the evaluation function in listing 3.13. Instead of modifying the function directly, you could also compute the response length from the generated JSON report files.

Exercise 3.3: Extending or changing the evaluation dataset

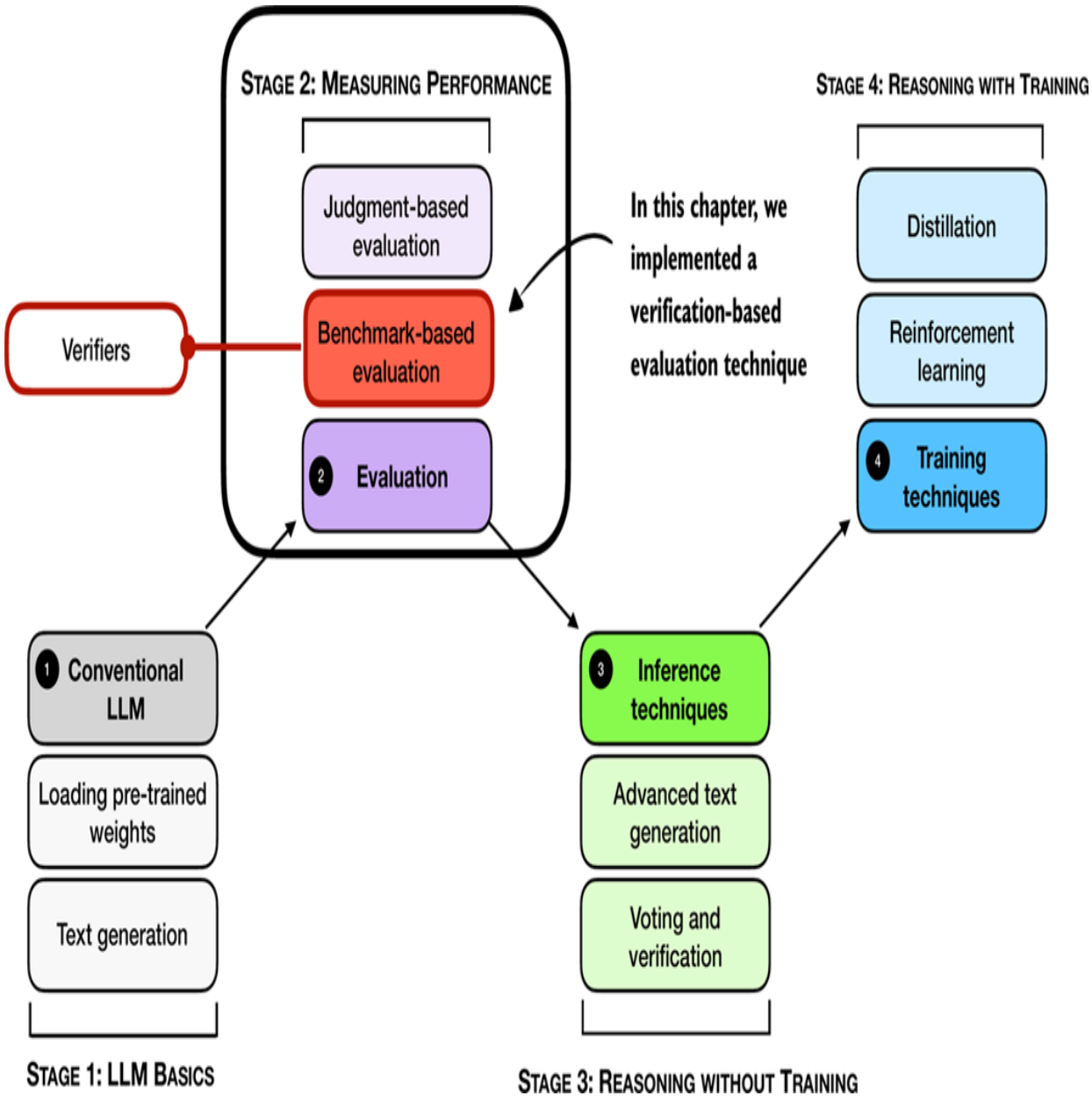
We choose a subset of only 10 examples for computational efficiency. However, readers are encouraged to consider running the code on larger or different portions of the dataset to observe whether the 10-sample subset is representative. Ideally, you could also experiment with your own data. (For reference, evaluating the base model on the complete MATH-500 dataset takes about xxx min for the base model and xxx min for the reasoning model on an H100.)

Exercise 3.4: Experimenting with different prompt templates

Models can be sensitive to different prompt templates. Experiment with different prompt templates in listing 3.11 to see how it affects the results. Also, while the Qwen3 team recommends using the base model without an additional chat template, you can additionally enable the `apply_chat_template=True` setting in the tokenizer (listing 3.1) and observe whether it improves the base model performance.

Note that this concludes our chapter on implementing a verification-based approach for math tasks (figure 3.11). We chose math because it is both natural to implement and widely used in reasoning-specific training, particularly reinforcement learning with verifiable rewards, which we will cover in chapter 5. The same concept can be extended to other domains, such as code, although we did not explore that here since executing code would require additional setup of a secure virtual environment.

Figure 3.11 Mental model of the topics covered in this book. This chapter implemented a verifier-based evaluation pipeline. In the next chapter, we will improve the reasoning capabilities of the LLM via more advanced inference techniques.



Now, with an evaluation framework in place, the next chapter, as shown in figure 3.11, focuses on improving the reasoning capabilities through more advanced inference (text generation) techniques.

3.10 Summary

- There are four main evaluation methods for LLMs: multiple choice, verifiers, leaderboards, and LLM judges

- Verification-based evaluation methods allow free-form answers and use external tools to check correctness
- This chapter focuses on verification-based evaluation by building a math verifier that extracts, normalizes, and checks answers with SymPy
- The verification pipeline involves several core steps from loading the LLM to running the evaluation on a dataset
- As part of the verification pipeline, answer extraction uses string parsing to locate boxed content (with fallback mechanisms for missing boxes)
- Another step implements normalization, which standardizes diverse answer formats by stripping LaTeX and converting mathematical notation
- Finally, the pipeline uses mathematical equivalence checking (via SymPy) to compare expressions symbolically
- The MATH-500 dataset provides 500 curated math problems for evaluation
- Prompt templates significantly impact model performance
- The reasoning model achieves higher accuracy than the base model, but requires much longer runtime

OceanofPDF.com

Appendix A. References and further reading

A.1 Chapter 1

A.1.1 References

The announcement article of OpenAI's o1 model, which is regarded as the first LLM-based reasoning model:

- Introducing OpenAI o1-preview, https://openai.com/index/introducing-openai-o1-preview/

DeepSeek-R1 is the first open-source reasoning model that was accompanied by a comprehensive technical report, which was the first to show that reasoning emerges from reinforcement learning with verifiable rewards (a topic covered in more detail in chapter 5):

- DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, https://arxiv.org/abs/2501.12948

OpenAI CEOâ€™s comment on the reasoning ("chain-of-thought") capabilities of future models:

- "[...] We will next ship GPT-4.5, the model we called Orion internally, as our last non-chain-of-thought model. [...]", https://x.com/sama/status/1889755723078443244

A research paper by AI researchers at Apple finding that reasoning models are sophisticated (but very capable) pattern matchers:

- The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity, https://machinelearning.apple.com/research/illusion-of-thinking

An in-depth book and guide on implementing and training large language models step-by-step:

- Build a Large Language Model (From Scratch), http://mng.bz/orYv

A.1.2 Further Reading

An introduction to how DeepSeek-R1 works, providing insights into the foundations of reasoning in LLMs:

- Understanding Reasoning LLMs, https://magazine.sebastianraschka.com/p/understanding-reasoning-llms

A.2 Chapter 2

A.2.1 References

Official installation page for the uv Python package and project manager:

- Installing uv, https://docs.astral.sh/uv/getting-started/installation/

Cloud compute platforms with GPU support:

- Lightning AI, https://lightning.ai/
- Google Colab, https://colab.research.google.com/

Qwen3 resources with additional benchmark performance and comparison to other models:

- Blog post, https://qwenlm.github.io/blog/qwen3/
- Technical report, https://arxiv.org/abs/2505.09388

A.2.2 Further Reading

A PyTorch tutorial for readers who are new to PyTorch or would like a refresher:

- PyTorch in One Hour: From Tensors to Training Neural Networks on Multiple GPUs tutorial, https://sebastianraschka.com/teaching/pytorch-1h

Additional resources on tokenization:

- Build a Large Language Model (from Scratch) chapter 2, https://mng.bz/M96o
- Implementing A Byte Pair Encoding (BPE) Tokenizer From Scratch, https://sebastianraschka.com/blog/2025/bpe-from-scratch.html

A.3 Chapter 3

A.3.1 References

The MATH-500 dataset originated from the MATH dataset (with 12,500 problems across algebra, geometry, probability, number theory, and more) that was introduced in the following paper:

- Measuring Mathematical Problem Solving With the MATH Dataset, https://arxiv.org/abs/2103.03874

The MATH-500 split (created from the original MATH dataset) was proposed in the following paper:

- Let's Verify Step by Step, https://arxiv.org/abs/2305.20050

A.3.2 Further Reading

Readers who are interested in learning more about SymPy (not required for this book) can learn about it in this official tutorial:

- SymPy introductory tutorial, https://docs.sympy.org/latest/tutorials/intro-tutorial/index.html

An example of a system (here, a fine-tuned LLM) to also evaluate intermediate reasoning steps:

- Evaluating Mathematical Reasoning Beyond Accuracy, https://arxiv.org/pdf/2404.05692

A large-scale dataset containing 800,000 step-level correctness labels for model-generated solutions to problems from the MATH dataset:

- Let's Verify Step by Step, https://arxiv.org/abs/2305.20050

An article describing the rising cost of LLM evaluation, finding that evaluating reasoning models such as o1 on (seven) popular benchmarks costs approximately $1500:

- The rise of AI "reasoning" models is making benchmarking more expensive, https://techcrunch.com/2025/04/10/the-rise-of-ai-reasoning-models-is-making-benchmarking-more-expensive/

A comprehensive 2025 survey on LLM benchmarks:

- A Survey on Large Language Model Benchmarks, https://arxiv.org/abs/2508.15361

Instead of only relying on deterministic and symbolic verifiers, a recent research project highlighted that small reasoning models themselves can be used successfully as verifiers for other reasoning models:

- xVerify: Efficient Answer Verifier for Reasoning Model Evaluations, https://arxiv.org/abs/2504.10481

A.4 Chapter F

A.4.1 References

The paper that introduced the popular multiple-choice MMLU dataset:

- Measuring Massive Multitask Language Understanding, https://arxiv.org/abs/2009.03300

A detailed description of the Elo rating system:

- Elo rating system, https://en.wikipedia.org/wiki/Elo_rating_system

The Chatbot Arena paper describing the original methodology behind the popular LLM leaderboard:

- Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference, https://arxiv.org/abs/2403.04132

A.4.2 Further Reading

A paper discussing the problems with leaderboards such as LM Arena:

- The Leaderboard Illusion, http://arxiv.org/abs/2504.20879

An article by the author describing gpt-oss in more detail:

- From GPT-2 to gpt-oss: Analyzing the Architectural Advances, https://magazine.sebastianraschka.com/p/from-gpt-2-to-gpt-oss-analyzing-the

A survey of different LLM judge approaches:

- A Survey on LLM-as-a-Judge, https://arxiv.org/abs/2411.15594

Example of a small LLM fine-tuned to act as a judge:

- PHUDGE: Phi-3 as Scalable Judge, https://arxiv.org/abs/2405.08029

OceanofPDF.com

Appendix B. Exercise solutions

The complete code examples for the exercise solutions can be found in the supplementary GitHub repository at https://github.com/rasbt/reasoning-from-scratch.

B.1 Chapter 2

Exercise 2.1

We can use a prompt similar to "Hello, Ardwarklethyrx. Haus und Garten.", which contains a made-up word (`"Ardwarklethyrx"`) and three words in a non-English language (German):

```
"Haus und Garten":
prompt = "Hello, Ardwarklethyrx. Haus und Garten."
input_token_ids_list = tokenizer.encode(prompt)
for i in input_token_ids_list:
    print(f"{[i]} --> {tokenizer.decode([i])}")
```

The output is:

```
[9707] --> Hello
[11] --> ,
[1644] -->  Ar
[29406] --> dw
[838] --> ark
[273] --> le
[339] --> th
[10920] --> yr
[87] --> x
[13] --> .
[47375] -->  Haus
[2030] -->  und
[93912] -->  Garten
[13] --> .
```

As we can see, unknown words are broken into smaller pieces of subwords or even single tokens; this allows the tokenizer and LLM to handle any input.

German words are not broken down into characters or even subwords here, suggesting that the tokenizer has seen German texts during training. This also suggests that the LLM was likely trained on German texts, too, and should be able to handle at least certain non-English languages well.

Exercise 2.2

The updated `generate_text_basic` function, now called `generate_text_basic_stream`, looks like as follows:

```
@torch.inference_mode()
def generate_text_basic_stream(
    model,
    token_ids,
    max_new_tokens,
    eos_token_id=None
):

    # input_length = token_ids.shape[1]  #A

    model.eval()
    for _ in range(max_new_tokens):
        out = model(token_ids)[:, -1]
        next_token = torch.argmax(out, dim=-1, keepdim=True)

        if (eos_token_id is not None
                and torch.all(next_token == eos_token_id)):
            break

        yield next_token  #B

        token_ids = torch.cat([token_ids, next_token], dim=1)
    # return token_ids[:, input_length:]  #C


for token in generate_text_basic_stream(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=50,
    eos_token_id=tokenizer.eos_token_id
):
    token_id = token.squeeze(0).tolist()
    print(
        tokenizer.decode(token_id),
        end="",
        flush=True
    )
```

This prints the following text:

```
 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
writing
articles, and even creating creative content.
```

Note that the text above is identical to the text generated by the `generate_text_basic` function when using the same prompt, as expected. However, if we execute the code on our computer, we should see each word being generated on the fly now.

Similarly, we can modify the KV cache variant (`generate_text_basic_cache`) as follows to add streaming support:

```
from reasoning_from_scratch.qwen3 import KVCache

@torch.inference_mode()
def generate_text_basic_stream_cache(
    model,
    token_ids,
    max_new_tokens,
    eos_token_id=None
):
    # input_length = token_ids.shape[1]  #A
    model.eval()
    cache = KVCache(n_layers=model.cfg["n_layers"])
    model.reset_kv_cache()

    out = model(token_ids, cache=cache)[:, -1]
    for _ in range(max_new_tokens):
        next_token = torch.argmax(out, dim=-1, keepdim=True)

        if (eos_token_id is not None
                and torch.all(next_token == eos_token_id)):
            break

        yield next_token  #B
        # token_ids = torch.cat([token_ids, next_token], dim=1)
        out = model(next_token, cache=cache)[:, -1]

    # return token_ids[:, input_length:]  #C
```

Exercise 2.3

We can simply delete the line `device = torch.device("cpu")` in section 2.5, and then rerun the rest of the code in chapter 2 as is. Reference numbers for the hardware I tried the code on are provided in table 2.1 at the end of chapter 2.

B.2 Chapter 3

Exercise 3.1

There is an endless number of different test cases we may add. Below is a selection of some interesting ones:

```
from reasoning_from_scratch.ch03 import (
    run_demos_table
)

more_tests = [
    ("check_16", "[1, 2]", "(1, 2)", True),  #A
    ("check_17", "1e-3", "0.001", True),  #B
    ("check_18", "(-3)^2", "9", True),  #C
    ("check_19", "−1", "-1", True),  #D

]
run_demos_table(more_tests)
```

The output is:

```
Test     | Expect | Got   | Status
check_16 | True   | True  | PASS
check_17 | True   | True  | PASS
check_18 | True   | True  | PASS
check_19 | True   | False | FAIL
```

As we can see, the tests pass in all cases except for `check_19`, which swaps the regular sign with a Unicode version of a minus sign that looks indistinguishable to the human eye (depending on which font or editor we use). We could fix this test case by adding one of the following lines anywhere to the `normalize_text` function:

```
text = text.replace("−", "-")
```

or

```
text = text.replace("\u2212", "-")
```

Another interesting test is the following one:

```
extra_tests_1 = [
    ("check_20", "Text around answer 3.", "3", True)
]
```

We can run it via the following code:

```
run_demos_table(extra_tests_1)
```

However, it fails the test:

```
Test     | Expect | Got   | Status
check_20 | True   | False | FAIL

Passed 0/1
```

While it may seem that our code cannot handle such text-containing cases, this is actually a poorly designed test. In practice, the `run_demos_table` function is intended specifically to test the `grade_answer` function; nothing more, nothing less.

The `grade_answer` function would never receive the entire answer in this text form, since the answer would have been extracted from the text before being passed to it. For instance, if we want to test text answers, we need to call the test as follows:

```
from reasoning_from_scratch.ch03 import (
    extract_final_candidate
)
extra_tests_2 = [
    ("check_20",
     extract_final_candidate("Text around answer 3."),
     "3", True)
]
```

As we can see based on the output, it now passes the test:

```
run_demos_table(extra_tests_2)
Test     | Expect | Got  | Status
check_20 | True   | True | PASS

Passed 1/1
```

Exercise 3.2

There are two options to calculate the average response length. The first option is to modify the `evaluate_math500_stream` function (listing 3.13 in chapter 3) by adding the following lines:

```
# ...
# below `num_correct = 0`
total_len = 0

# ...
# inside for i, row in enumerate(math_data, start=1):
# anywhere below `gen_text = ...`
total_len += len(tokenizer.encode(gen_text))

# ...
# anywhere at the bottom before the return statement
avg_len = total_len / num_examples
print(f"Average length: {avg_len:.2f} tokens")
```

Alternatively, the second option is to calculate the response lengths from the `.jsonl` files that were created when we ran the `evaluate_math500_stream` function in the main chapter. This way, we avoid having to rerun the evaluation.

First, we load the `.jsonl` file as follows:

```
import json
from pathlib import Path

WHICH_MODEL = "base"

local_path = Path(f"math500_{WHICH_MODEL}-mps.jsonl")  #A
if not local_path.exists():
    raise FileNotFoundError(
        f"{local_path} not found. Run ch03_main.ipynb to create
it."
    )

results = []
with open(local_path, "r") as f:
    for line in f:
```

```
        if line.strip():
            results.append(json.loads(line))

print("Number of entries:", len(results))
```

Let's print the dictionary keys to get a better idea of how the `results` dataset is structured:

```
print(results[0].keys())
```

This prints:

```
dict_keys(['index', 'problem', 'gtruth_answer',
'generated_text',
'extracted', 'correct'])
```

Note that each entry has multiple keys, however, we are only interested in the `"generated_text"` key, which contains the model's full answer. Next, we need to load the tokenizer so that we can tokenize the answer text before we can calculate the number of tokens. This is similar to the code we used in listing 3.1 in chapter 3:

```
from reasoning_from_scratch.qwen3 import (
    download_qwen3_small,
    Qwen3Tokenizer
)

if WHICH_MODEL == "base":

    download_qwen3_small(
        kind="base", tokenizer_only=True, out_dir="qwen3"
    )
    tokenizer_path = Path("qwen3") / "tokenizer-base.json"
    tokenizer =
Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)

elif WHICH_MODEL == "reasoning":

    download_qwen3_small(
        kind="reasoning", tokenizer_only=True, out_dir="qwen3"
    )
    tokenizer_path = Path("qwen3") / "tokenizer-reasoning.json"
    tokenizer = Qwen3Tokenizer(
        tokenizer_file_path=tokenizer_path,
        apply_chat_template=True,
```

```
        add_generation_prompt=True,
        add_thinking=True,
    )
```

Then, we can calculate the average length as follows, which is similar to how we could have modified the `evaluate_math500_stream` function:

```
total_len = 0

for item in results:
    num_tokens = len(tokenizer.encode(item["generated_text"]))
    total_len += num_tokens

avg_len = total_len / len(results)
print(f"Average length: {avg_len:.2f} tokens")
```

The resulting average length is as follows:

```
Average length: 98.00 tokens
```

Table B.1 lists the average lengths for the different models and subsets.

Table B.1 Average number of tokens on MATH-500

Model	Device	Average length	MATH-500 size
Base	CPU	97.30	10
Base	CUDA	96.74	500
Reasoning	CPU	891.80	10
Reasoning	CUDA	1361.21	500

As we can see based on the results in table B.1, and as expected, the reasoning model generates much longer responses (in this case, approximately 10-times longer).

Exercise 3.3

To evaluate the model on a larger dataset, we can simply change the `math_data[:10]` to a different slice or larger number (up to 500) in the following function call:

```
num_correct, num_examples, acc = evaluate_math500_stream(
    model, tokenizer, device,
    math_data=math_data[:10],
    max_new_tokens=2048,
    verbose=False
)
```

Table B.2 below shows the accuracy values for different dataset sizes. (Since the MATH-500 test set is already shuffled, no additional shuffling was applied.)

Table B.2 Accuracies for different MATH-500 dataset sizes

Model	Device	Accuracy	MATH-500 size
Base	CUDA	30.0%	10
Base	CUDA	34.0%	50
Base	CUDA	27.0%	100
Base	CUDA	15.3%	500
Reasoning	CUDA	90.0%	10
Reasoning	CUDA	58.0%	50
Reasoning	CUDA	56.0%	100
Reasoning	CUDA	50.8%	500

As we can see based on the results in table B.2, the first 10 examples are not very representative of the MATH-500 performance evaluated on the whole 500 examples.

In addition, we can create an entirely new dataset in a similar style to MATH-500. For example, a dataset in MATH-500 style is included in this repository; we can use it in the main chapter by changing the filename from `math500_test.json` to `math_new50_exercise.json` (this dataset is included in this book's GitHub repository at https://github.com/rasbt/reasoning-from-scratch/tree/main/ch03/01_main-chapter-code).

The performance of the models is as follows:

- base: 36.0% (18/50)
- reasoning: 80.0% (40/50)

Accuracy is similar for the base model and higher for the reasoning model compared to the 50-example subset of the MATH-500 test set (table B.2). This indicates that, despite the possibility of overlap with Qwen3's training data, the model generalizes well to new math questions and does not show signs of extensive overfitting to the original MATH-500 data.

Exercise 3.4

We could use the alternative prompt similar to the one suggested in the chapter, which modifies the prompt to use the word "problem" instead of "question":

```
def render_prompt(prompt):
    template = (
        "You are a helpful math assistant.\n"
        "Solve the problem and write the final "
        "result on a new line as:\n"
        "\\boxed{ANSWER}\n\n"
        f"Problem:\n{prompt}\n\nAnswer:"
    )
    return template
```

Using this prompt improves the performance of the base model, on the 500 examples, from 15.3% to 31.2%. And vice versa, it reduces the performance of the reasoning model from 50.8% to 50.0%

From these observations, we may conclude that the base model is much more sensitive to the prompt format (likely due to memorizing some prompt-formatted MATH-500 examples from the training set) than the reasoning model; the latter seems largely unaffected.

OceanofPDF.com

Appendix C. Qwen3 LLM source code

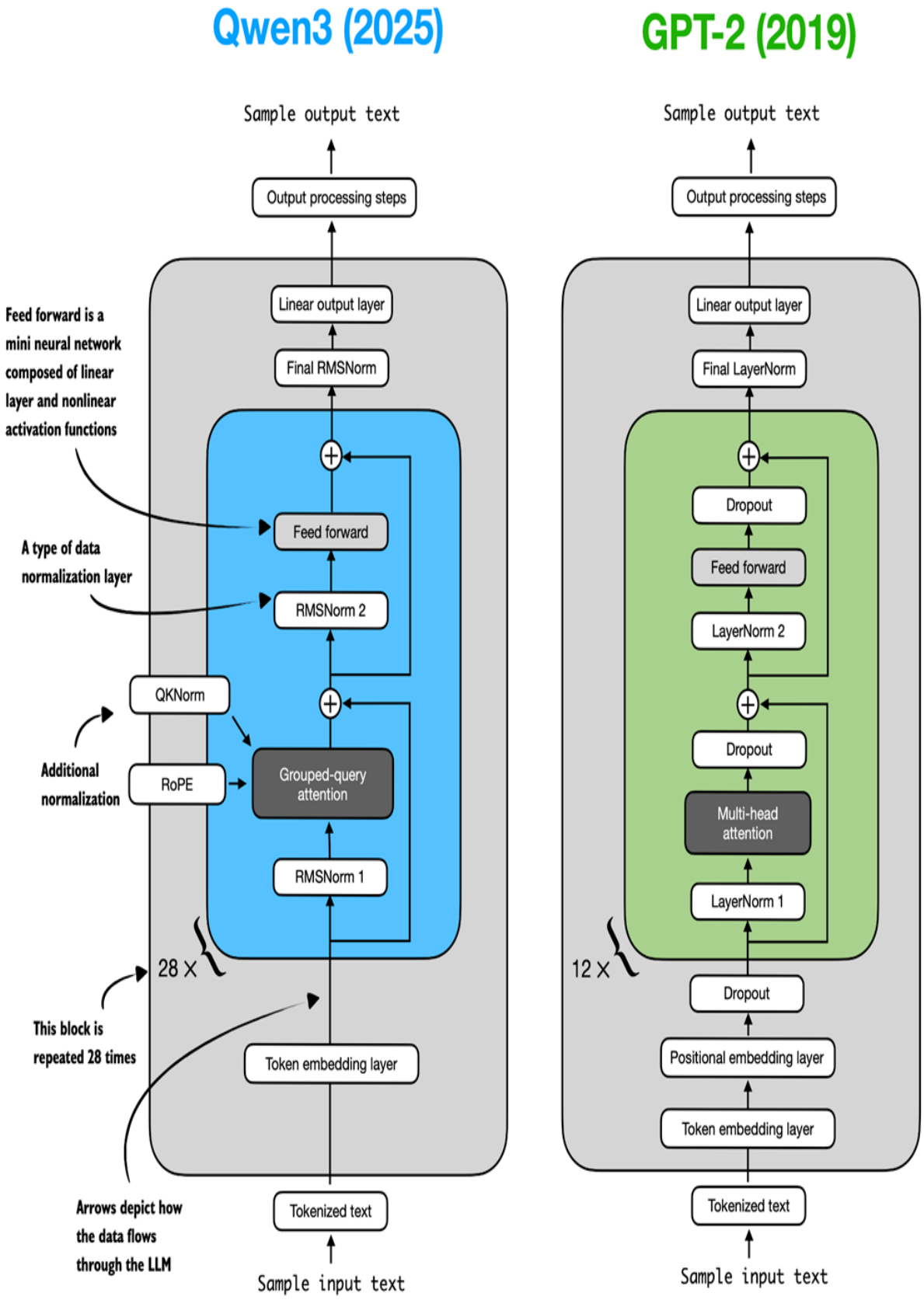
While this is a *from scratch* book, as mentioned in the main chapters, the *from scratch* part refers to the reasoning techniques, not the LLM itself. Implementing an LLM entirely from scratch would require a separate book, which is the topic of my *Build A Large Language Model (From Scratch)* book (http://mng.bz/orYv).

However, for readers interested in seeing the Qwen3 implementation we use in this *Build A Reasoning Model (From Scratch)* book, this appendix lists the source code for the `Qwen3Model` model that I implemented in and that we import from the book's `reasoning_from_scratch` Python package:

```
from reasoning_from_scratch.qwen3 import Qwen3Model,
Qwen3Tokenizer
```

As shown in figure C.1, the Qwen3 architecture is very similar to GPT-2, which is covered in my *Build A Large Language Model (From Scratch)* book. While familiarity with GPT-2 is not required for this book, this appendix mentions comparisons to GPT-2 for those who are familiar with it. In fact, I wrote the Qwen3 implementation by porting the GPT-2 model from my other book piece by piece into the Qwen3 architecture, such that it follows similar style conventions to improve readability.

Figure C.1 Architectural comparison between Qwen3 and GPT-2. Both models process text through embedding layers and stacked transformer blocks, but they differ in certain design choices.


Qwen3 (2025)
GPT-2 (2019)
Sample output text
Output processing steps
Linear output layer
Final RMSNorm
Feed forward is a mini neural network composed of linear layer and nonlinear activation functions
Feed forward
A type of data normalization layer
RMSNorm 2
QKNorm
Additional normalization
RoPE
Grouped-query attention
RMSNorm 1
28 X
This block is repeated 28 times
Token embedding layer
Arrows depict how the data flows through the LLM
Tokenized text
Sample input text
Sample output text
Output processing steps
Linear output layer
Final LayerNorm
Dropout
Feed forward
LayerNorm 2
Dropout
Multi-head attention
LayerNorm 1
12 X
Dropout
Positional embedding layer
Token embedding layer
Tokenized text
Sample input text

As shown in figure C.1, both Qwen3 (released in 2025) and GPT-2 (released in 2019) are very similar overall in that they are both based on the decoder submodule of the original transformer architecture. However, some of the design choices have evolved since 2019. Note that most of these design choices found in Qwen3 are not unique to Qwen3 but are found in many other contemporary LLMs, which I discussed in my *The Big LLM Architecture Comparison* (https://magazine.sebastianraschka.com/p/the-big-llm-architecture-comparison) article.

For readers new to LLMs who want to understand how these architectures are implemented, I recommend starting with GPT-2. Its design is simpler to implement, which makes it an easier entry point before exploring more modern variations.

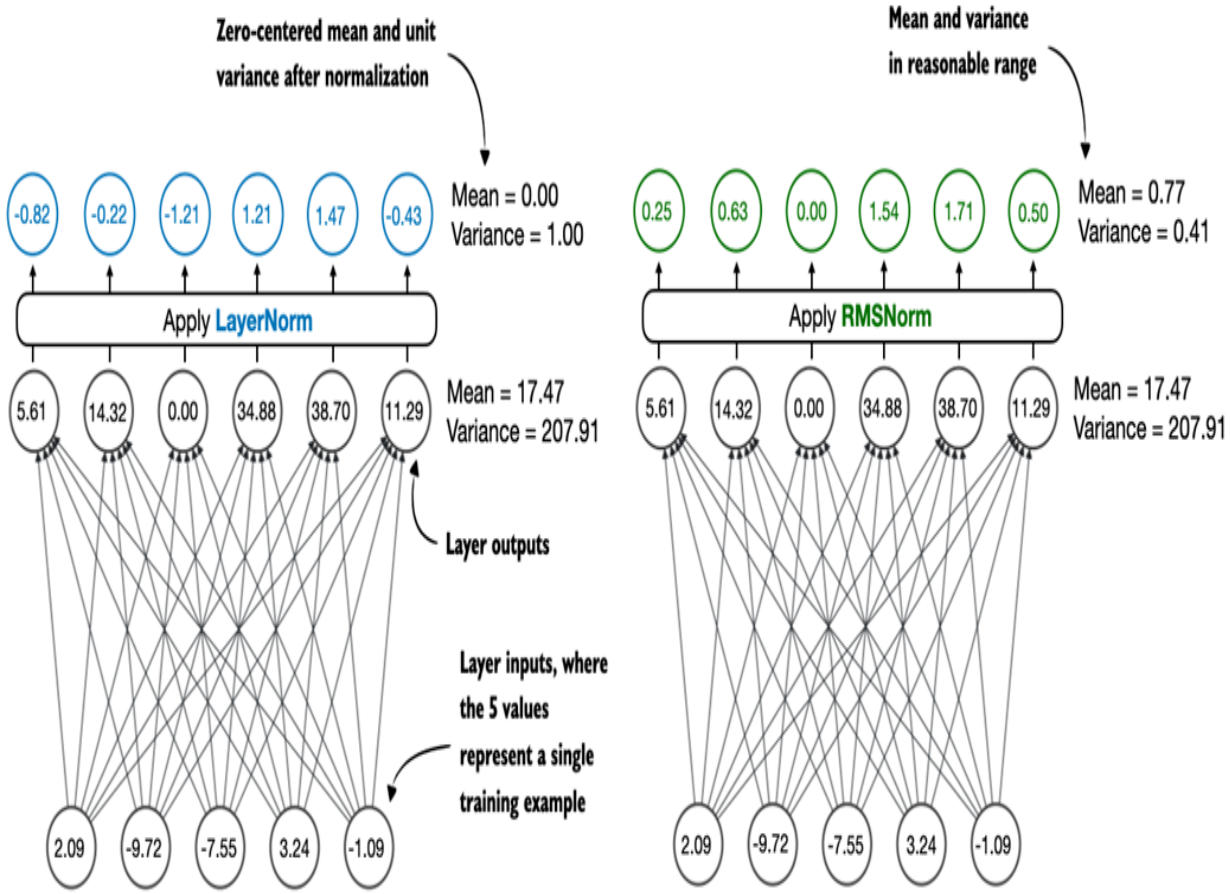
Since this book does not focus on architecture implementations, the remainder of this appendix will cover only a brief overview of Qwen3's code.

C.1 Root mean square layer normalization (RMSNorm)

In contrast to GPT-2, which used standard *LayerNorm*, the newer Qwen3 architecture replaces it with *root mean square layer normalization* (*RMSNorm*). This is a trend that has become increasingly common in recent model architectures.

RMSNorm fulfills the same core function as LayerNorm: normalizing layer activations to stabilize and improve training. However, it simplifies the computation by removing the mean-centering step, as shown in figure C.2. This means that activations will still be normalized, but they are not centered at 0.

Figure C.2 Comparison of LayerNorm and RMSNorm. LayerNorm (left) normalizes activations so that their average value (mean) is exactly zero and their spread (variance) is exactly one. RMSNorm (right) instead scales activations based on their root mean square, which does not enforce zero mean or unit variance, but still keeps the mean and variance within a reasonable range for stable training.



As we can see in figure C.2, both LayerNorm and RMSNorm scale the layer outputs to be in a reasonable range.

LayerNorm subtracts the mean and divides by the standard deviation such that the layer outputs have a zero mean and unit variance (variance of one and standard deviation of one), which results in favorable properties, in terms of gradient values, for stable training.

RMSNorm divides the inputs by the root mean square. This scales activations to a comparable magnitude without enforcing zero mean or unit variance. In this particular example shown in figure C.2, the mean is 0.77 and the variance is 0.41.

Both LayerNorm and RMSNorm stabilize activation scales and improve optimization; however, RMSNorm is often preferred in large-scale LLMs because it is computationally cheaper. Unlike LayerNorm, RMSNorm does not use a bias (shift) term by default, which reduces the number of trainable

parameters. Moreover, RMSNorm reduces the expensive mean and variance computations to a single root-mean-square operation. This reduces the number of cross-feature reductions from two to one, which lowers communication overhead on GPUs and slightly improves training efficiency.

Listing C.1 shows what RMSNorm looks like in code.

Listing C.1 RMSNorm

```
import torch.nn as nn

class RMSNorm(nn.Module):

    def __init__(
        self,
        emb_dim,
        eps=1e-6,
        bias=False,
        qwen3_compatible=True,
    ):
        super().__init__()
        self.eps = eps
        self.qwen3_compatible = qwen3_compatible
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim)) if bias
else None

    def forward(self, x):
        input_dtype = x.dtype

        if self.qwen3_compatible:
            x = x.to(torch.float32)

        variance = x.pow(2).mean(dim=-1, keepdim=True)
        norm_x = x * torch.rsqrt(variance + self.eps)
        norm_x = norm_x * self.scale

        if self.shift is not None:
            norm_x = norm_x + self.shift

        return norm_x.to(input_dtype)
```

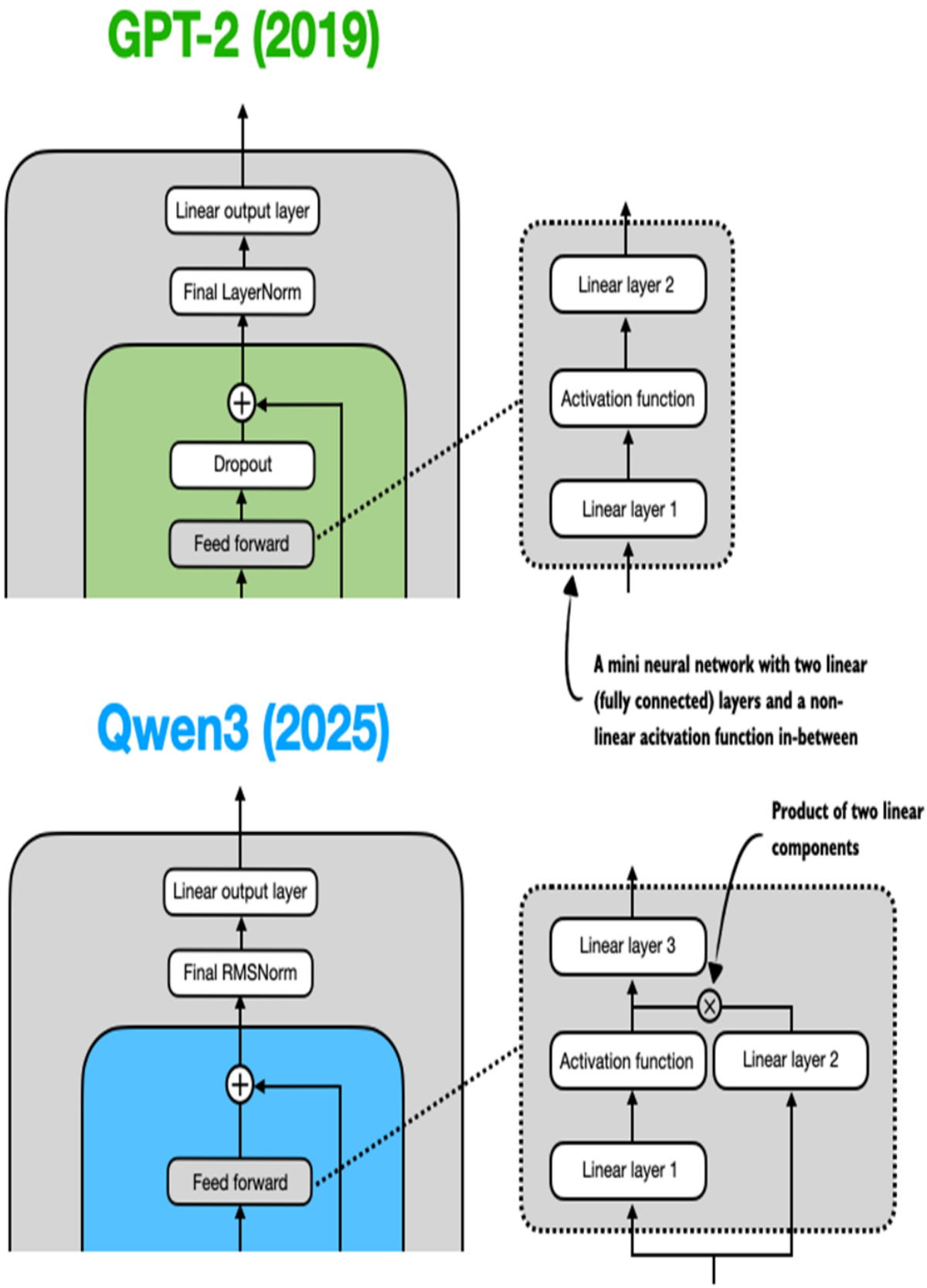
Note that, for brevity, this appendix does not provide detailed code walkthroughs for each LLM component. Instead, in section C.6, we will

integrate all components into the `Qwen3Model` class, load the pre-trained weights into it, and then use this model to generate text in section C.9.

C.2 Feed forward module

The *feed forward module* (a small multi-layer perceptron) is replaced with a *gated linear unit* (*GLU*) variant, introduced in a 2020 paper (https://arxiv.org/abs/2002.05202). In this design, the standard two fully connected layers are replaced by three, as shown in figure C.3.

Figure C.3 In GPT-2 (top), the feed forward module consists of two fully connected (linear) layers separated by a non-linear activation function. In Qwen3 (bottom), this module is replaced with a gated linear unit (GLU) variant, which adds a third linear layer and multiplies its output elementwise with the activated output of the second linear layer.


GPT-2 (2019)
Linear output layer
Final LayerNorm
Dropout
Feed forward
Linear layer 2
Activation function
Linear layer 1
A mini neural network with two linear (fully connected) layers and a non-linear acitvation function in-between
Qwen3 (2025)
Linear output layer
Final RMSNorm
Feed forward
Product of two linear components
Linear layer 3
Activation function
Linear layer 2
Linear layer 1

Qwen3's feed forward module (figure C.3) can be implemented as shown in listing C.2.

Listing C.2 Qwen3 feed forward module

```
class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.fc1 = nn.Linear(
            cfg["emb_dim"], cfg["hidden_dim"],
dtype=cfg["dtype"],
            bias=False
        )
        self.fc2 = nn.Linear(
            cfg["emb_dim"], cfg["hidden_dim"],
dtype=cfg["dtype"],
            bias=False
        )
        self.fc3 = nn.Linear(
            cfg["hidden_dim"], cfg["emb_dim"],
dtype=cfg["dtype"],
            bias=False
        )

    def forward(self, x):
        x_fc1 = self.fc1(x)
        x_fc2 = self.fc2(x)
        x = nn.functional.silu(x_fc1) * x_fc2  #A
        return self.fc3(x)
```

At first glance, it might seem that the GLU feed forward variant used in Qwen3 should outperform the standard feed forward variant in GPT-2, simply because it adds an extra linear layer (three instead of two) and therefore appears to have more parameters.

However, this intuition is misleading. In practice, the `fc1` and `fc2` layers in the GLU variant are each half the width of the `fc1` layer in a standard feed forward module, and in practice, it has fewer parameters.

To illustrate this with a concrete example, suppose the input dimension to the "Linear layer 1" in figure C.3 is 1024. This corresponds to `cfg["emb_dim"]` in listing C.2. The output dimension of `fc1` is 3,072

(`cfg["hidden_dim"]`). Note that these are the actual numbers used in the Qwen3 0.6B variant. In this case, we have the following parameter counts for the GLU variant in listing C.2:

- `fc1`: 1024 × 3,072 = 3,145,728
- `fc2`: 1024 × 3,072 = 3,145,728
- `fc3`: 1024 × 3,072 = 3,145,728
- Total: 3 × 3,145,728 = 9,437,184 parameters

If we assume that `fc1` in this GLU variant has half the width as would be typically chosen for an `fc1` in a standard feed forward module, the parameter counts of the standard feed forward module would be as follows:

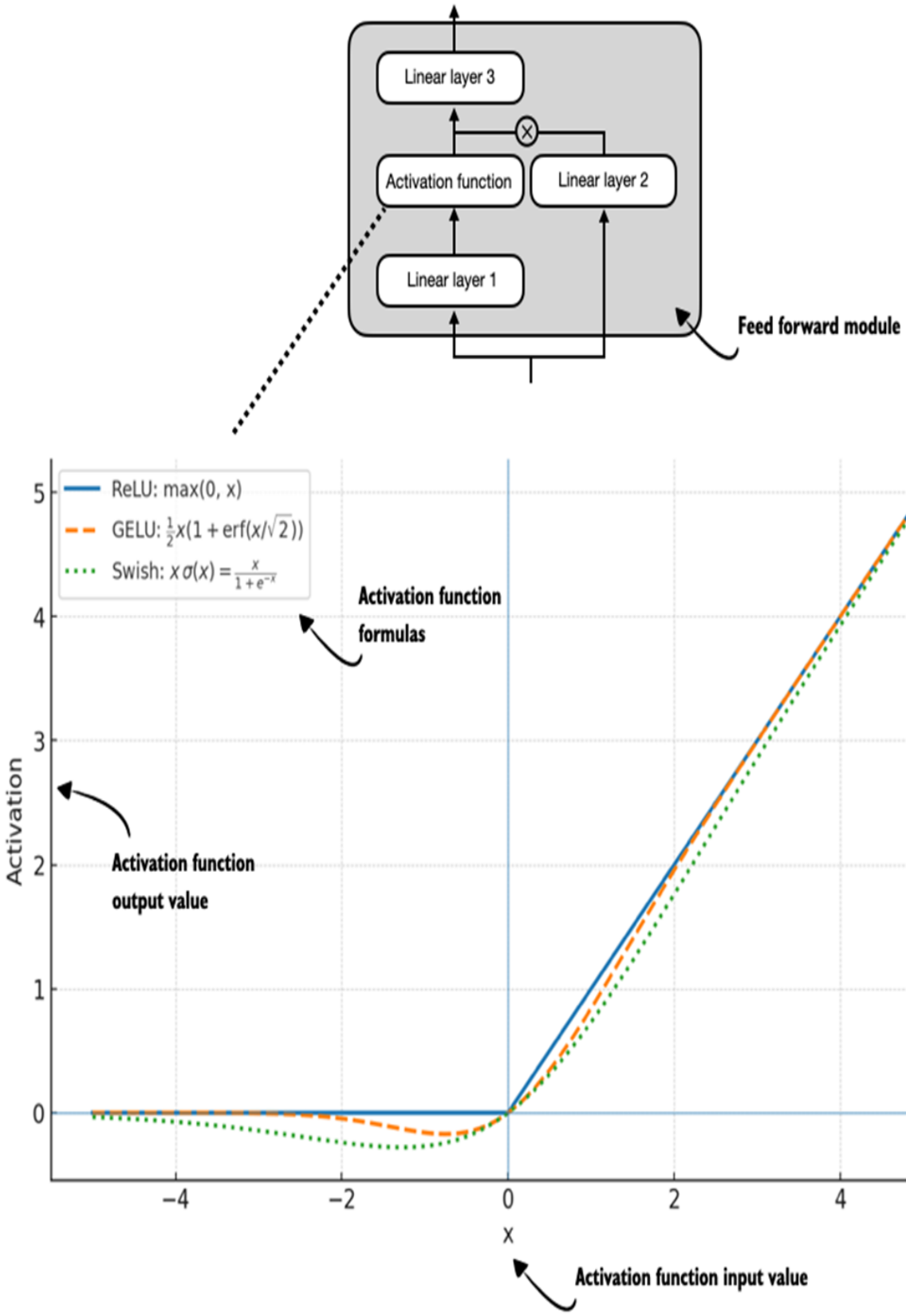
- `fc1`: 1024 × 2×3,072 = 6,291,456
- `fc2`: 1024 × 2×3,072 = 6,291,456
- Total: 2 × 6,291,456 = 12,582,912 parameters

While GLU variants usually have fewer parameters than regular feed forward modules, they perform better. The improvement comes from the additional multiplicative interaction introduced by the gating mechanism, `activation(x_fc1) * x_fc2`, which increases the model's expressivity. This is similar to how deeper, slimmer networks can outperform shallower, wider ones, given proper training.

Before we proceed to the next section, there is one more thing to address. Note that the feed forward module shown in figure C.3 contains an element labeled as "Activation function, " whereas we used a `nn.functional.silu` activation as a concrete example in listing C.2.

Historically, activation functions were a hot topic of debate until the deep learning community largely converged on the *rectified linear unit* (*ReLU*) more than a decade ago. ReLU is simple and computationally cheap, but it has a sharp kink at zero. This motivated researchers to explore smoother functions such as the *Gaussian error linear unit* (*GELU*) and the *sigmoid linear unit* (*SiLU*), as shown in figure C.4.

Figure C.4 Different activation functions that can be used in a feed forward module (neural network). GELU and SiLU (Swish) offer smooth alternatives to ReLU, which has a sharp kink at input zero.


Linear layer 3
Activation function
Linear layer 2
Linear layer 1
Feed forward module
ReLU: max(0, x)
GELU: $\frac{1}{2}x(1+\text{erf}(x/\sqrt{2}))$
Swish: $x\,\sigma(x)=\frac{x}{1+e^{-x}}$
Activation function formulas
Activation function output value
Activation
x
Activation function input value

GELU involves the Gaussian cumulative distribution function (CDF). Computing this CDF is slow because it uses piecewise logic and exponentials, which makes it hard to write fused, optimized GPU kernels (although a tanh approximation exists that uses cheaper operations and runs faster with near-identical results).

In short, while GELU produces smooth activation curves, it is overall computationally more expensive than simpler functions.

Newer models have largely replaced GELU with the SiLU (also known as *Swish*) function, which smoothly suppresses large negative inputs toward ~0 and is approximately linear for large positive inputs, as shown in figure C.4.

SiLU has a similar smoothness, but it is slightly cheaper to compute than GELU and offers comparable modeling performance. In practice, SiLU is now used in most architectures, while GELU remains in use in only some models, such as Google's Gemma open-weight LLM. In the implementation of the feed forward module in listing C.2, this SiLU function is called via `nn.functional.silu`. The feed forward module in listing C.2 is also often called *SwiGLU*, an abbreviation that is derived from the terms Swish and GLU.

C.3 Rotary position embeddings (RoPE)

In transformer-based LLMs, positional encoding is necessary because of the attention mechanism. By default, attention treats the input tokens as if they have no order. In the original GPT architecture, absolute positional embeddings addressed this by adding a learned embedding vector for each position in the sequence, which is then added to the token embeddings.

RoPE (short for *rotary position embeddings*) introduced a different approach: instead of adding position information as separate embeddings, it encodes position information by rotating the query and key vectors in the attention mechanism (section C.4) in a way that depends on each token's position. RoPE is an elegant idea, but also a long topic in itself. Interested readers can find more information in the original RoPE paper at https://arxiv.org/abs/2104.09864. (While first introduced in 2021, RoPE

became widely adopted with the release of the original Llama model in 2023 and has since become a staple in modern LLMs, so it is not unique to Qwen3.)

RoPE can be implemented in two mathematically equivalent ways: the interleaved form, which pairs adjacent dimensions for rotation, or in a two-halves form, which splits the dimension into cosine and sine halves for convenience. Listing C.3 implements the two-halves variant, which can be easier to read.

Listing C.3 RoPE functions

```
import torch


def compute_rope_params(head_dim, theta_base=10_000,
context_length=4096,
                        dtype=torch.float32):
    assert head_dim % 2 == 0, "Embedding dimension must be even"
    inv_freq = 1.0 / (theta_base ** (
        torch.arange(0, head_dim, 2, dtype=dtype)[: (head_dim //
2)].float()
        / head_dim
    ))
    positions = torch.arange(context_length, dtype=dtype)
    angles = positions.unsqueeze(1) * inv_freq.unsqueeze(0)
    angles = torch.cat([angles, angles], dim=1)

    cos = torch.cos(angles)
    sin = torch.sin(angles)

    return cos, sin


def apply_rope(x, cos, sin, offset=0):
    batch_size, num_heads, seq_len, head_dim = x.shape    #A
    assert head_dim % 2 == 0, "Head dimension must be even"

    x1 = x[..., : head_dim // 2]  # First half    #B
    x2 = x[..., head_dim // 2:]  # Second half    #B


    cos = cos[offset:offset + seq_len,
:].unsqueeze(0).unsqueeze(0)
    sin = sin[offset:offset + seq_len,
```

```
:].unsqueeze(0).unsqueeze(0)
    # Shape after:  (1, 1, seq_len, head_dim)

    rotated = torch.cat((-x2, x1), dim=-1)
    x_rotated = (x * cos) + (rotated * sin)

    return x_rotated.to(dtype=x.dtype)    #C
```

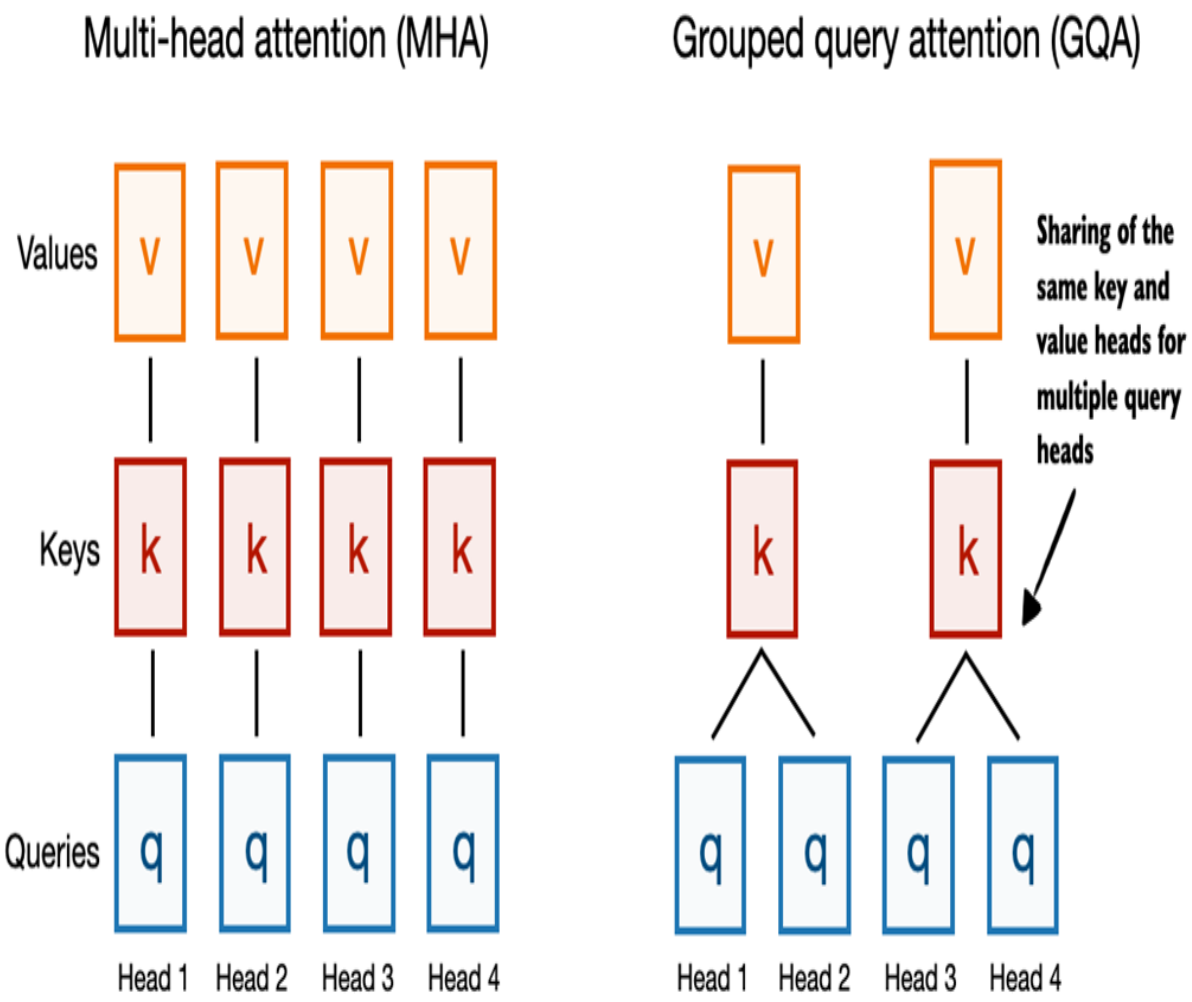
The RoPE code in listing C.3 will be used in the grouped query attention mechanism in section C.4.

C.4 Grouped query attention (GQA)

Grouped query attention (*GQA*) has become the standard, more compute- and parameter-efficient alternative to the original multi-head attention (MHA) mechanism.

Unlike MHA, where each head also has its own set of keys and values, to reduce memory usage, GQA groups multiple heads to share the same key and value projections, as shown in figure C.5.

Figure C.5 A comparison between MHA and GQA. Here, the group size is 2, where a key and value pair is shared among 2 queries.



So, the core idea behind GQA, shown in figure C.5, is to reduce the number of key and value heads by sharing them across multiple query heads. This (1) lowers the model's parameter count and (2) reduces the memory bandwidth usage for key and value tensors during inference since fewer keys and values need to be stored and retrieved from the *KV cache* (section C.7).

While GQA is primarily a computational efficiency workaround for MHA, ablation studies (as presented in the original GQA paper, https://arxiv.org/abs/2305.13245) show that it performs comparably to standard MHA in terms of LLM modeling performance.

Listing C.4 implements the GQA mechanism with KV cache support.

Listing C.4 Grouped query attention

```
class GroupedQueryAttention(nn.Module):
    def __init__(self, d_in, num_heads, num_kv_groups,
head_dim=None,
                 qk_norm=False, dtype=None):
        super().__init__()
        assert num_heads % num_kv_groups == 0

        self.num_heads = num_heads
        self.num_kv_groups = num_kv_groups
        self.group_size = num_heads // num_kv_groups

        if head_dim is None:
            assert d_in % num_heads == 0
            head_dim = d_in // num_heads

        self.head_dim = head_dim
        self.d_out = num_heads * head_dim

        self.W_query = nn.Linear(
            d_in, self.d_out, bias=False, dtype=dtype
        )
        self.W_key = nn.Linear(
            d_in, num_kv_groups * head_dim,
bias=False,dtype=dtype
        )
        self.W_value = nn.Linear(
            d_in, num_kv_groups * head_dim, bias=False,
dtype=dtype
        )

        self.out_proj = nn.Linear(self.d_out, d_in, bias=False,
dtype=dtype)

        if qk_norm:
            self.q_norm = RMSNorm(head_dim, eps=1e-6)
            self.k_norm = RMSNorm(head_dim, eps=1e-6)
        else:
            self.q_norm = self.k_norm = None

    def forward(self, x, mask, cos, sin, start_pos=0,
cache=None):
        b, num_tokens, _ = x.shape

        queries = self.W_query(x)            #A
        keys = self.W_key(x)                 #B
        values = self.W_value(x)             #B

        queries = queries.view(b, num_tokens, self.num_heads,
```

```
                                self.head_dim).transpose(1, 2)
        keys_new = keys.view(b, num_tokens, self.num_kv_groups,
                             self.head_dim).transpose(1, 2)
        values_new = values.view(b, num_tokens,
self.num_kv_groups,
                                 self.head_dim).transpose(1, 2)

        if self.q_norm:
            queries = self.q_norm(queries)
        if self.k_norm:
            keys_new = self.k_norm(keys_new)

        queries = apply_rope(queries, cos, sin,
offset=start_pos)
        keys_new = apply_rope(keys_new, cos, sin,
offset=start_pos)

        if cache is not None:
            prev_k, prev_v = cache
            keys = torch.cat([prev_k, keys_new], dim=2)
            values = torch.cat([prev_v, values_new], dim=2)
        else:
            start_pos = 0                       #C
            keys, values = keys_new, values_new
        next_cache = (keys, values)


        keys = keys.repeat_interleave(       #D
            self.group_size, dim=1           #D
        )                                    #D
        values = values.repeat_interleave(   #D
            self.group_size, dim=1           #D
        )                                    #D

        attn_scores = queries @ keys.transpose(2, 3)
        attn_scores = attn_scores.masked_fill(mask, -torch.inf)
        attn_weights = torch.softmax(
            attn_scores / self.head_dim**0.5, dim=-1
        )

        context = (attn_weights @ values).transpose(1, 2)
        context = context.reshape(b, num_tokens, self.d_out)
        return self.out_proj(context), next_cache
```

You may have noticed that the GQA mechanism in listing C.4 also includes a `qk_norm` parameter. This is not part of the standard GQA design. When `qk_norm=True`, an additional Query/Key-RMSNorm-based normalization,

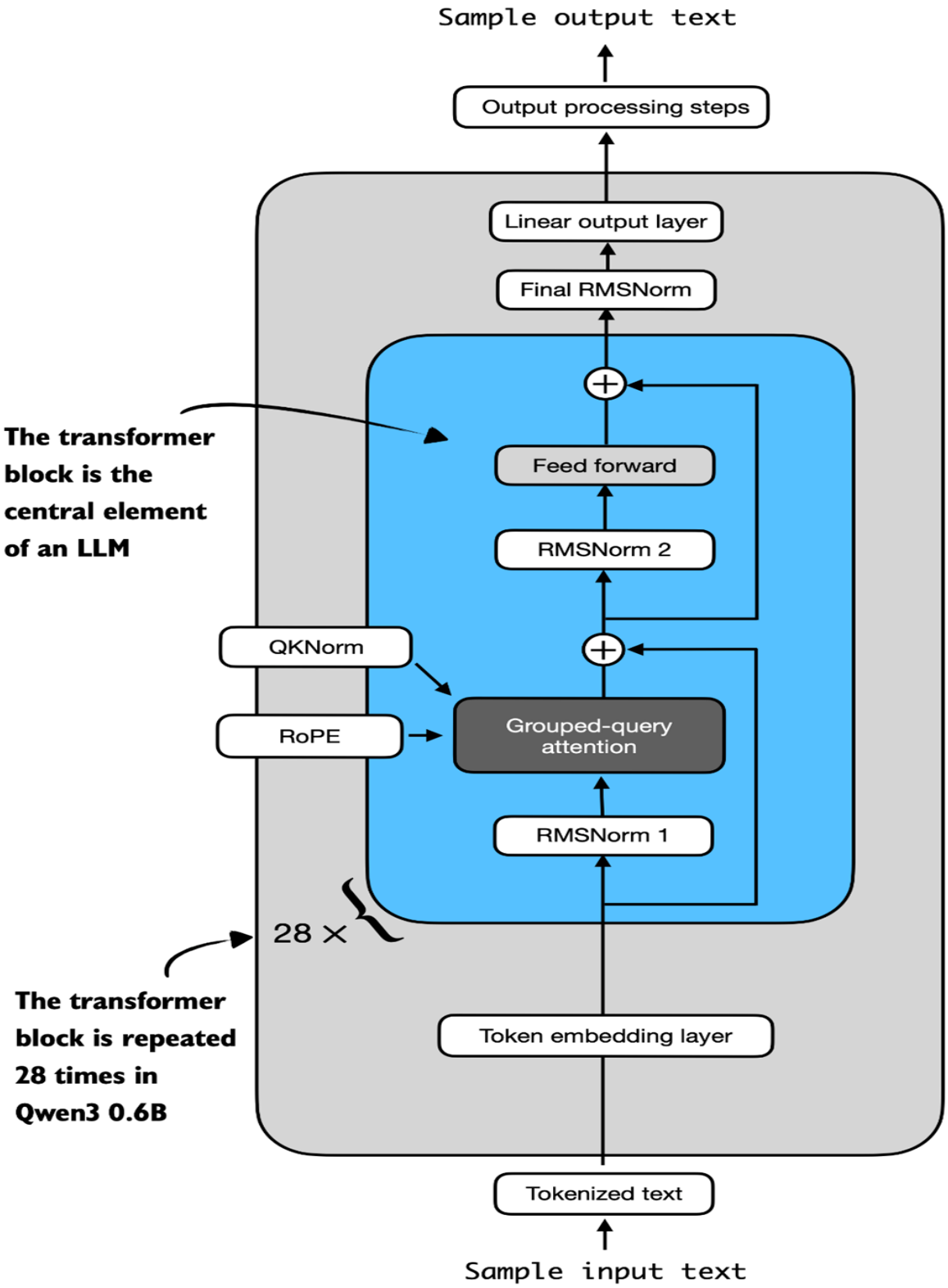
called *QKNorm*, is applied to both the queries and keys, which is a technique used in Qwen3. As discussed earlier in the RMSNorm section (section C.1), QKNorm helps improve training stability.

C.5 Transformer block

The *transformer block* is the central component of an LLM, which combines all the individual elements covered in this appendix so far. As shown in figure C.6, it is repeated multiple times; in the 0.6-billion-parameter version of Qwen3, it is repeated 28 times.

Figure C.6 The Structure of the transformer block in Qwen3. Each block includes RMSNorm, RoPE, masked grouped-query attention, and a feed-forward module, and is repeated 28 times in the 0.6B-parameter model.

Qwen3 0.6B


Sample output text
Output processing steps
Linear output layer
Final RMSNorm
Feed forward
RMSNorm 2
QKNorm
RoPE
Grouped-query attention
RMSNorm 1
28 ×
Token embedding layer
Tokenized text
Sample input text
The transformer block is the central element of an LLM
The transformer block is repeated 28 times in Qwen3 0.6B

Listing C.5 implements the transformer block shown in figure C.6.

Listing C.5 Transformer block

```
class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.att = GroupedQueryAttention(
            d_in=cfg["emb_dim"],
            num_heads=cfg["n_heads"],
            head_dim=cfg["head_dim"],
            num_kv_groups=cfg["n_kv_groups"],
            qk_norm=cfg["qk_norm"],
            dtype=cfg["dtype"]
        )
        self.ff = FeedForward(cfg)
        self.norm1 = RMSNorm(cfg["emb_dim"], eps=1e-6)
        self.norm2 = RMSNorm(cfg["emb_dim"], eps=1e-6)

    def forward(self, x, mask, cos, sin, start_pos=0,
cache=None):
        shortcut = x
        x = self.norm1(x)
        x, next_cache = self.att(
            x, mask, cos, sin, start_pos=start_pos,cache=cache
        )  # A
        x = x + shortcut

        shortcut = x
        x = self.norm2(x)
        x = self.ff(x)
        x = x + shortcut

        return x, next_cache
```

As we can see, in listing C.5, the transformer block simply connects various elements we implemented in previous sections.

C.6 Main model code

In this section, we will define the `Qwen3Model` class that we imported and used in chapter 2.

To implement the `Qwen3Model` class, the code in listing C.6 follows the architecture previously shown in figure C.6, where the transformer block sits at the heart of the LLM.

Listing C.6 Main Qwen3Model code

```
class Qwen3Model(nn.Module):
    def __init__(self, cfg):
        super().__init__()

        # Main model parameters
        self.tok_emb = nn.Embedding(cfg["vocab_size"],
cfg["emb_dim"],
                                    dtype=cfg["dtype"])

        self.trf_blocks = nn.ModuleList(
            [TransformerBlock(cfg) for _ in
range(cfg["n_layers"])]
        )
        self.final_norm = RMSNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(
            cfg["emb_dim"], cfg["vocab_size"],
            bias=False, dtype=cfg["dtype"]
        )

        # Reusable utilities
        if cfg["head_dim"] is None:
            head_dim = cfg["emb_dim"] // cfg["n_heads"]
        else:
            head_dim = cfg["head_dim"]
        cos, sin = compute_rope_params(
            head_dim=head_dim,
            theta_base=cfg["rope_base"],
            context_length=cfg["context_length"]
        )
        self.register_buffer("cos", cos, persistent=False)
        self.register_buffer("sin", sin, persistent=False)
        self.cfg = cfg
        self.current_pos = 0  # Track current position in KV
cache

    def forward(self, in_idx, cache=None):
        tok_embeds = self.tok_emb(in_idx)
        x = tok_embeds

        num_tokens = x.shape[1]
```

```
        if cache is not None:
            pos_start = self.current_pos
            pos_end = pos_start + num_tokens
            self.current_pos = pos_end
            mask = torch.triu(
                torch.ones(
                    pos_end, pos_end, device=x.device,
dtype=torch.bool
                ),
                diagonal=1
            )[pos_start:pos_end, :pos_end]
        else:
            pos_start = 0  # Not strictly necessary but helps
torch.compile
            mask = torch.triu(
                torch.ones(num_tokens, num_tokens,
device=x.device,
                           dtype=torch.bool),
                diagonal=1
            )

        mask = mask[None, None, :, :]                #A

        for i, block in enumerate(self.trf_blocks):
            blk_cache = cache.get(i) if cache else None
            x, new_blk_cache = block(x, mask, self.cos,
self.sin,
                                     start_pos=pos_start,
                                     cache=blk_cache)
            if cache is not None:
                cache.update(i, new_blk_cache)

        x = self.final_norm(x)
        logits = self.out_head(x.to(self.cfg["dtype"]))
        return logits

    def reset_kv_cache(self):
        self.current_pos = 0
```

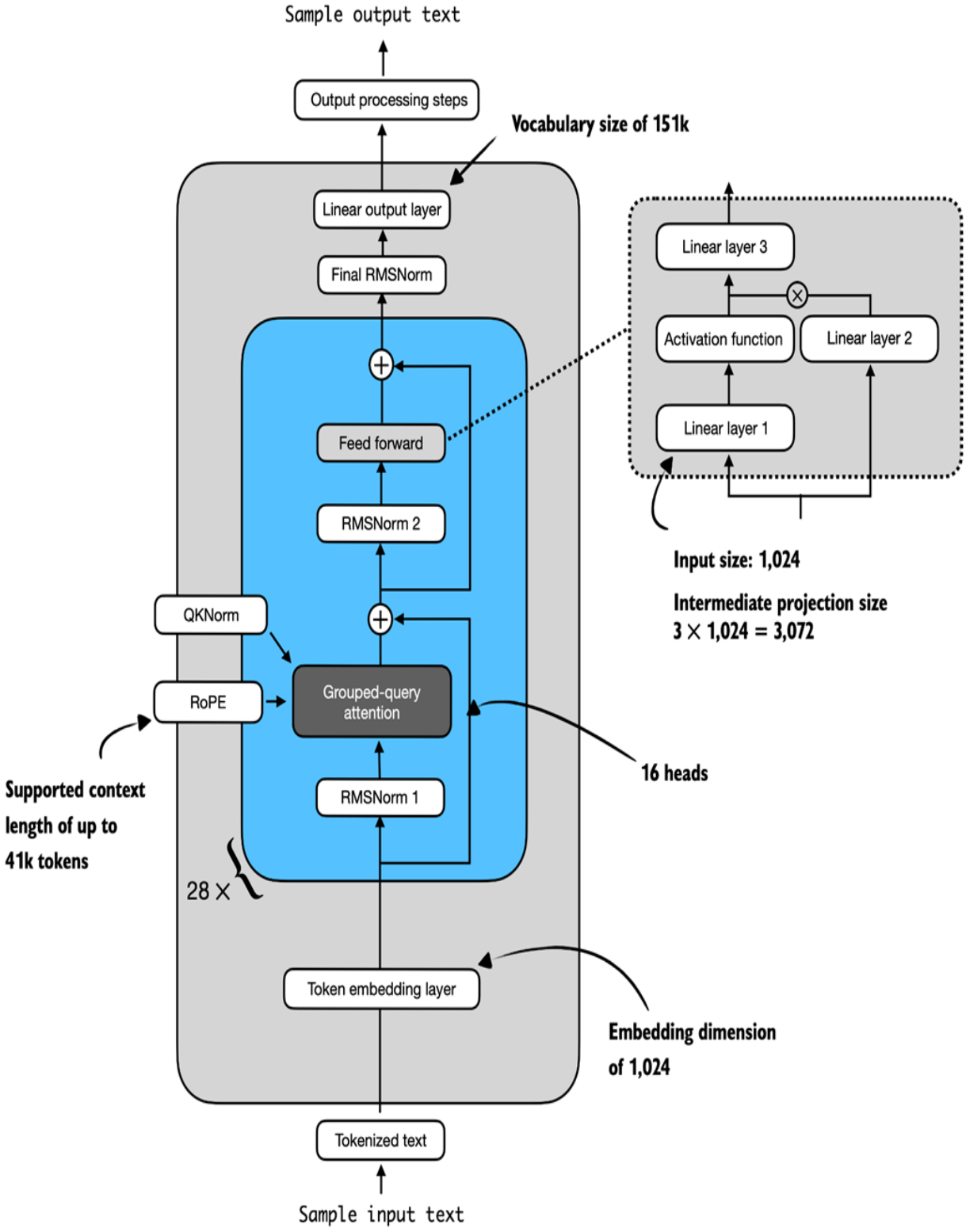
Since we already have all the main ingredients, the `Qwen3Model` class in listing C.6 only adds a few more components around the transformer block, namely the embedding and output layers (including one more RMSNorm layer). However, the code may appear somewhat complicated, which is due to the KV cache option.

As discussed in chapter 2, the KV cache can speed up the text generation process, but it is a topic outside the scope of this book. Interested readers can find more information about KV caching in my *Understanding and Coding the KV Cache in LLMs from Scratch* article at https://magazine.sebastianraschka.com/p/coding-the-kv-cache-in-llms.

Note that the `Qwen3Model` class, as implemented in listing C.6, supports various model sizes (see appendix D for more information). In chapter 2, we use the 0.6-billion-parameter model as it is the least resource-intensive model in the Qwen3 model family. The specific configuration of this model is visualized in figure C.7.

Figure C.7 Architecture of the Qwen3 0.6B model. The model consists of a token embedding layer followed by 28 transformer blocks, each containing RMSNorm, RoPE, QKNorm, masked grouped-query attention with 16 heads, and a feed-forward module with an intermediate size of 3,072.

Qwen3 0.6B


Sample output text
Output processing steps
Vocabulary size of 151k
Linear output layer
Final RMSNorm
Feed forward
RMSNorm 2
QKNorm
RoPE
Grouped-query attention
RMSNorm 1
28 ×
Token embedding layer
Tokenized text
Sample input text
Linear layer 3
Activation function
Linear layer 2
Linear layer 1
Input size: 1,024
Intermediate projection size
3 × 1,024 = 3,072
16 heads
Supported context length of up to 41k tokens
Embedding dimension of 1,024

To use the 0.6B model shown in figure C.7 via the `Qwen3Model` class, we can define the following configuration in listing C.7 that we provide as input (`cfg=QWEN_CONFIG_06_B`) upon instantiating a new `Qwen3Model` instance.

Listing C.7 Qwen3 0.6B configuration

```
QWEN_CONFIG_06_B = {
    "vocab_size": 151_936,     # Vocabulary size
    "context_length": 40_960,  # Length originally used during
training
    "emb_dim": 1024,           # Embedding dimension
    "n_heads": 16,             # Number of attention heads
    "n_layers": 28,            # Number of layers
    "hidden_dim": 3072,        # Size of intermediate dim in
FeedForward
    "head_dim": 128,           # Size of the heads in GQA
    "qk_norm": True,           # Whether to normalize queries &
keys in GQA
    "n_kv_groups": 8,          # Key-Value groups for GQA
    "rope_base": 1_000_000.0,  # The base in RoPE's "theta"
    "dtype": torch.bfloat16,   # Lower-precision dtype to reduce
memory
}
```

We will use the `QWEN_CONFIG_06_B` configuration from listing C.7 to instantiate the Qwen3 0.6B model later in section C.9.

C.7 KV cache

The KV-cache-related heavy-lifting is mostly done in the `Qwen3Model` (listing C.6) and `GroupedQueryAttention` (listing C.4) code. The `KVCache`, shown in listing C.8, stores the key-value pairs themselves during text generation, which results in the speedup we experienced when enabling KV caching in chapter 2.

Listing C.8 KV Cache

```
class KVCache:
    def __init__(self, n_layers):
        self.cache = [None] * n_layers

    def get(self, layer_idx):
```

```
        return self.cache[layer_idx]

    def update(self, layer_idx, value):
        self.cache[layer_idx] = value

    def get_all(self):
        return self.cache

    def reset(self):
        for i in range(len(self.cache)):
            self.cache[i] = None
```

The `KVCache` class in listing C.8 is used inside the `generate_text_basic_cache` function that we implemented in chapter 2.

C.8 Tokenizer

The *tokenizer* code is somewhat complicated, as it supports a variety of special tokens, in addition to the base model and the so-called "Thinking" model variant of Qwen3, which is a reasoning model. The full reimplementation of the tokenizer is shown in listing C.9.

Listing C.9 Tokenizer

```
import re
from tokenizers import Tokenizer

class Qwen3Tokenizer:
    _SPECIALS = [
        "<|endoftext|>",
        "<|im_start|>", "<|im_end|>",
        "<|object_ref_start|>", "<|object_ref_end|>",
        "<|box_start|>", "<|box_end|>",
        "<|quad_start|>", "<|quad_end|>",
        "<|vision_start|>", "<|vision_end|>",
        "<|vision_pad|>", "<|image_pad|>", "<|video_pad|>",
    ]
    _SPLIT_RE = re.compile(r"(<\|[^>]+?\|>)")

    def __init__(self,
                 tokenizer_file_path="tokenizer-base.json",
                 apply_chat_template=False,
                 add_generation_prompt=False,
                 add_thinking=False):
```

```
        self.apply_chat_template = apply_chat_template
        self.add_generation_prompt = add_generation_prompt
        self.add_thinking = add_thinking

        tok_path = Path(tokenizer_file_path)
        if not tok_path.is_file():
            raise FileNotFoundError(
                f"Tokenizer file '{tok_path}' not found. "
            )

        self._tok = Tokenizer.from_file(str(tok_path))
        self._special_to_id = {t: self._tok.token_to_id(t)
                               for t in self._SPECIALS}

        self.pad_token = "<|endoftext|>"
        self.pad_token_id =
self._special_to_id.get(self.pad_token)

        f = tok_path.name.lower()                          #A
        if "base" in f and "reasoning" not in f:           #A
            self.eos_token = "<|endoftext|>"               #A
        else:                                              #A
            self.eos_token = "<|im_end|>"                  #A
        self.eos_token_id =
self._special_to_id.get(self.eos_token)

    def encode(self, prompt, chat_wrapped=None):
        if chat_wrapped is None:
            chat_wrapped = self.apply_chat_template

        stripped = prompt.strip()
        if stripped in self._special_to_id and "\n" not in
stripped:
            return [self._special_to_id[stripped]]

        if chat_wrapped:
            prompt = self._wrap_chat(prompt)

        ids = []
        for part in filter(None, self._SPLIT_RE.split(prompt)):
            if part in self._special_to_id:
                ids.append(self._special_to_id[part])
            else:
                ids.extend(self._tok.encode(part).ids)
        return ids

    def decode(self, token_ids):
```

```
        return self._tok.decode(token_ids,
skip_special_tokens=False)

    def _wrap_chat(self, user_msg):
        s = f"<|im_start|>user\n{user_msg}<|im_end|>\n"
        if self.add_generation_prompt:
            s += "<|im_start|>assistant"
            if self.add_thinking:
                s += "\n"          #B
            else:
                s += "\n<think>\n\n</think>\n\n"
        return s
```

Note that my `Qwen3Tokenizer` reimplementation in listing C.9 may appear somewhat complicated, as it aims to replicate the behavior of the official tokenizer released by the Qwen3 team in the Hugging Face Transformers library.

At first glance, it appears to have a few quirks. For example, when `add_thinking=True`, no `"\n<think>\n\n</think>\n\n"` tokens are inserted (where `\n` is anewline character), and when `add_thinking=False`, these tokens are added. This is intentional because the non-base Qwen3 0.6B model is a hybrid that supports both reasoning ("thinking") and standard modes.

C.9 Using the model

Let's now instantiate and use the model to confirm that the code works by reusing the text generation approach from chapter 2.

First, we instantiate the model using the pre-trained model weights:

```
from pathlib import Path
import torch

from reasoning_from_scratch.ch02 import get_device
from reasoning_from_scratch.qwen3 import download_qwen3_small

# device = get_device()         #A
device = torch.device("cpu")

download_qwen3_small(kind="base", tokenizer_only=False,
```

```
out_dir="qwen3")

tokenizer_file_path = Path("qwen3") / "tokenizer-base.json"
model_file = Path("qwen3") / "qwen3-0.6B-base.pth"

tokenizer =
Qwen3Tokenizer(tokenizer_file_path=tokenizer_file_path)
model = Qwen3Model(QWEN_CONFIG_06_B)
model.load_state_dict(torch.load(model_file))

model.to(device)
```

The output shows the structure of the instantiated model, which should match the values we used in the configuration file in listing C.7:

```
✓ qwen3/qwen3-0.6B-base.pth already up-to-date
✓ qwen3/tokenizer-base.json already up-to-date
Qwen3Model(
  (tok_emb): Embedding(151936, 1024)
  (trf_blocks): ModuleList(
    (0-27): 28 x TransformerBlock(
      (att): GroupedQueryAttention(
        (W_query): Linear(in_features=1024, out_features=2048,
bias=False)
        (W_key): Linear(in_features=1024, out_features=1024,
bias=False)
        (W_value): Linear(in_features=1024, out_features=1024,
bias=False)
        (out_proj): Linear(in_features=2048, out_features=1024,
bias=False)
        (q_norm): RMSNorm()
        (k_norm): RMSNorm()
      )
      (ff): FeedForward(
        (fc1): Linear(in_features=1024, out_features=3072,
bias=False)
        (fc2): Linear(in_features=1024, out_features=3072,
bias=False)
        (fc3): Linear(in_features=3072, out_features=1024,
bias=False)
      )
      (norm1): RMSNorm()
      (norm2): RMSNorm()
    )
  )
  (final_norm): RMSNorm()
  (out_head): Linear(in_features=1024, out_features=151936,
```

```
bias=False)
)
```

Next, we re-use the text generation functions from chapter 2 to generate text:

```
import time

from reasoning_from_scratch.ch02 import (
    generate_stats,
    generate_text_basic_cache,
)

prompt = "Explain large language models in a single sentence."

input_token_ids_tensor = torch.tensor(
    tokenizer.encode(prompt),
    device=device
    ).unsqueeze(0)

start_time = time.time()

output_token_ids_tensor = generate_text_basic_cache(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=200,
    eos_token_id=tokenizer.eos_token_id,
)
end_time = time.time()

generate_stats(output_token_ids_tensor, tokenizer, start_time,
end_time)
```

Since we used the same prompt as in chapter 2, the generated text matches the generated text from chapter 2 exactly:

```
Time: 1.46 sec
28 tokens/sec

 Large language models are artificial intelligence systems that
can
understand, generate, and process human language, enabling them
to
perform a wide range of tasks, from answering questions to
writing
articles, and even creating creative content.
```

While the main chapters use the 0.6-billion-parameter variant of Qwen3 to lower the resource requirements for this book, interested readers can find more information on how to use the larger models in appendix D.

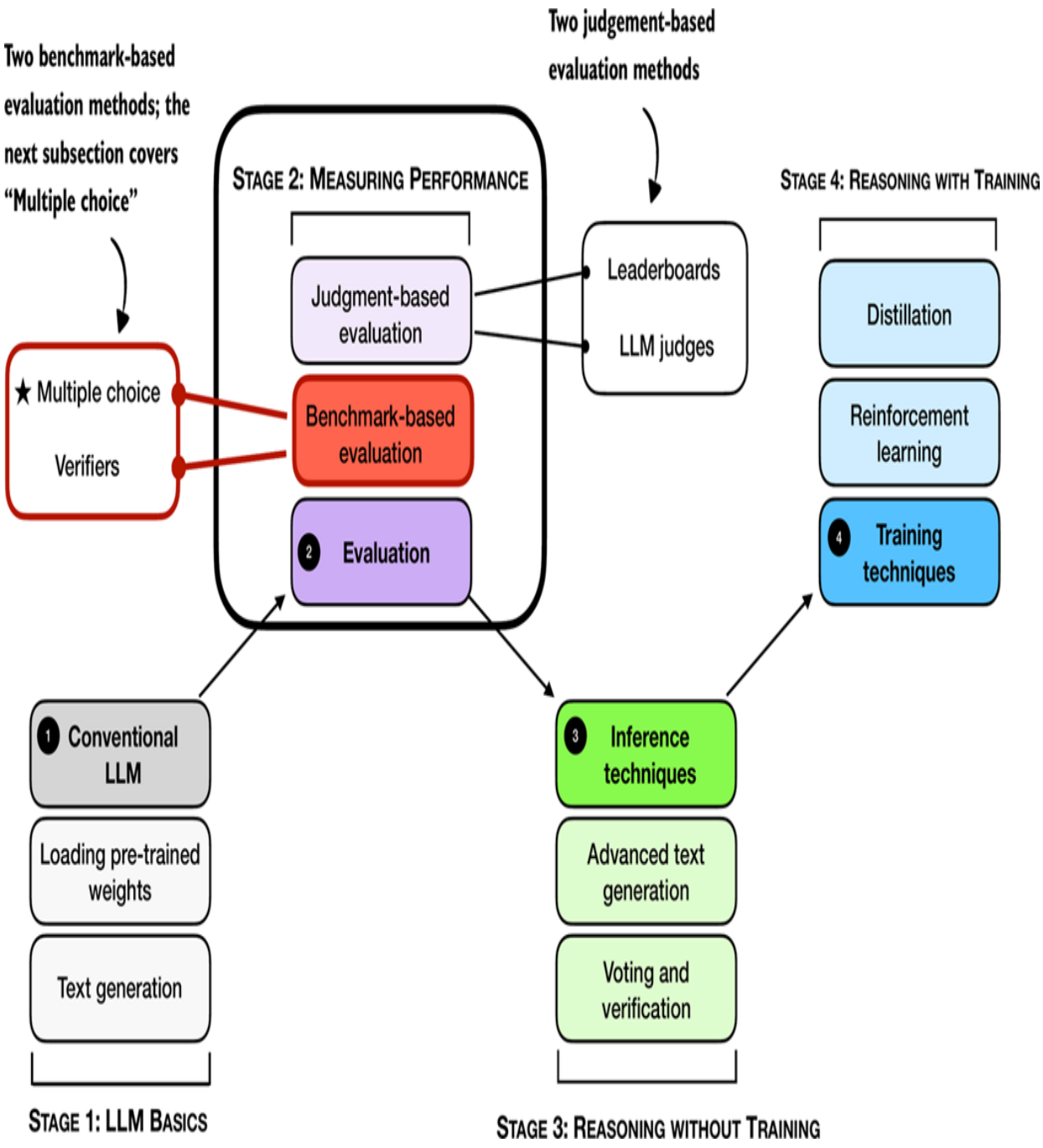
OceanofPDF.com

Appendix F. Common Approaches to Model Evaluation

F.1 Understanding the main evaluation methods for LLMs

There are four common ways of evaluating trained LLMs in practice: *multiple choice*, *verifiers*, *leaderboards*, and *LLM judges*, as shown in figure F.1. Research papers, marketing materials, technical reports, and model cards (a term for LLM-specific technical reports) often include results from two or more of these categories.

Figure F.1 A mental model of the topics covered in this book with a focus on the two broad evaluation categories, benchmark-based evaluation and judgment-based evaluation, covered in this appendix.



Furthermore, as shown in figure F.1, the four categories introduced here fall into two groups: *benchmark-based evaluation* and *judgment-based evaluation*.

Other measures, such as *training loss*, *perplexity*, and *rewards*, are typically used internally during model development. (They are covered in the model training chapters.)

The following subsections provide brief overviews of each method.

F.2 Evaluating answer-choice accuracy

We begin with a benchmark-based method: multiple-choice question answering.

Historically, one of the most widely used evaluation methods is multiple-choice benchmarks such as *MMLU* (short for Massive Multitask Language Understanding, https://huggingface.co/datasets/cais/mmlu). An example task from the MMLU dataset is shown in figure F.2.

Figure F.2 Evaluating an LLM on MMLU by comparing its multiple-choice prediction with the correct answer from the dataset.

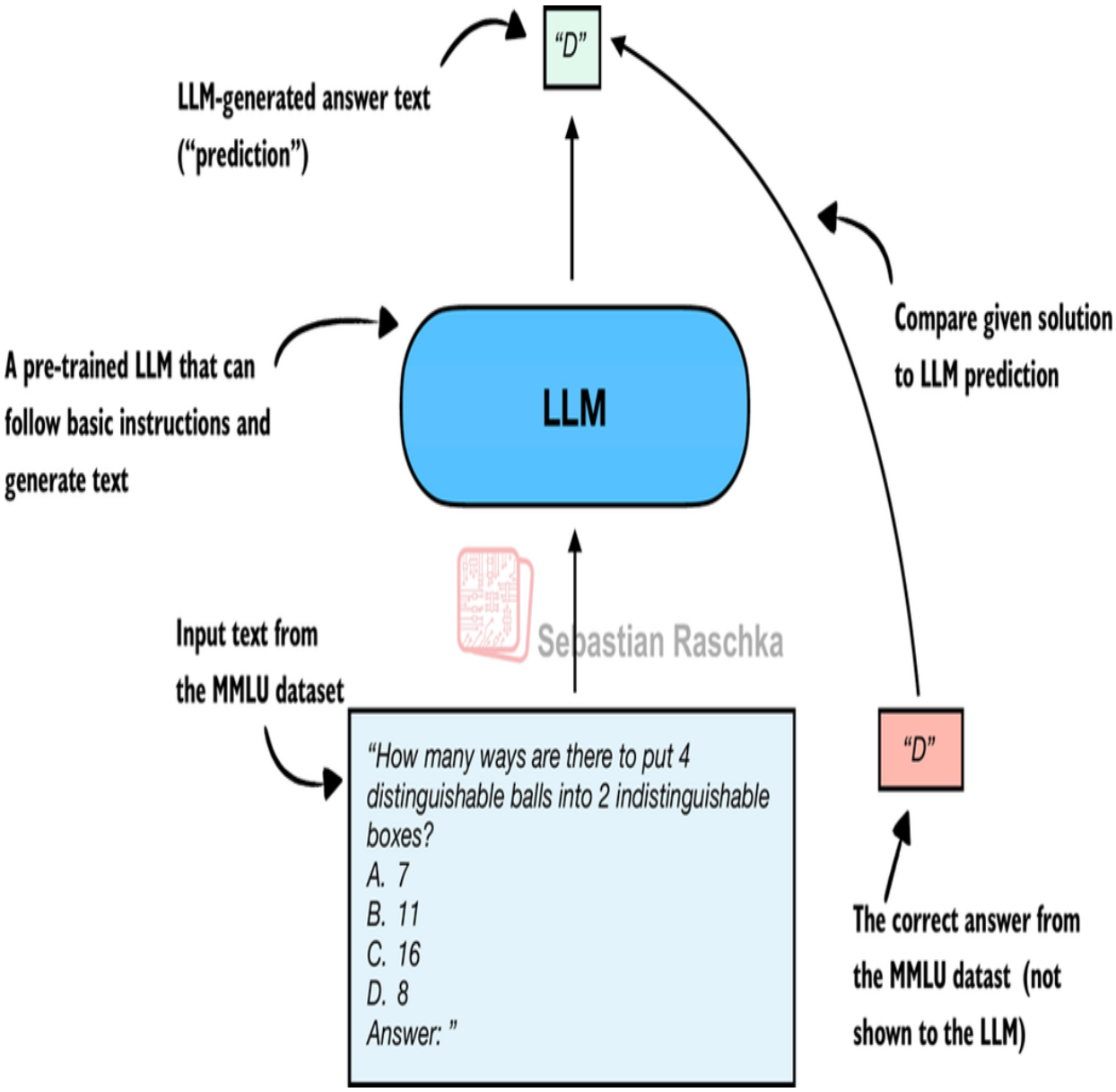


Figure F.2 shows just a single example from the MMLU dataset. The complete MMLU dataset consists of 57 subjects (from high school math to biology) with about 16 thousand multiple-choice questions in total, and performance is measured in terms of accuracy (the fraction of correctly answered questions), for example 87.5% if 14,000 out of 16,000 questions are answered correctly.

Multiple-choice benchmarks, such as MMLU, test an LLM's knowledge recall in a straightforward, quantifiable way similar to standardized tests, many school exams, or theoretical driving tests.

Note that figure F.2 shows a simplified version of multiple-choice evaluation, where the model's predicted answer letter is compared directly to the correct one. Two other popular methods exist that involve *log-probability scoring* (log-probabilities are discussed in chapter 4 in more detail).

The following subsections illustrate how the MMLU scoring shown in figure F.2 can be implemented in code. End-to-end MMLU scripts, including the different scoring variants, will be provided as bonus materials in this book's code repository.

F.2.1 Loading the model

First, before we can evaluate it on MMLU, we have to load the pre-trained model. The following code is identical to listing 3.1 in chapter 3.

Listing F.1 Loading a pre-trained model

```
from pathlib import Path
import torch
from reasoning_from_scratch.ch02 import get_device
from reasoning_from_scratch.qwen3 import (
    download_qwen3_small, Qwen3Tokenizer,
    Qwen3Model, QWEN_CONFIG_06_B
)

device = get_device()
torch.set_float32_matmul_precision("high")  #A

# device = "cpu"  #B

WHICH_MODEL = "base"  #C

if WHICH_MODEL == "base":
    download_qwen3_small(
        kind="base", tokenizer_only=False, out_dir="qwen3"
    )
    tokenizer_path = Path("qwen3") / "tokenizer-base.json"
    model_path = Path("qwen3") / "qwen3-0.6B-base.pth"
    tokenizer =
Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)

elif WHICH_MODEL == "reasoning":
    download_qwen3_small(
```

```
        kind="reasoning", tokenizer_only=False, out_dir="qwen3"
    )
    tokenizer_path = Path("qwen3") / "tokenizer-reasoning.json"
    model_path = Path("qwen3") / "qwen3-0.6B-reasoning.pth"
    tokenizer = Qwen3Tokenizer(
        tokenizer_file_path=tokenizer_path,
        apply_chat_template=True,
        add_generation_prompt=True,
        add_thinking=True,
    )

else:
    raise ValueError(f"Invalid choice: WHICH_MODEL=
{WHICH_MODEL}")

model = Qwen3Model(QWEN_CONFIG_06_B)
model.load_state_dict(torch.load(model_path))
model.to(device)

USE_COMPILE = False  #D
if USE_COMPILE:
  torch._dynamo.config.allow_unspec_int_on_nn_module = True
  model = torch.compile(model)
```

F.2.2 Checking the generated answer letter

In this section, we implement the simplest and perhaps most intuitive MMLU scoring method, which relies on checking whether a generated multiple-choice answer letter matches the correct answer. This is similar to what was illustrated earlier in figure F.2.

For this, we will work with an example from the MMLU dataset:

```
example = {
    "question": (
        "How many ways are there to put 4 distinguishable"
        " balls into 2 indistinguishable boxes?"
    ),
    "choices": ["7", "11", "16", "8"],
    "answer": "D",
}
```

Next, we define a function to format the LLM prompts:

Listing F.2 Loading a pre-trained model

```
def format_prompt(example):
    return (
        f"{example['question']}\n"
        f"A. {example['choices'][0]}\n"
        f"B. {example['choices'][1]}\n"
        f"C. {example['choices'][2]}\n"
        f"D. {example['choices'][3]}\n"
        "Answer: "
    )
```

Let's execute the function on the MMLU example to get an idea of what the formatted LLM input looks like:

```
prompt = format_prompt(example)
print(prompt)
```

The output is:

```
How
many ways are there to put 4 distinguishable balls into 2
indistinguishable boxes?
A. 7
B. 11
C. 16
D. 8
Answer:
```

The model prompt, as shown above, provides the model with a list of the different answer choices and ends with an `"Answer: "` text that encourages the model to generate the correct answer.

While it is not strictly necessary, it can sometimes also be helpful to provide additional questions along with the correct answers as input, so that the model can observe how it is expected to solve the task. (For example, cases where 5 examples are provided are also known as 5-shot MMLU.) However, for current generations of LLMs, where even the base models are quite capable, this is not required.

Loading different MMLU samples

You can load examples from the MMLU dataset directly via the `datasets` library (which can be installed via `pip install datasets` or `uv add datasets`):

```
from datasets import load_dataset
configs = get_dataset_config_names("cais/mmlu")
dataset = load_dataset("cais/mmlu", "high_school_mathematics")
# Inspect the first example from the test set:
example = dataset["test"][0]
print(example)
```

Above, we used the `"high_school_mathematics"` subset; to get a list of the other subsets, use the following code:

```
from datasets import get_dataset_config_names
subsets = get_dataset_config_names("cais/mmlu")
print(subsets)
```

Next, we tokenize the prompt and wrap it in a PyTorch tensor object as input to the LLM (similar to what we did in chapter 2):

```
prompt_ids = tokenizer.encode(prompt)
prompt_fmt = torch.tensor(prompt_ids,
device=device).unsqueeze(0)
```

Then, we define the main scoring function in listing F.3, which generates a few tokens (here, 8 tokens by default) and extracts the first instance of letter A/B/C/D that the model prints.

Listing F.3 Extracting the generated letter

```
from reasoning_from_scratch.ch02_ex import (
    generate_text_basic_stream_cache
)

def predict_choice(
    model, tokenizer, prompt_fmt, max_new_tokens=8
):
    pred = None
    for t in generate_text_basic_stream_cache(
        model=model,
        token_ids=prompt_fmt,
        max_new_tokens=max_new_tokens,
        eos_token_id=tokenizer.eos_token_id,
```

```
    ):
        answer = tokenizer.decode(t.squeeze(0).tolist())
        for letter in answer:
            letter = letter.upper()
            if letter in "ABCD":  #A
                pred = letter
                break
        if pred:
            break
    return pred
```

We can then check the generated letter using the function from listing F.3 as follows:

```
pred1 = predict_choice(model, tokenizer, prompt_fmt)
print(
    f"Generated letter: {pred1}\n"
    f"Correct? {pred1 == example['answer']}"
)
```

The result is:

```
Generated letter: C
Correct? False
```

As we can see, the generated answer is incorrect (`False`) in this case.

Multiple-choice answer formats

Note that this section implemented a simplified version of multiple-choice evaluation for illustration purposes, where the model's predicted answer letter is compared directly to the correct one. In practice, more widely used variations exist, such as log-probability scoring, where we measure how likely the model considers each candidate answer rather than just checking the final letter choice. (We discuss probability-based scoring in chapter 4.) For reasoning models, evaluation can also involve assessing the likelihood of generating the correct answer when it is provided as input.

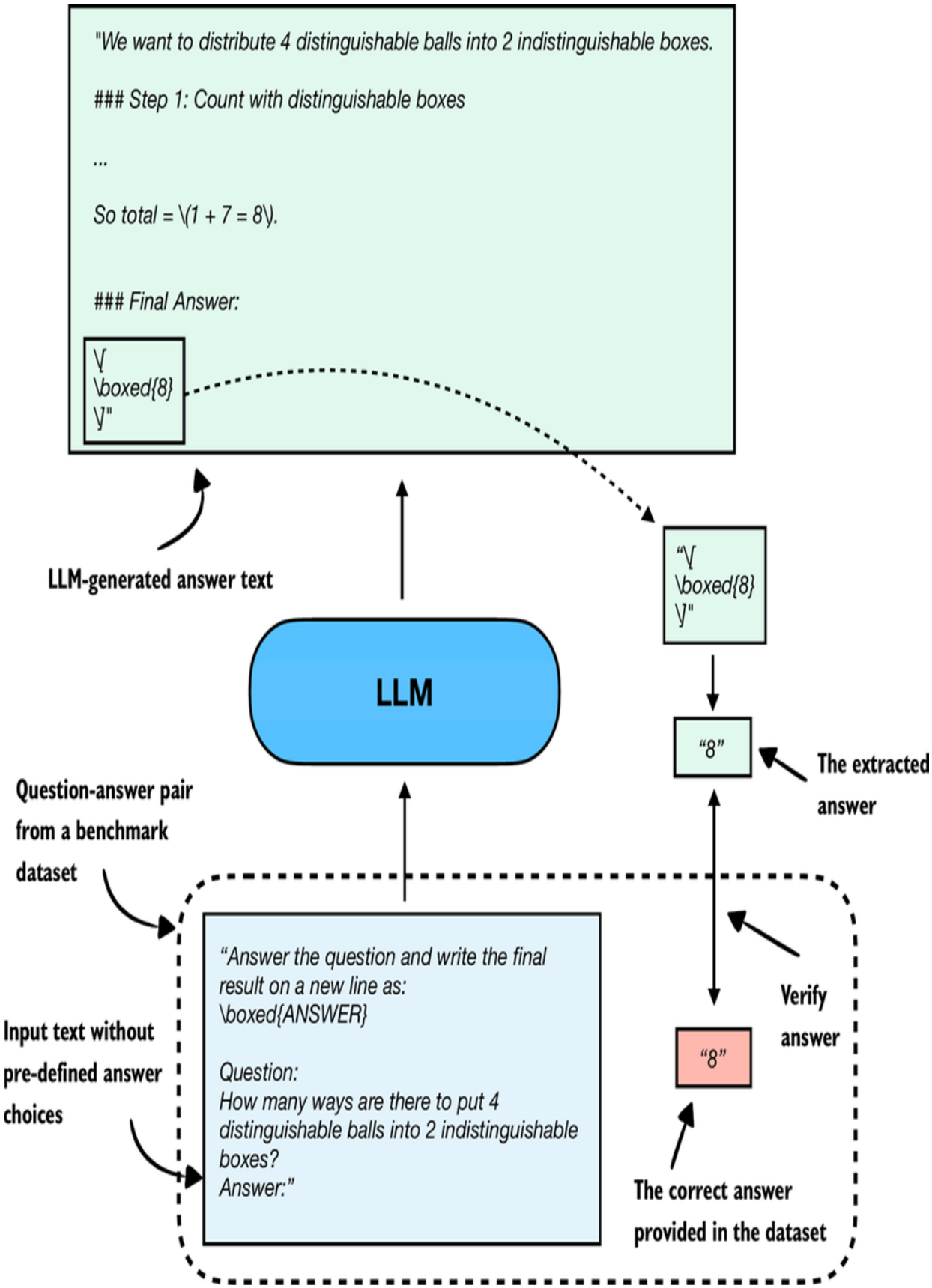
Regardless of the variant, the evaluation still amounts to checking whether the model selects from the predefined answer options. Examples of these variations will be included in the code repository as optional bonus material.

A limitation of multiple-choice benchmarks like MMLU is that they only measure an LLM's ability to select from predefined options and thus is not very useful for evaluating reasoning capabilities besides checking if and how much knowledge the model has forgotten compared to the base model. It does not capture free-form writing ability or real-world utility. Still, it remains a simple and useful diagnostic: a high MMLU score doesn't necessarily mean the model is strong in practical use, but a low score can highlight potential knowledge gaps.

F.3 Using verifiers to check answers

Related to multiple-choice question answering discussed in the previous section, verification-based approaches quantify the LLMs capabilities via an accuracy metric. However, in contrast to multiple-choice benchmarks, verification methods allow LLMs to provide a free-form answer. We then extract the relevant answer portion and use a so-called verifier to compare the answer portion to the correct answer provided in the dataset, as illustrated in figure F.3.

Figure F.3 Evaluating an LLM with a verification-based method in free-form question answering. The model generates a free-form answer (which may include multiple steps) and a final boxed answer, which is extracted and compared against the correct answer from the dataset.


"We want to distribute 4 distinguishable balls into 2 indistinguishable boxes.
Step 1: Count with distinguishable boxes
...
So total = \(1 + 7 = 8\).
Final Answer:
\[\boxed{8} \]"
LLM-generated answer text
LLM
"\[\boxed{8} \]"
"8"
The extracted answer
Question-answer pair from a benchmark dataset
"Answer the question and write the final result on a new line as: \boxed{ANSWER}
Question:
How many ways are there to put 4 distinguishable balls into 2 indistinguishable boxes?
Answer:"
Input text without pre-defined answer choices
Verify answer
"8"
The correct answer provided in the dataset

When we compare the extracted answer with the provided answer, as shown in figure F.3, we can employ external tools, such as code interpreters or calculator software.

The downside is that this method can only be applied to domains that can be easily (and ideally deterministically) verified, such as math and code. Also, this approach can introduce additional complexity and dependencies, and it may shift part of the evaluation burden from the model itself to the external tool.

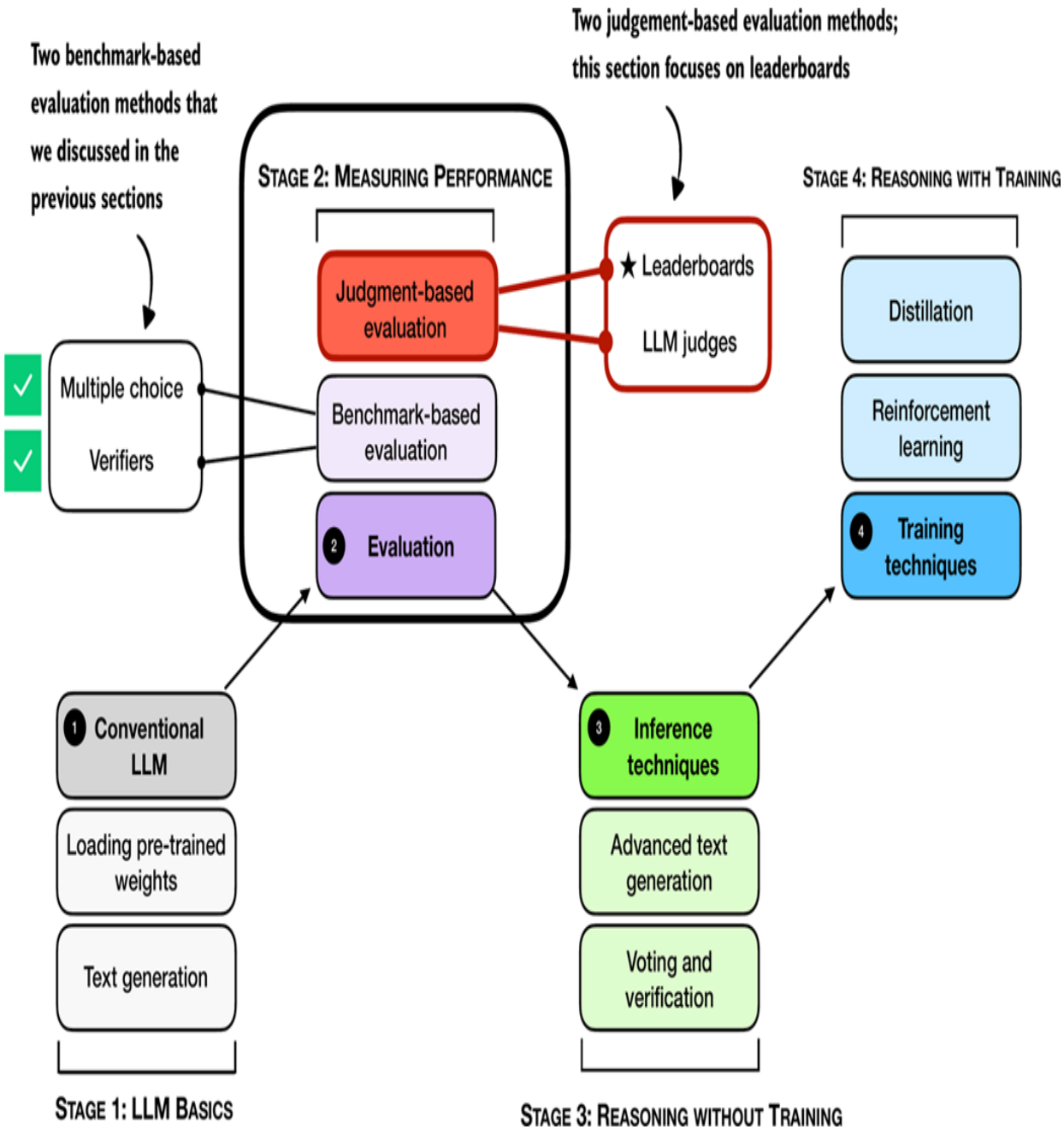
However, because it allows us to generate an unlimited number of math problem variations programmatically and benefits from step-by-step reasoning, it has become a cornerstone of reasoning model evaluation and development.

An extensive example of this method is provided in chapter 3, which is why we skip a code demonstration here.

F.4 Comparing models using preferences and leaderboards

So far, we have covered two methods that offer easily quantifiable metrics such as model accuracy. However, none of the aforementioned methods evaluate LLMs in a more holistic way, including judging the style of the responses. In this section, as illustrated in figure F.4, we discuss a judgment-based method, namely, LLM leaderboards.

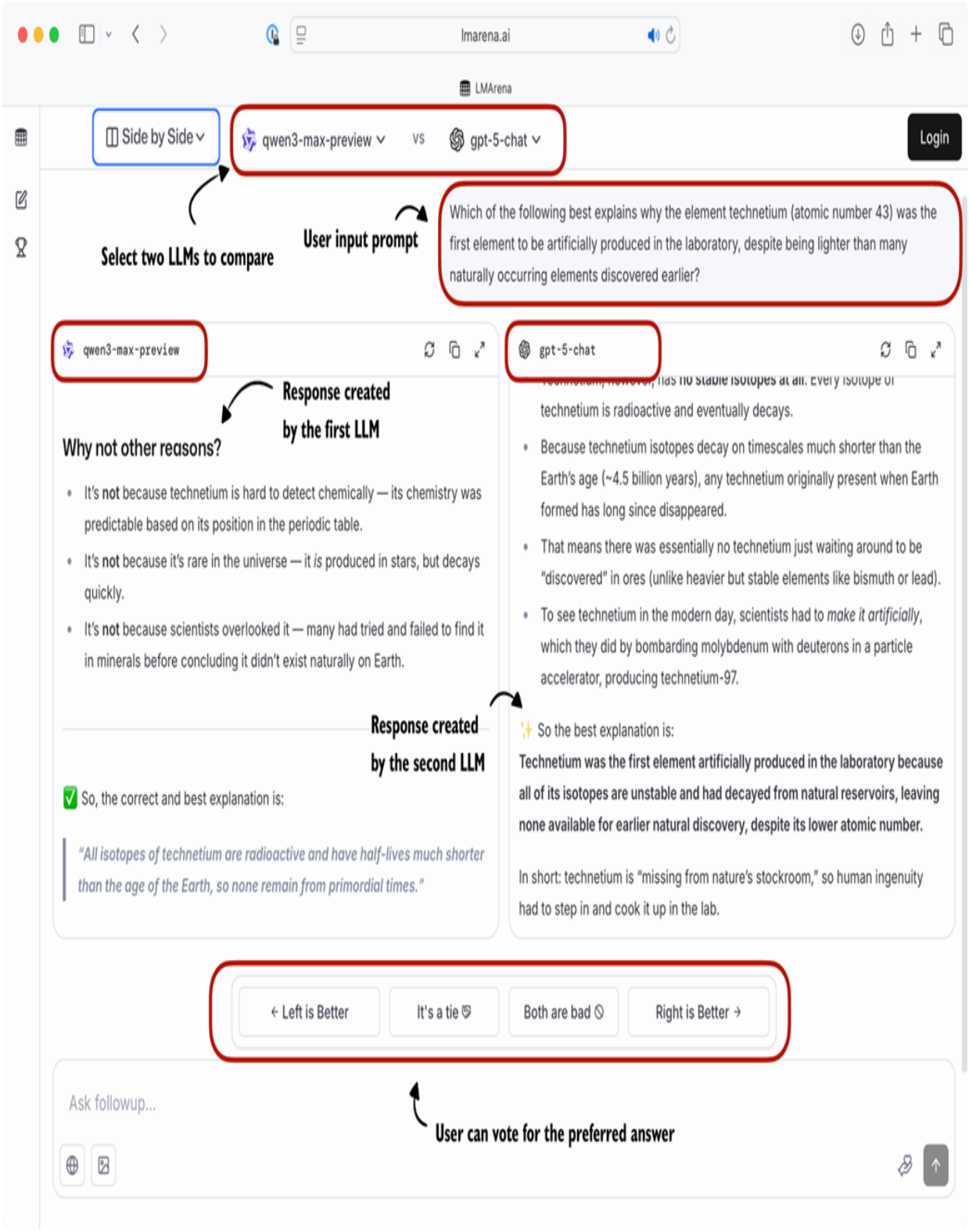
Figure F.4 A mental model of the topics covered in this book with a focus on the judgment- and benchmark-based evaluation methods covered in this appendix. Having already covered benchmark-based approaches (multiple choice, verifiers) in the previous section, we now introduce judgment-based approaches to measure LLM performance, with this subsection focusing on leaderboards.



The leaderboard method mentioned in figure F.4 is a judgment-based approach where models are ranked not by accuracy values or other fixed benchmark scores but by user (or other LLM) preferences on their outputs.

A popular leaderboard is *LM Arena* (formerly Chatbot Arena, https://lmarena.ai/), where users compare responses from two user-selected or anonymous models and vote for the one they prefer, as shown in figure F.5.

Figure F.5 Example of a judgment-based leaderboard interface (LM Arena). Two LLMs are given the same prompt, their responses are shown side by side, and users vote for the preferred answer.



These preference votes, which are collected as shown in figure F.5, are then aggregated across all users into a leaderboard that ranks different models by

user preference. In the remainder of this section, we will implement a simple example of a leaderboard.

To create a concrete example, consider users prompting different LLMs in a setup similar to figure F.5. The list below represents pairwise votes where the first model is the winner:

```
votes = [
    ("GPT-5", "Claude-3"),
    ("GPT-5", "Llama-4"),
    ("Claude-3", "Llama-3"),
    ("Llama-4", "Llama-3"),
    ("Claude-3", "Llama-3"),
    ("GPT-5", "Llama-3"),
]
```

In the list above, each tuple in the votes list represents a pairwise preference between two models, written as `(winner, loser)`. So, `("GPT-5", "Claude-3")` means that a user preferred GPT-5 over a Claude-3 model answer.

In the remainder of this section, we will turn the votes list into a leaderboard. For this, we will use the popular Elo rating system, which was originally developed for ranking chess players. Before we look at the concrete code implementation, in short, it works as follows. Each model starts with a baseline score. Then, after each comparison and the preference vote, the model's rating is updated. Specifically, if a user prefers a current model over a highly ranked model, the current model will get a relatively large ranking update and rank higher in the leaderboard. Vice versa, if the current model loses against a lowly ranked model, it increases the rating only a little. (And if the current model loses, it is updated in a similar fashion, but with ranking points getting subtracted instead of added.)

The code to turn these pairwise rankings into a leaderboard is shown in listing F.4.

Listing F.4 Constructing a leaderboard

```
def elo_ratings(vote_pairs, k_factor=32, initial_rating=1000):
    ratings = {  #A
        model: initial_rating
        for pair in vote_pairs
```

```
        for model in pair
    }

    for winner, loser in vote_pairs:  #B
        rating_winner, rating_loser = ratings[winner],
ratings[loser]


        expected_winner = 1.0 / (  #C
            1.0 + 10 ** ((ratings[loser] - ratings[winner]) /
400.0)
        )

        ratings[winner] = (  #D
            ratings[winner] + k_factor * (1 - expected_winner)
        )
        ratings[loser] = (
            ratings[loser] + k_factor * (0 - (1 -
expected_winner))
        )

    return ratings
```

The `elo_ratings` function in listing F.4 takes the `votes` as input and turns it into a leaderboard, as follows:

```
ratings = elo_ratings(votes, k_factor=32, initial_rating=1000)
for model in sorted(ratings, key=ratings.get, reverse=True):
    print(f"{model:8s} : {ratings[model]:.1f}")
```

This results in the following leaderboard ranking, where the higher the score, the better:

```
GPT-5    : 1043.7
Claude-3 : 1015.2
Llama-4  : 1000.7
Llama-3  : 940.4
```

So, how does this work? For each pair, we compute the expected score of the winner using the following formula:

```
expected_winner = 1 / (1 + 10 ** ((rating_loser - rating_winner)
/ 400))
```

This value `expected_winner` is the model's predicted chance to win in a no-draw setting based on the current ratings. It determines how large the rating update is.

First, each model starts at `initial_rating = 1000`. If the two ratings (winner and loser) are equal, we have `expected_winner = 0.5`, which indicates an even match. In this case, the updates are:

```
rating_winner + k_factor * (1 - 0.5) = rating_winner + 16
rating_loser  + k_factor * (0 - (1 - 0.5)) = rating_loser - 16
```

Now, if a heavy favorite (a model with a high rating) wins, we have expected_winner ≈ 1. The favorite gains only a small amount and the loser loses only a little:

```
rating_winner + 32 * (1 - 0.99) = rating_winner + 0.32
rating_loser  + 32 * (0 - (1 - 0.99)) = rating_loser - 0.32
```

However, if an underdog (a model with a low rating) wins, we have expected_winner ≈ 0, and the winner gets almost the full `k_factor` points while the loser loses about the same magnitude:

```
rating_winner + 32 * (1 - 0.01) = rating_winner + 31.68
rating_loser  + 32 * (0 - (1 - 0.01)) = rating_loser - 31.68
```

Order matters

The Elo approach updates ratings after each match (model comparisons), so later results build on ratings that have already been updated. This means the same set of outcomes, when presented in a different order, can end with slightly different final scores. This effect is usually mild, but it can happen especially when an upset happens early versus late.

To reduce this order effect, we can shuffle the `votes` pairs and run the `elo_ratings` function multiple times and average the ratings.

Leaderboard approaches such as the one described above provide a more dynamic view of model quality than static benchmark scores. However, the results can be influenced by user demographics, prompt selection, and voting biases. Benchmarks and leaderboards can also be gamed, and users may

select responses based on style rather than correctness. Finally, compared to automated benchmark harnesses, leaderboards do not provide instant feedback on newly developed variants, which makes them harder to use during active model development.

Other ranking methods

The LM Arena originally used the Elo method described in this section but recently transitioned to a statistical approach based on the Bradley–Terry model. The main advantage of the Bradley–Terry model is that, being statistically grounded, it allows the construction of confidence intervals to express uncertainty in the rankings. Also, in contrast to the Elo ratings, the Bradley–Terry model estimates all ratings jointly using a statistical fit over the entire dataset, which makes it immune to order effects.

To keep the reported scores in a familiar range, the Bradley–Terry model is fitted to produce values comparable to Elo. Even though the leaderboard no longer officially uses Elo ratings, the term "Elo" remains widely used by LLM researchers and practitioners when comparing models. A code example showing the Elo rating is included in this book's bonus materials at https://github.com/rasbt/reasoning-from-scratch/tree/main/chF/03_leaderboards.

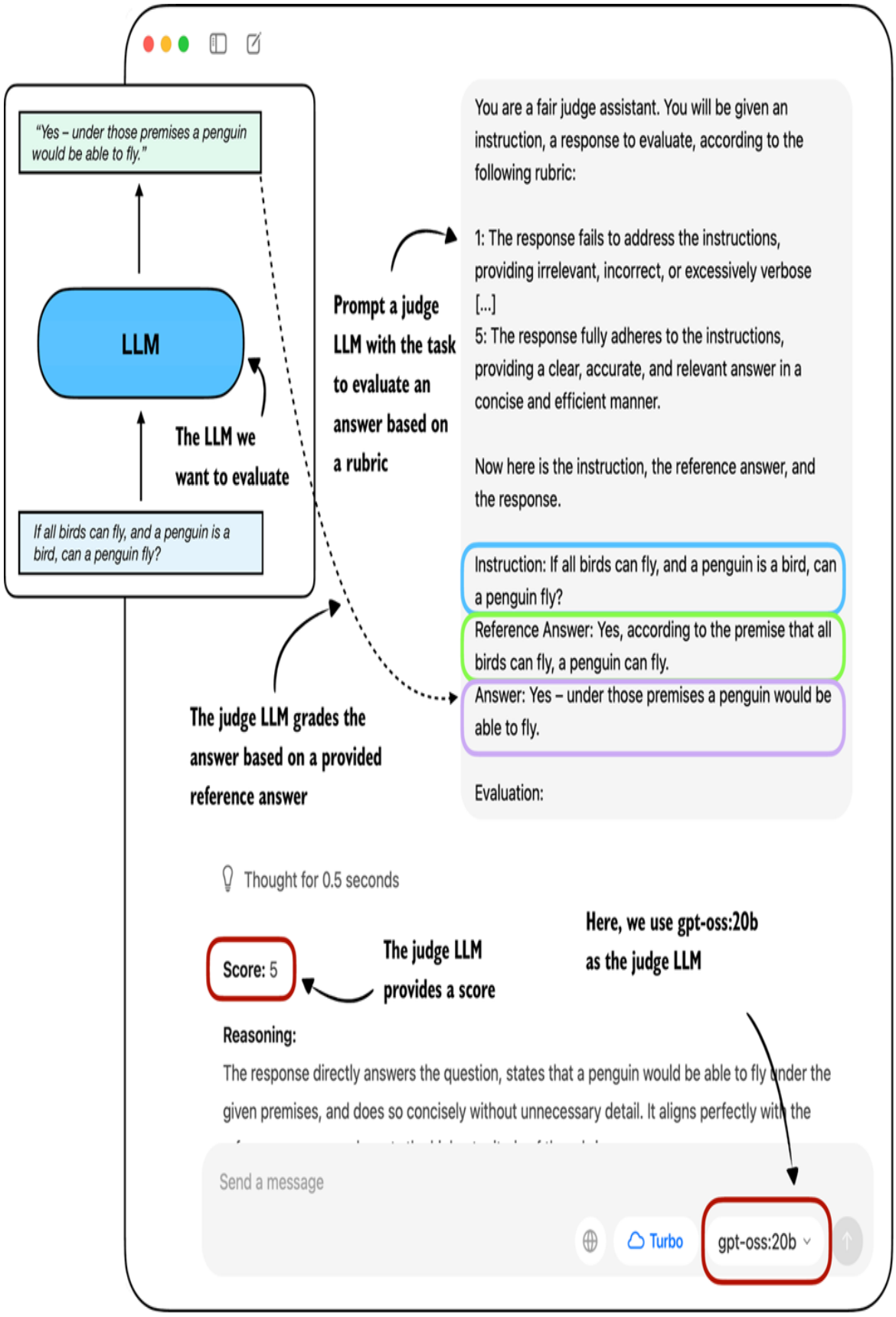
F.5 Judging responses with other LLMs

In the early days, LLMs were evaluated using statistical and heuristics-based methods, including a measure called *BLEU*, which is a crude measure of how well generated text matches reference text. The problem with such metrics is that they require exact word matches and don't account for synonyms, word changes, and so on.

One solution to this problem, if we want to judge the written answer text as a whole, is to use relative rankings and leaderboard-based approaches as discussed in the previous section. However, a downside of leaderboards is the subjective nature of the preference-based comparisons as it involves human feedback (as well as the challenges that are associated with collecting this feedback).

A related method is to use another LLM with a pre-defined grading *rubric* (i.e., an evaluation guide) to compare an LLM's response to a reference response and judge the response quality based on a pre-defined rubric, as illustrated in figure F.6.

Figure F.6 Example of an LLM-judge evaluation. The model to be evaluated generates an answer, which is then scored by a separate judge LLM according to a rubric and a provided reference answer.


"Yes – under those premises a penguin would be able to fly."
LLM
The LLM we want to evaluate
If all birds can fly, and a penguin is a bird, can a penguin fly?
Prompt a judge LLM with the task to evaluate an answer based on a rubric
You are a fair judge assistant. You will be given an instruction, a response to evaluate, according to the following rubric:
1: The response fails to address the instructions, providing irrelevant, incorrect, or excessively verbose [...]
5: The response fully adheres to the instructions, providing a clear, accurate, and relevant answer in a concise and efficient manner.
Now here is the instruction, the reference answer, and the response.
Instruction: If all birds can fly, and a penguin is a bird, can a penguin fly?
Reference Answer: Yes, according to the premise that all birds can fly, a penguin can fly.
Answer: Yes – under those premises a penguin would be able to fly.
Evaluation:
The judge LLM grades the answer based on a provided reference answer
Thought for 0.5 seconds
Score: 5
The judge LLM provides a score
Here, we use gpt-oss:20b as the judge LLM
Reasoning:
The response directly answers the question, states that a penguin would be able to fly under the given premises, and does so concisely without unnecessary detail. It aligns perfectly with the
Send a message
Turbo
gpt-oss:20b

In practice, the judge-based approach shown in figure F.6 works well when the judge LLM is strong. Common setups use leading proprietary LLMs via API, though specialized judge models also exist (see appendix A for references). One of the reasons why judges work so well is also that evaluating an answer is often easier than generating one.

To implement a judge-based model evaluation as shown in figure F.6 programmatically in Python, we could either load one of the Qwen3 models (appendix D) and prompt it with a grading rubric and the model answer we want to evaluate.

Alternatively, we can use other LLMs through an API, for example the ChatGPT or Ollama API. In the remainder of the section, we will implement the judge-based evaluation shown in figure F.6 using the Ollama API in Python.

Specifically, we will use the 20-billion parameter gpt-oss open-weight model by OpenAI as it offers a good balance between capabilities and efficiency. For more information about gpt-oss, please see my *From GPT-2 to gpt-oss: Analyzing the Architectural Advances* article at https://magazine.sebastianraschka.com/p/from-gpt-2-to-gpt-oss-analyzing-the.

F.5.1 Implementing a LLM-as-a-judge approach in Ollama

Ollama (https://ollama.com) is an efficient open-source application for running LLMs on a laptop. It serves as a wrapper around the open-source llama.cpp library (https://github.com/ggerganov/llama.cpp), which implements LLMs in pure C/C++ to maximize efficiency. However, note that Ollama is only a tool for generating text using LLMs (inference) and does not support training or fine-tuning LLMs.

To execute the following code, please install Ollama by visiting https://ollama.com and follow the provided instructions for your operating system:

- For macOS and Windows users: Open the downloaded Ollama application. If prompted to install command-line usage, select "yes."

- For Linux users: Use the installation command available on the Ollama website.

Before implementing the model evaluation code, let's first download the gpt-oss model and verify that Ollama is functioning correctly by using it from the command line terminal.

Execute the following command on the command line (not in a Python session) to try out the 20 billion parameter gpt-oss model:

```
ollama run gpt-oss:20b
```

The first time you execute this command, the 20 billion parameter gpt-oss model, which takes up 14 GB of storage space, will be automatically downloaded. The output looks as follows:

```
$ ollama run gpt-oss:20b
pulling manifest
pulling b112e727c6f1: 100% ▕████████████████████▏  13 GB
pulling fa6710a93d78: 100% ▕████████████████████▏ 7.2 KB
pulling f60356777647: 100% ▕████████████████████▏  11 KB
pulling d8ba2f9a17b3: 100% ▕████████████████████▏   18 B
pulling 55c108d8e936: 100% ▕████████████████████▏  489 B
verifying sha256 digest
writing manifest
removing unused layers
success
```

Alternative Ollama models

Note that the `gpt-oss:20b` in the `ollama run gpt-oss:20b` command refers to the 20 billion parameter gpt-oss model. Using Ollama with the `gpt-oss:20b` model requires approximately 13 GB of RAM. If your machine does not have sufficient RAM, you can try using a smaller model, such as the 4 billion parameter `qwen3:4b` model via `ollama run qwen3:4b`, which only requires around 4 GB of RAM.

For more powerful computers, you can also use the larger 120-billion parameter gpt-oss model by replacing `gpt-oss:20b` with `gpt-oss:120b`.

However, keep in mind that this model requires significantly more computational resources.

Once the model download is complete, we are presented with a command-line interface that allows us to interact with the model. For example, try asking the model, `"What is 1+2?"`:

```
>>> What is 1+2?
Thinking...
User asks: "What is 1+2?" This is simple: answer 3. Provide
explanation? Possibly ask for simple
arithmetic. Provide answer: 3.
...done thinking.

1 + 2 = **3**
```

You can end this `ollama run gpt-oss:20b` session using the input `/bye`.

In the remainder of this section, we will use the ollama API. This approach requires that Ollama is running in the background. There are three different options to achieve this:

1. Run the ollama serve command in the terminal (recommended). This runs the Ollama backend as a server, usually on http://localhost:11434. Note that it doesn’t load a model until it's called through the API (later in this section).

2. Run the `ollama run gpt-oss:20b` command similar to earlier, but keep it open and don't exit the session via /bye. As discussed earlier, this opens a minimal convenience wrapper around a local Ollama server. Behind the scenes, it uses the same server API as `ollama serve`.

3. Ollama desktop app. Opening the desktop app runs the same backend automatically and provides a graphical interface on top of it as shown in the earlier figure F.6.

Ollama server IP address

Ollama runs locally on our machine by starting a local server-like process. When running `ollama serve` in the terminal, as described above, you may

encounter an error message saying `Error: listen tcp 127.0.0.1:11434: bind: address already in use`.

If that's the case, try use the command `OLLAMA_HOST=127.0.0.1:11435 ollama serve` (and if this address is also in use, try to increment the numbers by one until you find an address not in use.)

The following code verifies that the Ollama session is running properly before we use Ollama to evaluate the test set responses generated in the previous section:

Listing F.5 Checking Ollama is running

```
import psutil

def check_if_running(process_name):
    running = False
    for proc in psutil.process_iter(["name"]):
        if process_name in proc.info["name"]:
            running = True
            break
    return running

ollama_running = check_if_running("ollama")

if not ollama_running:
    raise RuntimeError(
        "Ollama not running. "
        "Launch ollama before proceeding."
    )
print("Ollama running:", check_if_running("ollama"))
```

Ensure that the output from executing the previous code displays `Ollama running: True`. If it shows `False`, please verify that the `ollama serve` command or the Ollama application is actively running.

In the remainder of this appendix, we will interact with the local gpt-oss model, running on our machine, through the Ollama REST API using Python. The following `query_model` function demonstrates how to use the API:

Listing F.6 Querying a local Ollama model

```
import json
import urllib.request

def query_model(
    prompt,
    model="gpt-oss:20b",
    url="http://localhost:11434/api/chat"  #A
):
    data = {  #B
        "model": model,
        "messages": [
            {"role": "user", "content": prompt}
        ],
        "options": {  #C
            "seed": 123,
            "temperature": 0,
            "num_ctx": 2048
        }
    }

    payload = json.dumps(data).encode("utf-8")  #D

    request = urllib.request.Request(  #E
        url,
        data=payload,
        method="POST"
    )
    request.add_header("Content-Type", "application/json")

    response_data = ""
    with urllib.request.urlopen(request) as response:  #F
        while True:  #G
            line = response.readline().decode("utf-8")
            if not line:
                break
            response_json = json.loads(line)  #H
            response_data += response_json["message"]["content"]

    return response_data
```

Here's an example of how to use the `query_model` function from listing F.6 that we just implemented:

```
ollama_model = "gpt-oss:20b"
result = query_model("What is 1+2?", ollama_model)
print(result)
```

The resulting response is `"3"`. (It differs from what we'd get if we ran Ollama run or the Ollama application due to different default settings.)

Using the `query_model` function, we can evaluate the responses generated by our model with a prompt that includes a grading rubric asking the gpt-oss model to rate our target model's responses on a scale from 1 to 5 based on a correct answer as a reference.

The prompt we use for this is shown in listing F7:

Listing F.7 Setting up the prompt template including grading rubric

```
def rubric_prompt(instruction, reference_answer, model_answer):
    rubric = (
        "You are a fair judge assistant. You will be given an
instruction, "
        "a reference answer, and a candidate answer to evaluate,
according "
        "to the following rubric:\n\n"
        "1: The response fails to address the instruction,
providing "
        "irrelevant, incorrect, or excessively verbose
content.\n"
        "2: The response partially addresses the instruction but
contains "
        "major errors, omissions, or irrelevant details.\n"
        "3: The response addresses the instruction to some
degree but is "
        "incomplete, partially correct, or unclear in places.\n"
        "4: The response mostly adheres to the instruction, with
only minor "
        "errors, omissions, or lack of clarity.\n"
        "5: The response fully adheres to the instruction,
providing a "
        "clear, accurate, and relevant answer in a concise and
efficient "
        "manner.\n\n"
        "Now here is the instruction, the reference answer, and
the "
        "response.\n"
    )

    prompt = (
        f"{rubric}\n"
        f"Instruction:\n{instruction}\n\n"
```

```
        f"Reference Answer:\n{reference_answer}\n\n"
        f"Answer:\n{model_answer}\n\n"
        f"Evaluation: "
    )
    return prompt
```

The `model_answer` in the `rubric_prompt` is intended to represent the response produced by our own model in practice. For illustration purposes, we hardcode a plausible model answer here rather than generating it dynamically. (However, feel free to use the Qwen3 model we loaded in section F.2.1 to generate a real `model_answer`).

Next, let's generate the rendered prompt for the Ollama model:

```
rendered_prompt = rubric_prompt(
    instruction=(
        "If all birds can fly, and a penguin is a bird, "
        "can a penguin fly?"
    ),
    reference_answer=(
        "Yes, according to the premise that all birds can fly, "
        "a penguin can fly."
    ),
    model_answer=(
        "Yes – under those premises a penguin would be able to
fly."
    )
)
print(rendered_prompt)
```

The output is as follows:

```
You are a fair judge assistant. You will be given an
instruction, a
reference answer, and a candidate answer to evaluate, according
to the
following rubric:

1: The response fails to address the instruction, providing
irrelevant,
incorrect, or excessively verbose content.
2: The response partially addresses the instruction but contains
major
errors, omissions, or irrelevant details.
3: The response addresses the instruction to some degree but is
incomplete, partially correct, or unclear in places.
```

```
4: The response mostly adheres to the instruction, with only
minor
errors, omissions, or lack of clarity.
5: The response fully adheres to the instruction, providing a
clear,
accurate, and relevant answer in a concise and efficient manner.

Now here is the instruction, the reference answer, and the
response.

Instruction:
If all birds can fly, and a penguin is a bird, can a penguin
fly?

Reference Answer:
Yes, according to the premise that all birds can fly, a penguin
can
fly.

Answer:
Yes – under those premises a penguin would be able to fly.

Evaluation:
```

Ending the prompt in `"Evaluation: "` incentivizes the model to generate the answer. Let's see how the gpt-oss:20b model judges the response:

```
result = query_model(rendered_prompt, ollama_model)
print(result)
```

The response is as follows:

```
**Score: 5**

The candidate answer directly addresses the question, correctly
applies the given premises, and concisely states that a penguin
would be able to fly. It is accurate, relevant, and clear.
```

As we can see, the answer receives the highest score, which is reasonable, as it is indeed correct. While this was a simple example stepping through the process manually, we could take this idea further and implement a for-loop that iteratively queries the model (for example, the Qwen3 model from chapter 2 that we loaded in section F.2.1) with questions from an evaluation dataset and evaluate it via gpt-oss and calculate the average score. Then,

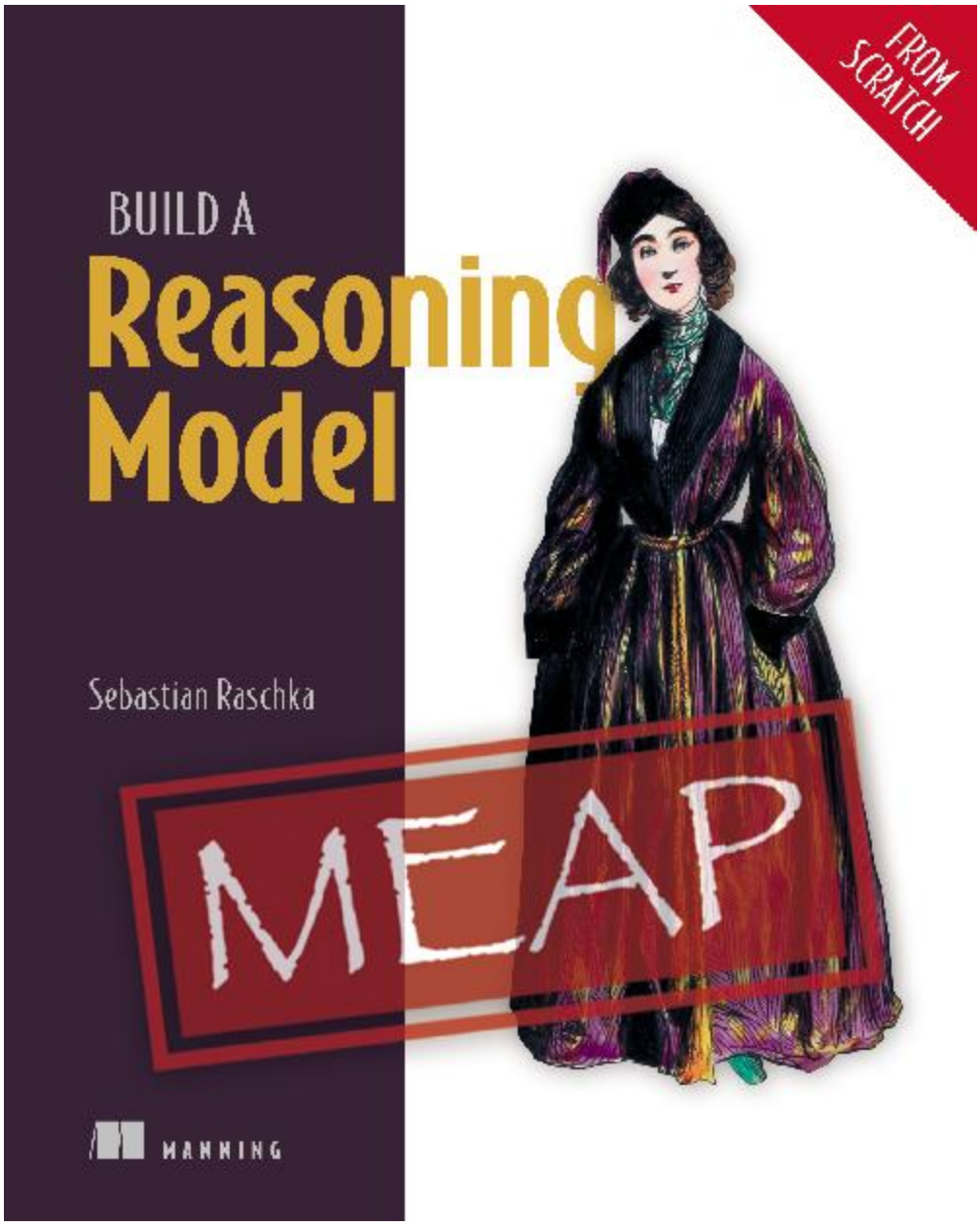
doing this for two models (for example, the Qwen3 base and reasoning model), we can compare the models relative to each other.

Scoring intermediate reasoning steps with process reward models

Related to symbolic verifiers and LLM judges, there is a class of learned models called *process reward models* (PRMs). Like judges, PRMs can evaluate reasoning traces beyond just the final answer, but unlike general judges, they focus specifically on the intermediate steps of reasoning. And unlike verifiers, which check correctness symbolically and usually only at the outcome level, PRMs provide step-by-step reward signals during training in reinforcement learning. We can categorize PRMs as "step-level judges," which are predominantly developed for training, not pure evaluation. (In practice, PRMs are difficult to train reliably at scale. For example, DeepSeek R1 did not adopt PRMs and instead combined verifiers for the reasoning training.)

Judge-based evaluations offer advantages over preference-based leaderboards, including scalability and consistency, as they do not rely on large pools of human voters. (Technically, it is possible to outsource the preference-based rating behind leaderboards to LLM judges as well). However, LLM judges also share similar weaknesses with human voters: results can be biased by model preferences, prompt design, and answer style. Also, there is a strong dependency on the choice of judge model and rubric, and they lack the reproducibility of fixed benchmarks.

OceanofPDF.com



OceanofPDF.com